

ระบบเว็บแอปพลิเคชันจัดการโฆษณาและเครื่องมือสำหรับโปรแกรม Unity

Free Mason

นาย นโรดม ญาตินิมิตร

ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิทยาศาสตรบัณฑิต

ภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ

คณะวิทยาศาสตร์ประยุกต์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ

ปีการศึกษา 2565

ปริญญานิพนธ์เรื่อง	:	ระบบเว็บแอปพลิเคชันจัดการโฆษณาและเครื่องมือสำหรับโปรแกรม Unity (Free Mason)
โดย	:	นายนโรดม ญาตินิมิตร
สาขาวิชา	:	วิทยาการคอมพิวเตอร์
ภาควิชา	:	วิทยาการคอมพิวเตอร์และสารสนเทศ
คณะ	:	วิทยาศาสตร์ประยุกต์
อาจารย์ที่ปรึกษา	:	ดร.ณัฐกิตติ์ จิตรเอื้อตระกูล
ปีการศึกษา	:	2565

คณะวิทยาศาสตร์ประยุกต์มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ อนุมัติให้ปริญญานิพนธ์นี้เป็นส่วนหนึ่งของการศึกษาตามหลักสูตรปริญญาวิทยาศาสตรบัณฑิต สาขาวิชาวิทยาการ-คอมพิวเตอร์

.....ประธานกรรมการ
(อาจารย์)
.....กรรมการ
(อาจารย์)
.....กรรมการ
(อาจารย์)

ลิขสิทธิ์ของภาควิชาวิทยาการคอมพิวเตอร์และสารสนเทศ คณะวิทยาศาสตร์ประยุกต์
มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ
ปีการศึกษา 2565

บทคัดย่อ

ปัจจุบันการโฆษณาสินค้าและบริการบนสื่อบันเทิงอย่างเกมมีบทบาทและอิทธิพลต่อความคิดผู้ใช้งาน โดยธุรกิจต่างปรับตัวและพยายามสร้างความสัมพันธ์ในรูปแบบวิธีการต่างๆให้กับผู้ใช้งานได้รับรู้ตัวสินค้าและบริการ

ซึ่งโครงการนี้จะกล่าวถึงรูปแบบการโฆษณาบนสื่อบันเทิงอย่างเกม โดยการใช้รูปแบบ “โฆษณาแฝง(Product placement)” คือ การให้มีวัตถุที่เป็นส่วนหนึ่งของฉากในเกม สามารถแสดงผลโฆษณาแบบวิดีโอผ่านวัตถุเหล่านั้นได้ ซึ่งรูปแบบวิธีการดังกล่าวจำเป็นต้องมาพร้อมกับการพัฒนาเกม(Game development) โดยวิธีการที่นำวัตถุเข้าไปเป็นส่วนหนึ่งของฉากในเกมที่กำลังถูกพัฒนา จึงอาจเกิดปัญหาในการออกแบบเกม(Game design) และอาจเกิดปัญหาในการจัดวางวัตถุในฉากต่างๆของเกม เช่น ขนาดของวัตถุ องศาการจัดวาง ความถูกต้องของผู้ใช้งานที่สามารถมองเห็นวัตถุ ความถูกต้องของวัตถุ เป็นต้น หรือการจัดการโฆษณาที่จะแสดงผลบนวัตถุเหล่านั้น

จากปัญหาข้างต้น ผู้จัดทำโครงการจึงได้นำระบบ การจัดการโฆษณาและเครื่องมือ มาใช้ในการแก้ไขปัญหา ข้างต้น โดยจะมีระบบการจัดการโฆษณาผ่านเว็บแอปพลิเคชัน ผู้ใช้งานสามารถนำโฆษณามาอัพโหลดและเลือกโฆษณาทั้งหมดมาโฆษณาบนเกมที่มีภายในระบบ รวมถึงเครื่องมือที่ช่วยในการจัดวางวัตถุ ตรวจสอบความถูกต้องของวัตถุและผู้ใช้งานที่มองเห็นวัตถุ และการแสดงผลโฆษณาบนวัตถุสำหรับการพัฒนาเกม ซึ่งเครื่องมือที่จะช่วยในการพัฒนาเกมจะทำให้ ซอฟต์แวร์โปรแกรมสร้างเกม (Game engine) อย่าง Unity เนื่องด้วยความนิยมในการพัฒนาเกมที่สูง และเอกสารอ้างอิง (documentation) ในการพัฒนาที่มีความละเอียด และง่ายต่อการพัฒนา

กิตติกรรมประกาศ

โครงงานนี้สำเร็จลุล่วงได้ ด้วยความกรุณาของ ดร.ณัฐกิตติ์ จิตรเอื้อตระกูล อาจารย์ที่ปรึกษาโครงงาน ที่ได้เสียสละเวลา ให้คำแนะนำ แนวคิด ตลอดจนการแก้ไขจนข้อบกพร่องต่าง ๆ มาโดยตลอด กระทั่งโครงงานนี้สำเร็จตามเป้าหมายที่วางไว้ทางกลุ่มพัฒนาจึงขอขอบพระคุณ

ขอขอบคุณ คณะพี่เลี้ยง และบุคลากรทุกท่านจากบริษัท เมทราไบต์ คลาวด์ จำกัด ที่สละเวลามาให้ข้อมูล คำแนะนำตลอดจนแนวทางในการแก้ไขปัญหา เพื่อที่ได้นำมาใช้ในการพัฒนาโครงงานชิ้นนี้

ขอขอบคุณ เพื่อนๆ และรุ่นพี่ ในคณะวิทยาศาสตร์ประยุกต์ สาขาวิทยาการคอมพิวเตอร์และสารสนเทศ สำหรับคำแนะนำ และความช่วยเหลือในทุก ๆ เรื่องที่ได้รับตลอดทั้งโครงงานนี้ ทำให้ทางผู้พัฒนาสามารถดำเนินงานต่อจนสำเร็จลุล่วงไปด้วยดี

ขอขอบพระคุณ คณะวิทยาศาสตร์ประยุกต์ มหาวิทยาลัยเทคโนโลยีพระจอมเกล้าพระนครเหนือ ที่ได้ให้โอกาสในการจัดทำโครงงานครั้งนี้ เพื่อเป็นการนำความรู้ที่ได้เรียนมาตลอด 3 ปี และการเรียนรู้สิ่งใหม่นอกห้องเรียนมาใช้ให้เกิดประโยชน์สูงสุด ตลอดจนเป็นการช่วยฝึกเตรียมความพร้อมสำหรับการทำงานจริงในอนาคตเช่นกัน

สุดท้ายนี้ขอขอบพระคุณท่านคณะกรรมการสอบโครงงานทุกท่านเป็นอย่างสูงที่ได้ช่วยพิจารณาให้คำแนะนำในการตรวจทานแก้ไข และอนุมัติให้โครงงานฉบับนี้สำเร็จลุล่วงได้ด้วยดี

นาย นโรดม ญาตินิมิตร

26 ธันวาคม 2565

สารบัญ

	หน้า
บทคัดย่อ	ข
กิตติกรรมประกาศ	ค
สารบัญ	ง
สารบัญตาราง	ฉ
สารบัญรูปภาพ	ซ
บทที่	
1 บทนำ	
1.1 ที่มาและความสำคัญ	1
1.2 วัตถุประสงค์ของโครงงาน	2
1.3 ขอบเขตการดำเนินงาน	2
1.4 ประโยชน์ที่คาดว่าจะได้รับ	3
2 ทฤษฎีที่เกี่ยวข้อง	
2.1 Unity (game engine)	4
3 วิธีการดำเนินงาน	
3.1 ภาพรวมของระบบ	6
3.2 ภาพรวมการทำงานของ Freemason	7
3.3 ภาพรวมฟังก์ชันของ Freemason	8
3.4 Use Case Diagram	10
3.5 Use case Description	11

สารบัญ (ต่อ)

หน้า

บทที่

3.6 ER diagram 27

3.7 Data dictionary 28

4 การพัฒนาระบบ

4.1 เครื่องมือที่ใช้ในการพัฒนาระบบ 33

4.2 การพัฒนาระบบของโครงงาน 34

5 การดำเนินงาน

5.1 ผลการดำเนินงาน 76

5.2 การแสดงผลของระบบ 76

6. บทสรุปและแนวทางการพัฒนาต่อ

6.1 สรุปผลการดำเนินงาน 97

6.2 ปัญหาที่พบระหว่างดำเนินงาน 97

6.3 แนวทางการแก้ปัญหา 98

6.4 แนวทางในการพัฒนาต่อ 98

บรรณานุกรม 99

สารบัญตาราง

หน้า

ตารางที่ 3-1 Use case Description : Get plugin 11
ตารางที่ 3-2 Use case Description : Plugin documentation 11
ตารางที่ 3-3 Use case Description : Game project list 12
ตารางที่ 3-4 Use case Description : Search by name 12
ตารางที่ 3-5 Use case Description : Choose game project to advertise 13
ตารางที่ 3-6 Use case Description : Create game project 13
ตารางที่ 3-7 Use case Description : Join game project 14
ตารางที่ 3-8 Use case Description : Code add users 14
ตารางที่ 3-9 Use case Description : View game project 15
ตารางที่ 3-10 Use case Description : View users status 15
ตารางที่ 3-11 Use case Description : View action users 16
ตารางที่ 3-12 Use case Description : View percent your action 16
ตารางที่ 3-13 Use case Description : View action chart every 3 days 17
ตารางที่ 3-14 Use case Description : View advertise service 17
ตารางที่ 3-15 Use case Description : View Unity plugin service 18
ตารางที่ 3-16 Use case Description : View scene 18
ตารางที่ 3-17 Use case Description : View table game objects 19
ตารางที่ 3-18 Use case Description : Key connect to Unity plugin 19
ตารางที่ 3-19 Use case Description : Plugin in Unity 20
ตารางที่ 3-20 Use case Description : Connect with game project 21
ตารางที่ 3-21 Use case Description : Camera Detector 21
ตารางที่ 3-22 Use case Description : Banner Template Detector 22
ตารางที่ 3-23 Use case Description : Controller 22
ตารางที่ 3-24 Use case Description : Banner Template 23
ตารางที่ 3-25 Use case Description : Timer 23
ตารางที่ 3-26 Use case Description : Unity game engine 24
ตารางที่ 3-27 Use case Description : Scene 24
ตารางที่ 3-28 Use case Description : Camera 25

สารบัญตาราง (ต่อ)

หน้า

ตารางที่ 3-29 Use case Description : Game objects 25
ตารางที่ 3-30 Use case Description : Add advertise video 26
ตารางที่ 3-31 Use case Description : Advertise list 26
ตารางที่ 3-32 Use case Description : Setting 27
ตารางที่ 3-33 Data Dictionary : advertising_videos 28
ตารางที่ 3-34 Data Dictionary : game_logs 28
ตารางที่ 3-35 Data Dictionary : game_objects 29
ตารางที่ 3-36 Data Dictionary : game_projects 29
ตารางที่ 3-37 Data Dictionary : game_tags 30
ตารางที่ 3-38 Data Dictionary : projects_users 30
ตารางที่ 3-39 Data Dictionary : role 30
ตารางที่ 3-40 Data Dictionary : tagging_projects 31
ตารางที่ 3-41 Data Dictionary : user 31
ตารางที่ 3-42 Data Dictionary : user_role 32
ตารางที่ 3-43 Data Dictionary : video_projects 32

สารบัญภาพ

หน้า

ภาพที่ 3-1 แสดงภาพรวมการทำงานของระบบ 7
ภาพที่ 3-2 แสดงภาพรวมการทำงานของ Freemason 8
ภาพที่ 3-3 แสดงภาพรวมฟังก์ชันใน Web Application 8
ภาพที่ 3-4 แสดงภาพรวมฟังก์ชันใน Web Application 9
ภาพที่ 3-5 Use Case Diagram (Freemason System) 10
ภาพที่ 3-6 ER Diagram 27
ภาพที่ 4-1 ส่วนการแสดงผลตัวเลือกของปลั๊กอินบน Editor 34
ภาพที่ 4-2 ส่วนของการเก็บค่าคงที่และถูกเรียกใช้งานจากคลาสอื่นๆ 35
ภาพที่ 4-3 ส่วนของการแปลงพื้นผิววัตถุให้เป็นวีดีโอของวัตถุทั้งหมดภายในปลั๊กอิน 36
ภาพที่ 4-4.1 การประกาศค่าตัวแปรต่าง ๆ บนคลาส 37
ภาพที่ 4-4.2 ฟังก์ชั่นวิธีการจัดการโฆษณาให้กับวัตถุทั้งหมด 38
ภาพที่ 4-4.3 ฟังก์ชั่นการตรวจสอบวัตถุโดยรอบ 38
ภาพที่ 4-4.4 ฟังก์ชั่นจัดการวัตถุด้วยการแบ่งกลุ่ม 39
ภาพที่ 4-5.1 การประกาศค่าตัวแปรต่าง ๆ บนคลาสและการรับข้อมูลตามวัฏจักรชีวิต 40
ภาพที่ 4-5.2 ตามกฎวัฏจักรชีวิตของ Game engine และมีฟังก์ชั่นใช้สำหรับการเพิ่มลบ วัตถุภายในระบบปลั๊กอิน รวมถึงฟังก์ชั่นเสริมในการแจ้งเตือน 41
ภาพที่ 4-5.3 ฟังก์ชั่นสำหรับการสั่งใช้งานให้ส่งข้อมูลไปยังเซิฟเวอร์และฟังก์ชั่นเสริม ในการมองเห็น 41
ภาพที่ 4-6.1 สคริปต์ที่ถูกบันทึกภายในวัตถุของปลั๊กอิน 43
ภาพที่ 4-6.2 ตามกฎวัฏจักรชีวิตของ Game engine 43
ภาพที่ 4-7.1 เครื่องจับเวลาภายในวัตถุ 44
ภาพที่ 4-7.2 เครื่องจับเวลาภายในวัตถุ 45
ภาพที่ 4-8 ส่วนของการรับข้อมูลวีดีโอโฆษณา 45
ภาพที่ 4-9 ส่วนการตรวจสอบระหว่างวัตถุและกล้อง 46
ภาพที่ 4-10 แบบอย่างการส่งข้อมูลไปยังเซิฟเวอร์ 47
ภาพที่ 4-11.1 ส่วนการส่งข้อมูลไปยังเซิฟเวอร์ 48
ภาพที่ 4-11.2 ส่วนการส่งข้อมูลไปยังเซิฟเวอร์ 48
ภาพที่ 4-12.1 แบบอย่างข้อมูลภายในปลั๊กอิน 49

สารบัญภาพ (ต่อ)

หน้า

ภาพที่ 4-12.2 แบบอย่างข้อมูลภายในปลั๊กอิน 50
ภาพที่ 4-12.3 แบบอย่างข้อมูลภายในปลั๊กอิน 51
ภาพที่ 4-13.1 ส่วนของการเข้าสู่ระบบ 52
ภาพที่ 4-13.2 ส่วนของการเข้าสู่ระบบและเรียกใช้ฟังก์ชั่นการสมัคร 53
ภาพที่ 4-14 แบบฟอร์มการสมัครเข้าใช้งานเว็บแอปพลิเคชัน 53
ภาพที่ 4-15.1 ฟังก์ชันการทำงานของการแสดงผลหน้าแรกให้ผู้ใช้งานสำหรับนักโฆษณา 54
ภาพที่ 4-15.2 ส่วนการแสดงผลหน้าแรกให้ผู้ใช้งานสำหรับนักโฆษณา 55
ภาพที่ 4-15.3 ส่วนประกอบการแสดงผลแบบตารางของรายการโฆษณา 55
ภาพที่ 4-15.4 ส่วนประกอบการแสดงผลแบบการ์ดของรายการโฆษณา 56
ภาพที่ 4-16.1 การตั้งค่าและการเลือกโฆษณาลงโปรเจคเกม 57
ภาพที่ 4-16.2 ส่วนประกอบการแสดงผลรายการโปรเจคเกมที่ให้ผู้ใช้งานโฆษณา 57
ภาพที่ 4-16.3 ส่วนประกอบฟังก์ชันของปุ่มเพิ่มโฆษณาเข้าสู่โปรเจคเกม 58
ภาพที่ 4-16.4 ส่วนประกอบการแสดงผลของปุ่มเพิ่มโฆษณาเข้าสู่โปรเจคเกม 58
ภาพที่ 4-17 ส่วนประกอบของฟังก์ชั่นการเพิ่มโฆษณาของผู้ใช้งาน 59
ภาพที่ 4-18.1 ฟังก์ชันสำหรับการแสดงผลตารางของรายการโปรเจคเกม 60
ภาพที่ 4-18.2 การแสดงผลตารางของรายการโปรเจคเกม 60
ภาพที่ 4-18.3 ส่วนประกอบการแสดงผลตารางโปรเจคเกม 61
ภาพที่ 4-19.1 ส่วนประกอบการแสดงผลตารางโปรเจคเกม 62
ภาพที่ 4-19.2 ส่วนประกอบการแสดงผลตารางโปรเจคเกม 63
ภาพที่ 4-20.1 หน้าการแสดงผลข้อมูลแผนภูมิของโปรเจคเกม 64
ภาพที่ 4-20.2 หน้าแสดงผลข้อมูลของการโฆษณามาสู่โปรเจคเกม 64
ภาพที่ 4-20.3 ฟังก์ชันการแสดงผลแบบตารางของข้อมูลโฆษณาบนโปรเจคเกม 65
ภาพที่ 4-20.4 ส่วนฟังก์ชันการแสดงผลข้อมูลของปลั๊กอินที่เชื่อมต่อกับ Unity game engine 66
ภาพที่ 4-20.5 ส่วนการแสดงผลข้อมูลของปลั๊กอินที่เชื่อมต่อกับ Unity game engine 67
ภาพที่ 4-20.6 ส่วนการแสดงตารางของรายการสมาชิกภายในโปรเจคเกม 67
ภาพที่ 4-20.7 ส่วนการแสดงข้อมูลกิจกรรมภายในโปรเจคเกม 68
ภาพที่ 4-20.8 ส่วนการแสดงข้อมูลการตั้งค่าภายในโปรเจคเกม 68

สารบัญภาพ (ต่อ)

หน้า

ภาพที่ 4-21 แสดงฟังก์ชันช่วยเหลือสำหรับหน้าการแสดงผลต่างๆ 69
ภาพที่ 5-1 หน้าแสดงผลการเข้าสู่ระบบ 76
ภาพที่ 5-2 หน้าแสดงผลแบบฟอร์มการสมัครบัญชีเพื่อเข้าใช้งาน 77
ภาพที่ 5-3 หน้าเลือกบทบาทของผู้ใช้งาน 77
ภาพที่ 5-4 หน้าแสดงรายการโฆษณาของผู้ใช้งานประเภทนักโฆษณา 78
ภาพที่ 5-5 หน้าแสดงรายการโฆษณาของผู้ใช้งานประเภทนักโฆษณาแบบตาราง 78
ภาพที่ 5-6 หน้าแบบฟอร์มสำหรับการเพิ่มวีดีโฆษณา 79
ภาพที่ 5-7 หลังจากการเพิ่มวีดีโอโฆษณา 79
ภาพที่ 5-8 หน้าแสดงการตั้งค่าการโฆษณาของผู้ใช้งาน 80
ภาพที่ 5-9 หน้าแสดงรายการโปรเจคเกมที่สามารถโฆษณา 80
ภาพที่ 5-10 แสดงรายการที่ผู้ใช้งานลงโฆษณากับโปรเจคเกมที่เลือก 81
ภาพที่ 5-11 หน้าแรกแสดงผลสำหรับผู้ใช้งานประเภทผู้พัฒนาเกม 81
ภาพที่ 5-12 หน้าแรกแสดงผลสำหรับผู้ใช้งานประเภทผู้พัฒนาเกม 82
ภาพที่ 5-13 หน้าแรกแสดงผลสำหรับผู้ใช้งานประเภทผู้พัฒนาเกม 82
ภาพที่ 5-14 ฟังก์ชันการค้นหาโปรเจคเกม 83
ภาพที่ 5-15 ฟอร์มสร้างโปรเจคเกมสำหรับผู้ใช้งานประเภทผู้พัฒนาเกม 83
ภาพที่ 5-16 ฟอร์มใส่รหัสเข้าโปรเจคเกมสำหรับผู้ใช้งานประเภทผู้พัฒนาเกม 84
ภาพที่ 5-17 หน้าแรกแสดงผลโปรเจคเกม 85
ภาพที่ 5-18 การบริการของปลั๊กอินภายในโปรเจคเกม 85
ภาพที่ 5-19 การแสดงผลแบบตารางของการบริการปลั๊กอินภายในโปรเจคเกม 86
ภาพที่ 5-20 การแสดงผลแบบแผนภูมิของการบริการของการปลั๊กอินภายในโปรเจคเกม 86
ภาพที่ 5-21 การแสดงรหัสการเชื่อมต่อระหว่างโปรเจคเกมและ
ปลั๊กอินภายใน Unity game engine 87
ภาพที่ 5-21 การบริการของการโฆษณาภายในโปรเจคเกม 87
ภาพที่ 5-22 การแสดงรหัสการให้ผู้ใช้งานอื่นเข้าร่วมโปรเจคเกม 88
ภาพที่ 5-23 การตั้งค่าเพิ่มแท็กของโปรเจคเกม 88

สารบัญภาพ (ต่อ)

หน้า

ภาพที่ 5-24 การตั้งค่าเปลี่ยนชื่อหรือรายละเอียดของโปรเจคเกม 89
ภาพที่ 5-25 การตั้งค่าการออกจากโปรเจคเกม 89
ภาพที่ 5-26 หน้าการแสดงของการบริการและช่วยเหลือ 90
ภาพที่ 5-27 หน้าการตั้งค่าการเปลี่ยนแปลงบทบาทของผู้ใช้งาน 90
ภาพที่ 5-28 หน้าการติดตั้งปลั๊กอินบน package manager ของ Unity game engine 91
ภาพที่ 5-29 หน้าการติดตั้งปลั๊กอินบน package manager ของ Unity game engine 91
ภาพที่ 5-30 หน้าการติดตั้งรับวัตถุตัวอย่างจากปลั๊กอิน 92
ภาพที่ 5-31 โฟลเดอร์เมื่อติดตั้งวัตถุตัวอย่างจากปลั๊กอิน 92
ภาพที่ 5-32 โฟลเดอร์เมื่อติดตั้งวัตถุตัวอย่างจากปลั๊กอิน 93
ภาพที่ 5-33 โฟลเดอร์เมื่อติดตั้งวัตถุตัวอย่างจากปลั๊กอิน 93
ภาพที่ 5-34 การเชื่อมต่อปลั๊กอินและโปรเจคเกมภายใน Editor ของ Unity game engine 94
ภาพที่ 5-35 การเชื่อมต่อปลั๊กอินและโปรเจคเกมภายใน Editor ของ Unity game engine 94
ภาพที่ 5-36 การวางวัตถุภายในเกมที่พัฒนา 95
ภาพที่ 5-37 การแจ้งเตือนเมื่อผู้ใช้งานวางวัตถุในระยะใกล้ตามเงื่อนไขที่กำหนด 95
ภาพที่ 5-38 การแสดงผลโฆษณาภายในเกม 96

บทที่ 1
บทนำ

1.1 ความเป็นมาและความสำคัญของโครงงาน

ปัจจุบัน โลกได้เข้าสู่ยุคดิจิทัลเต็มรูปแบบ และการโฆษณามีการเปลี่ยนแปลงไปจากเดิมอย่างมาก ทำให้พฤติกรรมของผู้บริโภคเปลี่ยนแปลงไปตามยุคตามสมัยอย่างรวดเร็ว โดยมีผู้ใช้งานอินเทอร์เน็ตเพื่อทำกิจกรรมและธุรกรรมต่างๆ ผ่านระบบออนไลน์ โดยจากสถิติเดือน มกราคม พ.ศ. 2564 จำนวนผู้ใช้งานโซเชียลมีเดียภายในประเทศไทย เช่น Facebook มีถึง 51 ล้านบัญชี (84.9% ของจำนวนประชากรอายุ 13 ปีขึ้นไป) , Youtube มี 37.3 ล้านบัญชี (54.3% ของจำนวนประชากรอายุ 13 ปีขึ้นไป) เป็นต้น จากสถิติสำหรับผู้ใช้งานหนึ่งคนสามารถสร้างบัญชีได้มากกว่า 1 บัญชี จึงไม่นับตัวเลขที่กรองคนซ้ำ

จากเหตุการณ์ดังกล่าว สื่อสังคมออนไลน์เข้ามามีบทบาทและอิทธิพลต่อความคิดต่อผู้ใช้งาน จึงทำให้เกิดผู้ลงทุนโฆษณาสินค้าและอื่นๆ บนสังคมออนไลน์เป็นจำนวนมาก และในช่วงเวลาเดียวกันมีอีกหนึ่งสถิติที่น่าสนใจเป็นอย่างมากคือ ประเทศไทยมีการใช้อินเทอร์เน็ตเพื่อเล่นเกมเป็นอันดับ 1 ของโลก (เฉลี่ย 96.6% ของผู้ใช้งานทั่วโลก) จากสถิติเห็นได้ว่า เกมในปัจจุบันมีอิทธิพลต่อผู้บริโภคเช่นเดียวกับสื่อสังคมออนไลน์ต่างๆ อีกทั้งยังเป็นสื่อที่ให้ความบันเทิงกับผู้ใช้งานอีกด้วย และเกมยังสามารถเป็นเครือข่ายสื่อสังคมออนไลน์ได้อีกรูปแบบหนึ่งเป็นเสมือนชุมชนคนและเพื่อนที่สนใจสิ่งเดียวกัน โดยลักษณะสัมพันธ์ดังกล่าวทำให้ธุรกิจปรับตัวและพยายามสร้างความสัมพันธ์กับผู้ใช้งานกลุ่มนี้มากขึ้น รวมไปถึงการโฆษณาเพื่อผู้ใช้งานกลุ่มนี้ได้รับรู้ในตัวสินค้าและบริการ ซึ่งการโฆษณาบนเกมตามแพลตฟอร์มต่างๆ มักมีปัญหาการนำเสนอโฆษณาบนตัวเกม เช่น การโฆษณาเกมบนโทรศัพท์มือถือที่ผู้ใช้งานเล่นเกมอยู่และมีโฆษณาหยุดกลางคันระหว่างเล่นเกม ทำให้ผู้ใช้งานเกิดความรู้สึกไม่พอใจได้ เป็นต้น

ดังนั้น เราจึงสร้างเครื่องมือให้สำหรับผู้พัฒนาเกมบนโปรแกรม Unity เนื่องจากเป็นโปรแกรมพัฒนาเกมที่สามารถพัฒนาเกมได้หลายรูปแบบ หลายแพลตฟอร์ม และมีผู้ใช้โปรแกรมนี้จำนวนมาก ซึ่งเครื่องมือที่จะให้ผู้พัฒนาเกมได้ใช้จะใช้วิธีการแบบ “โฆษณาแฝง” วิธีการดังกล่าว คือ การออกแบบวัตถุให้เหมือนกับของประกอบฉากหรือส่วนหนึ่งของเกม และผู้พัฒนาเกมสามารถนำวัตถุเหล่านั้นนำไปจัดวางในพื้นที่ต่างๆ ซึ่งระบบจะแสดงโฆษณาบนวัตถุเหล่านั้นตามเงื่อนไขที่ถูกกำหนดและจัดการจากผู้พัฒนาเกมว่าโฆษณาประเภทแบบใดบ้างที่สามารถนำมาโฆษณาบนตัวเกมของผู้พัฒนาได้ โดยการจัดการระบบต่างๆ จะถูกจัดการบนเว็บแอปพลิเคชัน รวมไปถึงผู้ลงทุนโฆษณาสินค้าและบริการสามารถใช้งานบนเว็บแอปพลิเคชันเพื่อลงโฆษณาของตนได้ ซึ่งมีค่าใช้จ่ายในการลงโฆษณาเช่นกัน และรายได้จากการลงโฆษณาของผู้ลงทุนจะถูกแบ่งให้กับผู้พัฒนาเกมที่ลงโฆษณาของผู้ลงทุน

1.2 **วัตถุประสงค์ของโครงงาน**

1.2.1 เพื่อพัฒนาเว็บแอปพลิเคชัน (**Web Application)** ให้ผู้ใช้งานสามารถจัดการโฆษณาเข้าสู่เกมได้

1.2.2 มีเครื่องมือในการจัดวางวัตถุในการแสดงผลโฆษณาภายในเกม

1.2.3 สามารถจัดการข้อมูลเกี่ยวการโฆษณาภายในเกมได้

1.3ขอบเขตของโครงงาน

1.3.1 เครื่องมือ(Plugins) สำหรับนักพัฒนาเกม Unity

1.3.1.1 เครื่องเล่นแสดงผลบนวัตถุ

1.3.1.1.1 เครื่องเล่นวีดีโอ(Video player)

1.3.1.1.2 ตัวอย่างวัตถุรูปแบบต่างๆ

1.3.1.2 กล้องตรวจจับวัตถุที่ถูกเครื่องเล่นแสดงผลบนวัตถุ

1.3.1.2.1 เครื่องนับเวลา(Timer)

1.3.1.2.2 เครื่องตรวจสอบจับวัตถุ

1.3.1.3 ตัวจัดการ API สำหรับการรับส่งข้อมูล

1.3.2 หน้าสำหรับแสดงรายละเอียด Web application

1.3.2.1 หน้าเว็บสำหรับผู้พัฒนาเกม

1.3.2.1.1 แสดงรายละเอียดข้อมูลการใช้งานและการติดตั้ง Plugin

1.3.2.1.2 แสดงรายละเอียดข้อมูลจากโปรเจคของเกมทั้งหมด

1.3.2.1.3 เพิ่มหรือเข้าร่วมโปรเจคของเกม

1.3.2.1.4 สามารถดูข้อมูลและแก้ไขรายละเอียดโปรเจคของเกม

1.3.2.1.5 สามารถเชื่อมต่อระหว่างโปรเจคของเกมกับ Plugin

1.3.2.1.6 แสดงรายละเอียดโฆษณาและรายละเอียด Plugin

1.3.2.1.7 สามารถตั้งค่าผู้ใช้งานและโปรเจคของเกม

1.3.2.2 หน้าเว็บสำหรับผู้ลงทุนโฆษณา

1.3.2.2.1 แสดงรายละเอียดข้อมูลวีดีโอโฆษณาที่อัพโหลด

1.3.2.2.2 สามารถเพิ่มวีดีโอโฆษณา

1.3.2.2.3 แสดงรายละเอียดข้อมูลของเกมทั้งหมด

1.3.2.2.4 สามารถลงหรือถอนโฆษณากับเกม

1.4 ประโยชน์ที่คาดว่าจะได้รับ

1.4.1 มีรูปแบบในการนำเสนอโฆษณาสินค้าและบริการกับเกมในรูปแบบใหม่

1.4.2 ช่วยลดการจัดการโฆษณาระหว่างเกมกับการโฆษณาสินค้าและบริการ

1.4.3 เว็บแอปพลิเคชัน (Web Application) สามารถจัดการโฆษณากับเกมต่างๆ

1.4.4 ช่วยให้นักพัฒนาเกมลดภาระในการสร้างวัตถุแสดงผลโฆษณาเพื่อใช้ในการจัดวางวัตถุภายในเกม

บทที่ 2
ทฤษฎีบทที่เกี่ยวข้อง

2.1 Unity (game engine)

คือ ซอฟต์แวร์แบบข้ามแพลตฟอร์ม ใช้เพื่อสำหรับการพัฒนาซอฟต์แวร์และการจำลองต่างๆ จำพวกเช่น วีดีโอเกม, อุตสาหกรรมยานยนต์, การขนส่ง, ภาพยนตร์, แอนิเมชัน, สถาปัตยกรรม, วิศวกรรมศาสตร์, วิศวกรรมก่อสร้าง, วิศวกรรมการบินและอวกาศ, การพนัน, และอื่นๆ

2.1.1 แพลตฟอร์มที่รองรับ

ตัวซอฟต์แวร์ที่ใช้สำหรับพัฒนานั้นรองรับบนแพลตฟอร์ม ไมโครซอฟท์วินโดวส์, แมคโอเอส และ ลินุกซ์ ส่วนตัวเอนจินเองนั้นรองรับมากถึง 20 แพลตฟอร์ม, อ้างอิงจากเอกสารคู่มือมีดังนี้

1. แพลตฟอร์มสำหรับเครื่องเดสก์ท็อป: ไมโครซอฟท์วินโดวส์, แมคโอเอส, ลินุกซ์
2. แพลตฟอร์มสำหรับเครื่องพกพา: ไอโอเอส, แอนดรอยด์, แอนดรอยด์ ทีวี, ทีวีโอเอส
3. แพลตฟอร์มบนเว็บ: เว็บจีแอล
4. แพลตฟอร์มสำหรับเครื่องคอนโซล: เพลย์สเตชัน 4, เพลย์สเตชัน 5, เอกซ์บอกซ์วัน, เอกซ์บอกซ์วันซีรีส์ X และ S, นินเท็นโด_สวิตช์, สตาเดีย
5. แพลตฟอร์มสำหรับอุปกรณ์ความเป็นจริงเสมือน และ ความเป็นจริงเสริม: Reality Labs, เพลย์สเตชัน วีอาร์, กูเกิล AR Core, แอปเปิล AR Kit, Windows Mixed Reality, Magic Leap, สตีม VR, กูเกิล Cardboard

2.1.2 ภาษาที่ใช้เขียน

ตัวเอนจินนั้นใช้ภาษาซีพลัสพลัสในการเขียนเป็นแกนกลางหลักของระบบและมีภาษาซีชาร์ปครอบทับการทำงานต่างๆ เช่น ระบบส่วนติดต่อกับพื้นหลัง, การทำงานของตัวเกมที่ผู้พัฒนาต้องการ และ อื่นๆ ด้วย Mono

ผู้พัฒนาสามารถเขียนด้วยภาษาซีพลัสพลัสได้ แต่ยังไงก็ตามผู้พัฒนาสามารถเขียนภาษาซีพลัสพลัสได้แค่ในส่วนการทำงานพื้นหลังเท่านั้นเช่น ระบบเสียง, บริการพื้นหลัง, ปลั๊กอิน ซึ่งการปฏิสัมพันธ์กับเอนจินทำได้ค่อนข้างน้อย ดังนั้นภาษาภาษาซีชาร์ปจึงเหมาะสมสำหรับกับการใช้ในการพัฒนามากกว่า

2.1.3 Asset Store

Unity Asset Store คือพื้นที่ ที่ผู้พัฒนาสามารถ ซื้อ หรือ ขายชิ้นงานของตัวเองได้, ในร้านค้ามีชิ้นงานมากมายหลากหลายรูปแบบเช่น โมเดลตัวละคร, โมเดลสิ่งก่อสร้าง, ไฟล์เสียง, สคริปต์, เครื่องมือเสริม และ อื่นๆ อีกมากมาย

2.1.4 Forum

Unity Forum คือพื้นที่ ที่ผู้พัฒนาสามารถใช้เพื่อติดตามข่าวสารจากยูนิตี, ตั้งถาม, พูดคุย กับผู้พัฒนารายอื่นๆ ในชุมชนได้

บทที่ 3
วิธีการดำเนินงาน

3.1 ภาพรวมการทำงานของระบบ

ภาพรวมการทำงานของระบบแสดงส่วนประกอบต่างๆของระบบโดยมีการทำงานดังนี้

3.1.1 ผู้พัฒนาเกม(Game Developer) สามารถจัดการข้อมูลเกี่ยวกับเกมที่เชื่อมกับเครื่องมือ (Plugin) ผ่าน Web Application และพัฒนา (Develop) เกมบน Unity (game engine)

3.1.2 ผู้ลงโฆษณา(Advertiser) สามารถอัพโหลดหรือถอนโฆษณา และสามารถเลือกเกมที่ต้องการโฆษณาได้

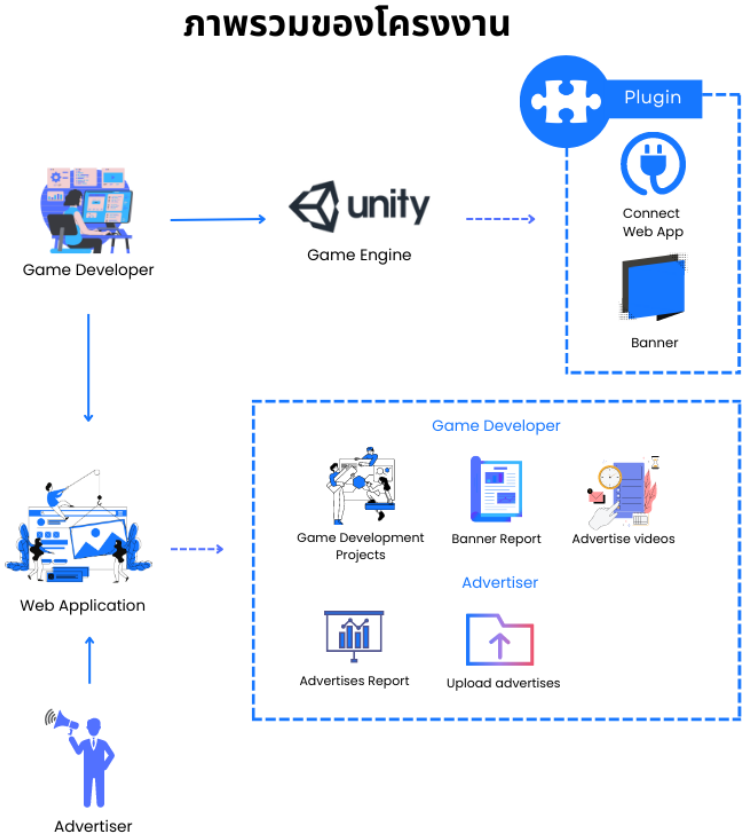
3.1.3 Plugin ภายใน Unity (game engine) ทำหน้าที่รับข้อมูลโฆษณาเพื่อนำไปแสดงผลบนเกม และส่งข้อมูลจำนวนเวลาที่มองเห็นโฆษณาให้กับ Web Application

3.1.4 Web Application ทำหน้าที่แสดงข้อมูลและจัดการข้อมูลให้กับ ผู้ใช้งาน นักพัฒนาเกม(Game Developer) และผู้ลงโฆษณา (Advertiser) รวมถึงการรับข้อมูลจากเครื่องมือ (Plugin) เพื่อนำไปแสดงผล

3.1.5 Server และ Database ทำหน้าที่รับ request การเข้า ดู / ปรับปรุง / แก้ไข ข้อมูลต่างๆใน Database และส่ง reply กลับไปเมื่อ process เสร็จสิ้น

3.1.6 Game Products ผลิตภัณฑ์เกมที่ผ่านการพัฒนาเกมแล้วให้ผู้ใช้งานมาเล่นบนผลิตภัณฑ์เกมได้

3.1.7 User ผู้ใช้งานที่ใช้งานบนผลิตภัณฑ์เกม



ภาพที่ 3-1 แสดงภาพรวมการทำงานของระบบ

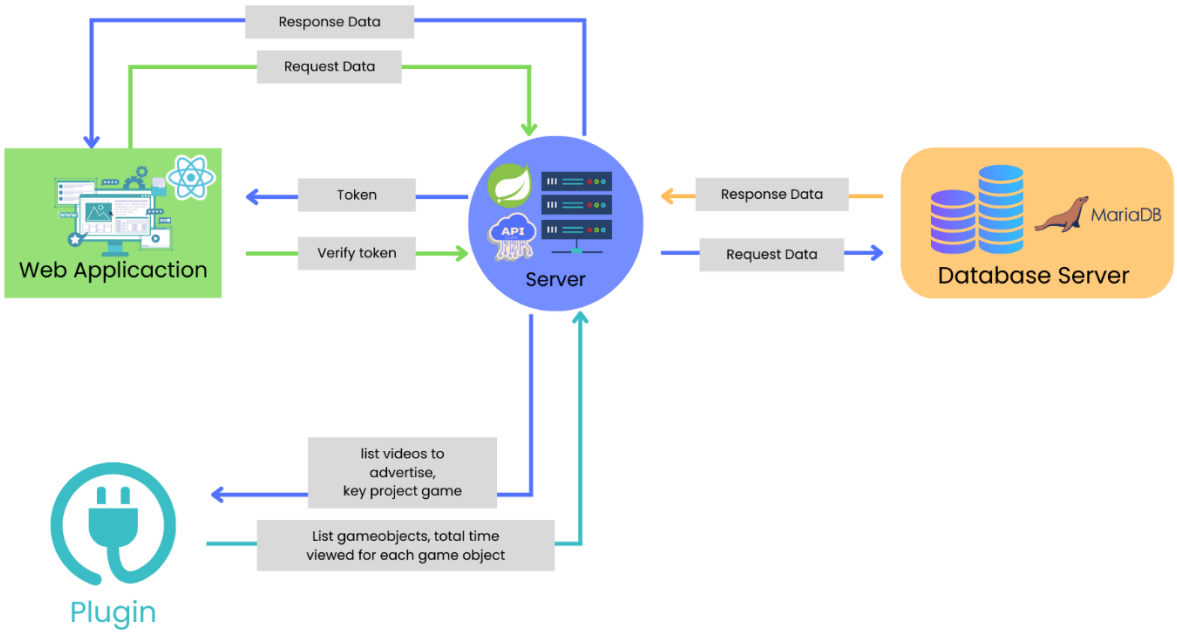
3.2 ภาพรวมการทำงานของ Freemason

ภาพรวมการทำงานของระบบแสดงส่วนประกอบต่างๆของระบบโดยมีการทำงานดังนี้

3.2.1 Web Application เป็นหน้าเว็บส่ง request มาให้ server และรอรับข้อมูลที่ server ส่งมามาแสดงที่หน้าเว็บ

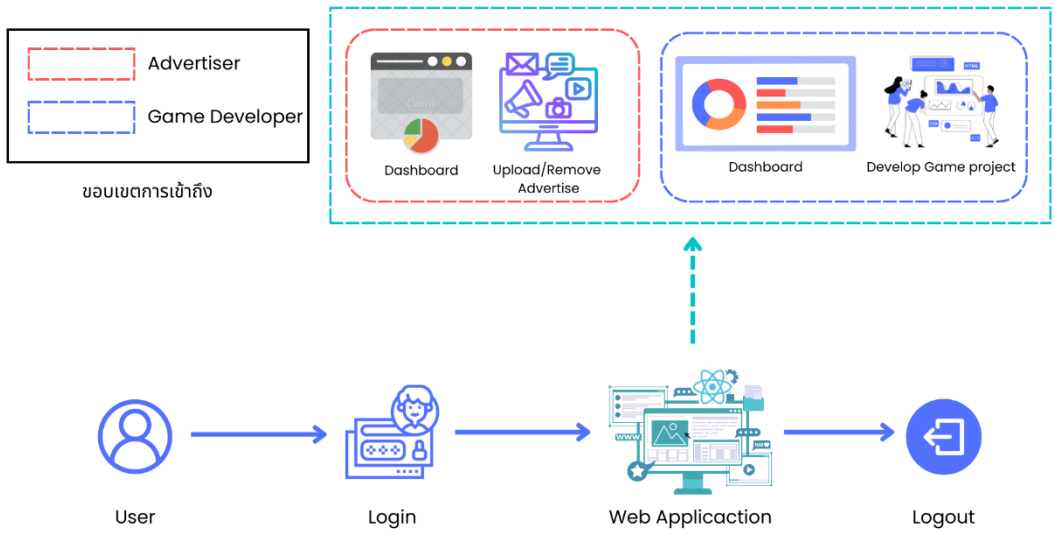
3.2.2 Database Server ทำหน้าที่รับ request การเข้า ดู / ปรับปรุง / แก้ไขข้อมูลต่างๆใน Database และส่ง reply กลับไปเมื่อ process เสร็จสิ้น

3.2.3 Plugin ทำหน้าที่รับข้อมูลวีดีโอโฆษณาเพื่อแสดงผลและส่งข้อมูลรายการวัตถุภายในเกมและยอดรับชมแต่ละวัตถุ



ภาพที่ 3-2 แสดงภาพรวมการทำงานของ Freemason

3.3 ภาพรวมฟังก์ชันของ Freemason



ภาพที่ 3-3 แสดงภาพรวมฟังก์ชันใน Web Application

ภาพรวมการทำงานของระบบแสดงฟังก์ชั่นต่างๆภายในระบบโดยมีการทำงานดังนี้

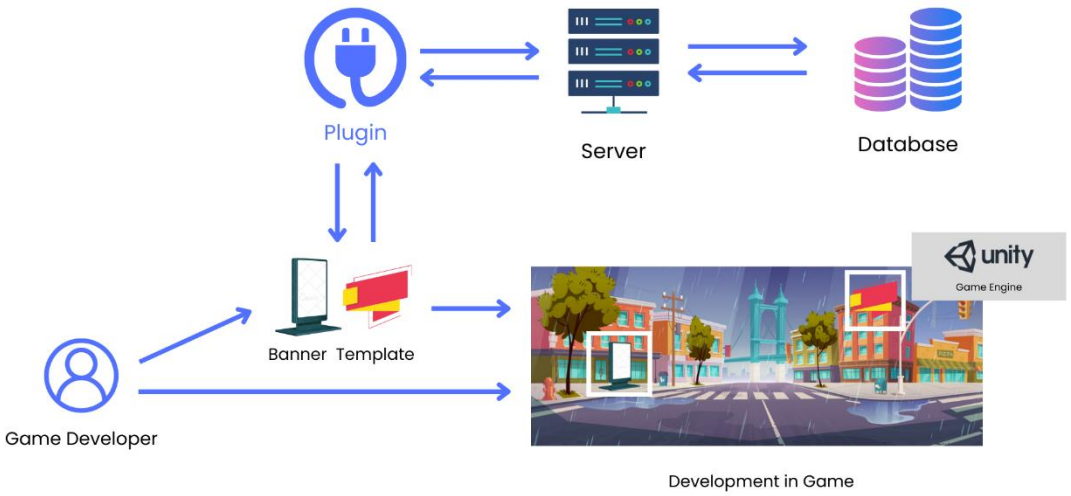
3.3.1 Game Developer ผู้พัฒนาเกมสามารถพัฒนาเกมและนำวัตถุ Banner Template จาก Plugin มาใช้งานบน Unity game engine ได้

3.3.2 Banner Template ตัวอย่างป้ายโฆษณา หรือวัตถุภายใน Unity game engine ที่มาพร้อมกับ Plugin ให้สำหรับ Game Developer ได้ใช้งาน

3.3.3 Plugin ทำหน้าที่รับข้อมูลวีดีโอโฆษณาเพื่อแสดงผลบนวัตถุหรือ Banner Template และส่งข้อมูลรายการวัตถุหรือ Banner Template ภายในเกม รวมถึงตรวจจับการมองเห็นบนวัตถุหรือ Banner Template และนับเวลาการมองเห็นรายวัตถุและส่งไปยังเซิฟเวอร์ในเวลาตามที่กำหนดไว้

3.3.4 Server มีหน้าที่ส่งข้อมูลรายการวีดีโอโฆษณาให้กับ Plugin และรับข้อมูลวัตถุ Banner Template รวมถึงข้อมูลต่างๆจาก Plugin และนำข้อมูลไปเก็บลง Database

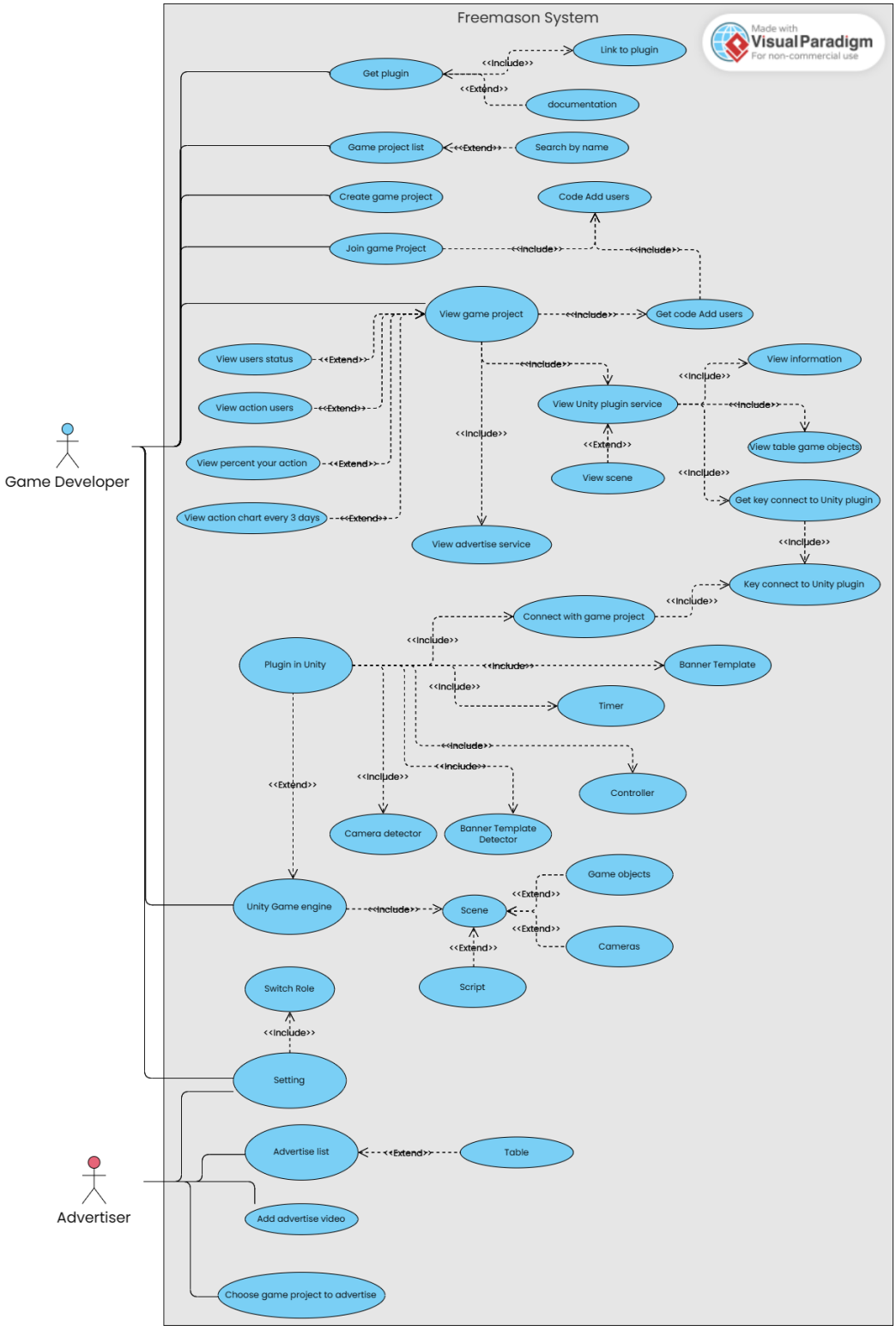
3.3.5 Database มีหน้าที่เก็บข้อมูลรายการต่างๆของ Banner Template และยอดการรับชมรวมถึงข้อมูลเพิ่มเติมต่างๆ



ภาพที่ 3-4 แสดงภาพรวมฟังก์ชันใน Web Application

3.4 Use Case Diagram

Use Case Diagram แสดงการทำงานของ Freemason System ดังภาพ



ภาพที่ 3-5 Use Case Diagram (Freemason System)

3.5 Use case Description

Use case Description สำหรับ Freemason System อธิบายจากภาพที่ 3.3

ตารางที่ 3-1 Use case Description : Get plugin

Use Case Name	Get plugin	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถรับข้อมูลของ plugin	
Pre – Conditions	-	
Post – Conditions	-	
Flow of Event	User	System
	1. กดปุ่มรับข้อมูลเกี่ยวกับ plugin	2. แสดงผลข้อมูล plugin และ URL ดาวน์โหลด plugin
Exception	-	

ตารางที่ 3-2 Use case Description : Plugin documentation

Use Case Name	Plugin documentation	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถรับข้อมูลเอกสารเกี่ยวกับ plugin	
Pre – Conditions	-	
Post – Conditions	-	
Flow of Event	User	System
	1. กดปุ่มรับข้อมูลเอกสารเกี่ยวกับ plugin	2. แสดงผลข้อมูลเอกสารเกี่ยวกับ plugin
Exception	หากไม่มีข้อมูลเอกสารเกี่ยวกับ plugin ก็จะไม่แสดงผล	

ตารางที่ 3-3 Use case Description : Game project list

Use Case Name	Game project list	
Actor	Game Developer	
Brief Description	แสดงข้อมูลเกี่ยวกับโปรเจคเกมที่พัฒนาของผู้ใช้งานทั้งหมด	
Pre – Conditions	ผู้ใช้ต้องมีการเข้าสู่ระบบอยู่แล้ว	
Post – Conditions	-	
Flow of Event	User	System
		1.ระบบแสดงข้อมูลเกี่ยวกับโปรเจคเกมที่พัฒนาทั้งหมด
Exception	-	

ตารางที่ 3-4 Use case Description : Search by name

Use Case Name	Search by name	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถค้นหาโปรเจคเกมที่พัฒนาได้ด้วยชื่อโปรเจคเกม	
Pre – Conditions	ต้องมีข้อมูลโปรเจคเกมที่พัฒนาอยู่	
Post – Conditions	-	
Flow of Event	User	System
	1. กรอกชื่อโปรเจคเกมที่ต้องการค้นหา 2. กดปุ่มค้นหาโปรเจคเกม	3. แสดงข้อมูลโปรเจคเกมทั้งหมดที่ตรงกับชื่อโปรเจคเกมที่ค้นหา
Exception	ค้นหาชื่อโปรเจคเกมไม่ตรงกับรายการโปรเจคเกมใดๆ	

ตารางที่ 3-5 Use case Description : Choose game project to advertise

Use Case Name	Choose game project to advertise	
Actor	Advertiser	
Brief Description	ผู้ใช้งานสามารถเลือกโปรเจคเกมที่ต้องการโฆษณา เพื่อโฆษณาวีดีโอของผู้ใช้งานทั้งหมดลงโปรเจคเกมที่ต้องการ	
Pre – Conditions	จำเป็นต้องมีวีดีโอที่จะโฆษณา และโปรเจคเกม	
Post – Conditions	-	
Flow of Event	User	System
	1.เลือกโปรเจคเกมที่ต้องการโฆษณา	2.นำข้อมูลวีดีโอของผู้ใช้งานทั้งหมดโฆษณาบนโปรเจคเกม
Exception	ไม่สามารถโฆษณาบนโปรเจคเกมได้ หากไม่มีวีดีโอโฆษณา และไม่สามารถโฆษณาได้ หากไม่มีโปรเจคเกม	

ตารางที่ 3-6 Use case Description : Create game project

Use Case Name	Create game project	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถสร้างโปรเจคเกมโดยกรอกข้อมูลที่ต้องการ	
Pre – Conditions	ผู้ใช้ต้องมีการเข้าสู่ระบบอยู่แล้ว	
Post – Conditions	-	
Flow of Event	User	System
	1.กดปุ่ม create project 2.กรอกข้อมูลที่ต้องการสร้าง game project	3.เพิ่มข้อมูลลงรายการโปรเจคเกมของผู้ใช้งาน
Exception	หากกรอกข้อมูลไม่ถูกต้อง จะไม่สามารถสร้างโปรเจคเกมได้	

ตารางที่ 3-7 Use case Description : Join game project

Use Case Name	Join game project	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถเข้าร่วมโปรเจคเกมของผู้ใช้งานคนอื่นๆได้	
Pre – Conditions	จำเป็นต้องมีรหัสเข้าร่วมโปรเจคเกม	
Post – Conditions	-	
Flow of Event	User	System
	1. กดปุ่ม join project 2. กรอกรหัสโปรเจคเกมที่ได้รับ	3.เพิ่มข้อมูลผู้ใช้งานบนโปรเจคเกม
Exception	หากรหัสโปรเจคเกมไม่ถูกต้องหรือไม่มีอยู่จริง จะไม่สามารถเข้าร่วมโปรเจคได้	

ตารางที่ 3-8 Use case Description : Code add users

Use Case Name	Code add users	
Actor	Game Developer	
Brief Description	รหัสโปรเจคเกมที่สามารถใช้ในการเข้าร่วมโปรเจคเกม	
Pre – Conditions	ผู้ใช้งานภายในโปรเจคเกมนั้นๆ ต้องกดรับรหัสโปรเจคเกม	
Post – Conditions	-	
Flow of Event	User	System
	1. กดปุ่ม Add people 2. คัดลอกรหัสหรือสามารถกดปุ่มคัดลอก	
Exception	-	

ตารางที่ 3-9 Use case Description : View game project

Use Case Name	View game project	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถเข้าใช้งานเกี่ยวกับโปรเจคเกมได้ เพื่อจัดการระบบและดูข้อมูลเกี่ยวกับโปรเจคเกม	
Pre – Conditions	ผู้ใช้งานต้องมีโปรเจคเกม	
Post – Conditions	-	
Flow of Event	User	System
	1. กดปุ่มดูหรือชื่อโปรเจคเกมที่รายการโปรเจคเกมทั้งหมด	2. แสดงข้อมูลและระบบต่างๆ ของโปรเจคเกม
Exception	-	

ตารางที่ 3-10 Use case Description : View users status

Use Case Name	View users status	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถดูข้อมูลเกี่ยวกับสถานะผู้ใช้งานทุกคนภายในโปรเจคเกม	
Pre – Conditions	ผู้ใช้งานต้องเข้าใช้งานโปรเจคเกม	
Post – Conditions	-	
Flow of Event	User	System
		1.แสดงผลตารางข้อมูลเกี่ยวกับสถานะผู้ใช้งานทุกคนภายในโปรเจคเกม
Exception	-	

ตารางที่ 3-11 Use case Description : View action users

Use Case Name	View action users	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถดูข้อมูลเกี่ยวกับกระทำต่างๆของผู้ใช้งานทุกคนภายในโปรเจคเกม	
Pre – Conditions	ผู้ใช้งานต้องเข้าใช้งานโปรเจคเกม	
Post – Conditions	-	
Flow of Event	User	System
		1.แสดงผลข้อมูล Timeline การกระทำของผู้ใช้งานทุกคนภายในโปรเจคเกม
Exception	-	

ตารางที่ 3-12 Use case Description : View percent your action

Use Case Name	View percent your action	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถดูข้อมูลเกี่ยวกับเปอร์เซ็นต์ของการกระทำผู้ใช้งานต่อโปรเจคเกมกับผู้ใช้งานทั้งหมดภายในโปรเจคเกม	
Pre – Conditions	ผู้ใช้งานต้องเข้าใช้งานโปรเจคเกม	
Post – Conditions	-	
Flow of Event	User	System
		1.แสดงข้อมูลแผนภูมิเปอร์เซ็นต์ของการกระทำผู้ใช้งานต่อโปรเจคเกมกับผู้ใช้งานทั้งหมดภายในโปรเจคเกม
Exception	-	

ตารางที่ 3-13 Use case Description : View action chart every 3 days

Use Case Name	View action chart every 3 days	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถดูข้อมูลแผนภูมิการกระทำทั้งหมดทุก 3 วัน ของโปรเจคเกม	
Pre – Conditions	ต้องมีข้อมูลการกระทำภายในโปรเจคเกมหรือโปรเจคเกมสร้างได้ไม่นานกว่า 3 วัน	
Post – Conditions	-	
Flow of Event	User	System
		1. แสดงข้อมูลแผนภูมิการกระทำทั้งหมดทุก 3 วันของโปรเจคเกม
Exception	หากสร้างได้ไม่นานกว่า 3 วันหรือไม่มีการกระทำภายในโปรเจคเกมจะไม่แสดงผล	

ตารางที่ 3-14 Use case Description : View advertise service

Use Case Name	View advertise service	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถดูข้อมูลตารางเกี่ยวกับโฆษณาที่เพิ่มกับโปรเจคเกมได้	
Pre – Conditions	ผู้ใช้งานต้องเข้าใช้งานโปรเจคเกมและต้องมีโฆษณาเพิ่มโปรเจคเกม	
Post – Conditions	-	
Flow of Event	User	System
		1.แสดงข้อมูลตารางเกี่ยวกับโฆษณาที่เพิ่มกับโปรเจคเกม
Exception	จะไม่แสดงผล หากไม่มีโฆษณาเพิ่มให้กับโปรเจคเกม	

ตารางที่ 3-15 Use case Description : View Unity plugin service

Use Case Name	View Unity plugin service	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถดูข้อมูลเกี่ยวกับโปรเจคเกมและข้อมูล plugin ภายในโปรเจคเกม	
Pre – Conditions	ผู้ใช้งานต้องเข้าใช้งานโปรเจคเกม	
Post – Conditions	-	
Flow of Event	User	System
		1.แสดงผลข้อมูลเกี่ยวกับโปรเจคเกมและข้อมูล plugin ภายในโปรเจคเกม
Exception	-	

ตารางที่ 3-16 Use case Description : View scene

Use Case Name	View scene	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถดูตำแหน่งต่างๆของวัตถุภายในแผนผัง	
Pre – Conditions	ต้องมีข้อมูลวัตถุจาก plugin อย่างน้อย 1 วัตถุ	
Post – Conditions	-	
Flow of Event	User	System
	1.กดแถบ scene	2.แสดงวัตถุตำแหน่งต่างๆในแผนผัง
Exception	ไม่สามารถแสดงผลหรือเลือกแถบ scene ได้หากไม่มีข้อมูลวัตถุ	

ตารางที่ 3-17 Use case Description : View table game objects

Use Case Name	View table game objects	
Actor	Game Developer	
Brief Description	ผู้ใช้งานสามารถดูข้อมูลตารางรายการวัตถุที่ได้รับจาก plugin	
Pre – Conditions	ต้องมีข้อมูลวัตถุจาก plugin อย่างน้อย 1 วัตถุ	
Post – Conditions	-	
Flow of Event	User	System
	1. กดแถบ table	2. แสดงตารางข้อมูลของรายการวัตถุทั้งหมด
Exception	จะไม่แสดงผล หากไม่มีข้อมูลวัตถุ	

ตารางที่ 3-18 Use case Description : Key connect to Unity plugin

Use Case Name	Key connect to Unity plugin	
Actor	Game Developer	
Brief Description	รหัสเชื่อมต่อกับ plugin Unity ที่สามารถใช้ในการเชื่อมต่อส่งข้อมูลระหว่าง plugin กับโปรเจคเกม	
Pre – Conditions	-	
Post – Conditions	-	
Flow of Event	User	System
	1. กดปุ่ม key connect to Unity 2. คัดลอกรหัสหรือสามารถกดปุ่มคัดลอก	
Exception	-	

ตารางที่ 3-19 Use case Description : Plugin in Unity

Use Case Name	Plugin in Unity	
Actor	Game Developer	
Brief Description	Plugin เครื่องมือที่ช่วยในการใช้งานภายใน Unity game engine	
Pre – Conditions	ต้องติดตั้ง plugin ภายใน Unity game engine	
Post – Conditions	หากหา documentation ที่ plugin ใช้งานไม่พบ หรือเวอร์ชั่นต่ำกว่าที่กำหนด ให้ติดตั้ง ตาม documentation ที่ขาดไปหรือเพิ่มเวอร์ชั่นของ Unity game engine	
Flow of Event	User	System
	1. ติดตั้ง plugin ให้กับ Unity game engine 3.กดติดตั้ง Banner template เข้าสู่ assets ภายใน editor ของ Unity game engine	2. ทำการเพิ่มส่วนเสริมและเครื่องมือให้กับ Unity game engine
Exception	หากถูกถอนการติดตั้งออก ขณะมี Banner template ภายใน assets หรือ scene ของผู้ใช้งาน ภายใน Unity game engine จะไม่สามารถรับ ส่ง และเชื่อมข้อมูลกับโปรเจคเกมได้ และเกิดการแสดงผลผิดพลาดภายใน scene ของผู้ใช้งาน (ข้อผิดพลาดหา Texture หรือ Material และ Script ไม่พบ)	

ตารางที่ 3-20 Use case Description : Connect with game project

Use Case Name	Connect with game project	
Actor	Game Developer	
Brief Description	รหัสเชื่อมต่อข้อมูลระหว่าง plugin กับ game project	
Pre – Conditions	ต้องมี plugin และมีรหัสเชื่อมต่อ	
Post – Conditions	-	
Flow of Event	User	System
	1. กดแถบ Freemason editor ของ unity game engine 3.กรอกรหัสเชื่อมต่อลงฟอร์ม	2.แสดงฟอร์มข้อมูลกรอกสำหรับการเชื่อมต่อ plugin
Exception	-	

ตารางที่ 3-21 Use case Description : Camera Detector

Use Case Name	Camera detector	
Actor	Game Developer	
Brief Description	เครื่องตรวจจับกล้องทั้งหมดภายใน scene เข้าสู่ระบบ plugin	
Pre – Conditions	ต้องมี plugin	
Post – Conditions	-	
Flow of Event	User	System
		1.หากล้องทั้งหมดภายใน scene 2.รับข้อมูลของกล้องเข้าสู่ระบบ plugin 3.นำ script การมองเห็นวัตถุของ Banner Template เพิ่มเข้าสู่กล้อง
Exception	หากไม่มีกล้องภายใน scene ไม่สามารถใช้งานได้	

ตารางที่ 3-22 Use case Description : Banner Template Detector

Use Case Name	Banner Template Detector	
Actor	Game Developer	
Brief Description	เครื่องตรวจจับวัตถุที่เป็น Banner Template ทั้งหมดภายใน scene และตรวจความถูกต้องของวัตถุ	
Pre – Conditions	ต้องมี plugin	
Post – Conditions	-	
Flow of Event	User	System
		1.ค้นหาวัตถุที่มีลักษณะและ script ต่างๆแบบ Banner Template 2.ตรวจสอบความถูกต้องรายวัตถุ 3.ตรวจสอบระยะห่างของวัตถุภายใน scene 4.ตรวจสอบการมองเห็นวัตถุกับกล้องที่มอง
Exception	-	

ตารางที่ 3-23 Use case Description : Controller

Use Case Name	Controller	
Actor	Game Developer	
Brief Description	ตัวควบคุมรับข้อมูลต่างๆจาก server รับข้อมูลรายการโฆษณา และส่งข้อมูลรายการวัตถุ รวมถึงรายละเอียดข้อมูลของวัตถุต่างๆ	
Pre – Conditions	ต้องมี plugin	
Post – Conditions	-	
Flow of Event	User	System
		1.รับข้อมูลรายการโฆษณา 2.ส่งข้อมูลรายการวัตถุและข้อมูลรายละเอียดตามเวลาที่กำหนด
Exception	จะไม่สามารถรับส่งข้อมูลได้ หาก server เกิดข้อผิดพลาดหรือรูปแบบข้อมูลมีการเปลี่ยนแปลง	

ตารางที่ 3-24 Use case Description : Banner Template

Use Case Name	Banner Template	
Actor	Game Developer	
Brief Description	ตัวอย่าง Game objects สำหรับ Plugin ใช้ในการวางเข้าสู่ Scene เป็น Game objects ที่ถูกบันทึก Scripts ของ Timer และมีการนำข้อมูลทั้งหมดเข้าระบบ plugin	
Pre – Conditions	ต้องมี Plugin และติดตั้ง Banner Template เข้าสู่ Assets	
Post – Conditions	-	
Flow of Event	User	System
	1.นำวัตถุตัวอย่างเข้าสู่ scene	2.เพิ่มข้อมูลเกี่ยวกับ game object เข้าสู่ระบบ plugin 3.เมื่อมีการมองเห็นด้วยกล้องใช้งาน script timer
Exception	จะไม่สามารถถูกใช้งานได้ หากไม่มี plugin และข้อมูลของ game object ไม่ถูกต้อง	

ตารางที่ 3-25 Use case Description : Timer

Use Case Name	Timer	
Actor	Game Developer	
Brief Description	Script ที่ถูกบันทึกในวัตถุของ Banner Template มีหน้าที่จับเวลาของวัตถุเมื่อมีการถูกเรียกใช้งาน	
Pre – Conditions	เมื่อถูกบันทึกภายในวัตถุและถูกเรียกใช้งานจากวัตถุ	
Post – Conditions	-	
Flow of Event	Banner Template	System
	1.สั่งใช้งาน Script Timer 3.ปิดใช้งาน Script Timer	2.นับเวลาภายในวัตถุ 4.หยุดนับเวลาภายในวัตถุ
Exception	-	

ตารางที่ 3-26 Use case Description : Unity game engine

Use Case Name	Unity game engine	
Actor	Game Developer	
Brief Description	Engine game สำหรับการใช้งานพัฒนาเกมของ Unity และสามารถถูกติดตั้ง plugin ได้	
Pre – Conditions	-	
Post – Conditions	-	
Flow of Event	User	System
	1.เปิดแถบ window 2.เลือก option package manager 3.เพิ่ม plugin ด้วย url	4.ติดตั้ง plugin
Exception	-	

ตารางที่ 3-27 Use case Description : Scene

Use Case Name	Scene	
Actor	Game Developer	
Brief Description	เป็นระบบส่วนหนึ่งของ Unity game engine มีหน้าที่ เป็นฉากสำหรับจัดวางวัตถุหลือทำสิ่งต่างๆภายในฉาก เช่น การตั้งค่า การจัดวาง การกำหนด เป็นต้น	
Pre – Conditions	-	
Post – Conditions	-	
Flow of Event	User	System
	1.สร้างไฟล์ scene	2.สร้างฉากสำหรับ editor ภายใน Unity game engine
Exception	-	

ตารางที่ 3-28 Use case Description : Camera

Use Case Name	Camera	
Actor	Game Developer	
Brief Description	เป็นระบบกล้องส่วนหนึ่งของ Unity game engine ไว้สำหรับแทนการมองเห็นของผู้ใช้งาน และจะถูกติดตั้ง Script เมื่อถูกเพิ่มเข้าไปใน Scene	
Pre – Conditions	จำเป็นต้องอยู่ภายใน Scene	
Post – Conditions	-	
Flow of Event	User	System
		1.ค้นหาวัตถุลักษณะแบบ Banner Template ภายในเฟรม 2.หากพบ Banner Template เริ่มตรวจสอบว่ามีการโฆษณาและลักษณะวัตถุถูกต้องหรือไม่ 3.เริ่มสั่งให้วัตถุที่มองเห็นนับเวลา
Exception	-	

ตารางที่ 3-29 Use case Description : Game objects

Use Case Name	Game objects	
Actor	Game Developer	
Brief Description	เป็นระบบส่วนหนึ่งของ Unity game engine มีหน้าที่ เป็นวัตถุภายในเกมที่ถูกจัดวางสำหรับการใช้งานกรณีต่างๆ อาจจะถูกติดตั้งด้วย Script หรืออื่นๆได้	
Pre – Conditions	จำเป็นต้องอยู่ภายใน Scene	
Post – Conditions	-	
Flow of Event	User	System
		1.ตรวจสอบลักษณะรูปแบบความเป็น Banner Template 2.หากพบเป็น Banner Template นำเข้าสู่ระบบของ plugin
Exception	-	

ตารางที่ 3-30 Use case Description : Add advertise video

Use Case Name	Add advertise video	
Actor	Advertiser	
Brief Description	มีหน้าที่สำหรับการเพิ่มโฆษณาให้กับผู้ใช้งาน	
Pre – Conditions	ผู้ใช้งานเข้าสู่ระบบ	
Post – Conditions	-	
Flow of Event	User	System
	1.กด Add advertise video 3.กรอกฟอร์มข้อมูลและกด submit	2.แสดงฟอร์มกรอกข้อมูลสำหรับการเพิ่มวีดีโอโฆษณา 4.เพิ่มวีดีโฆษณาเข้าสู่ระบบ
Exception	หากวีดีโฆษณาไม่เป็นประเภท mp4 และมีขนาดไม่เกิน 20MB จะไม่สามารถอัพโหลดได้	

ตารางที่ 3-31 Use case Description : Advertise list

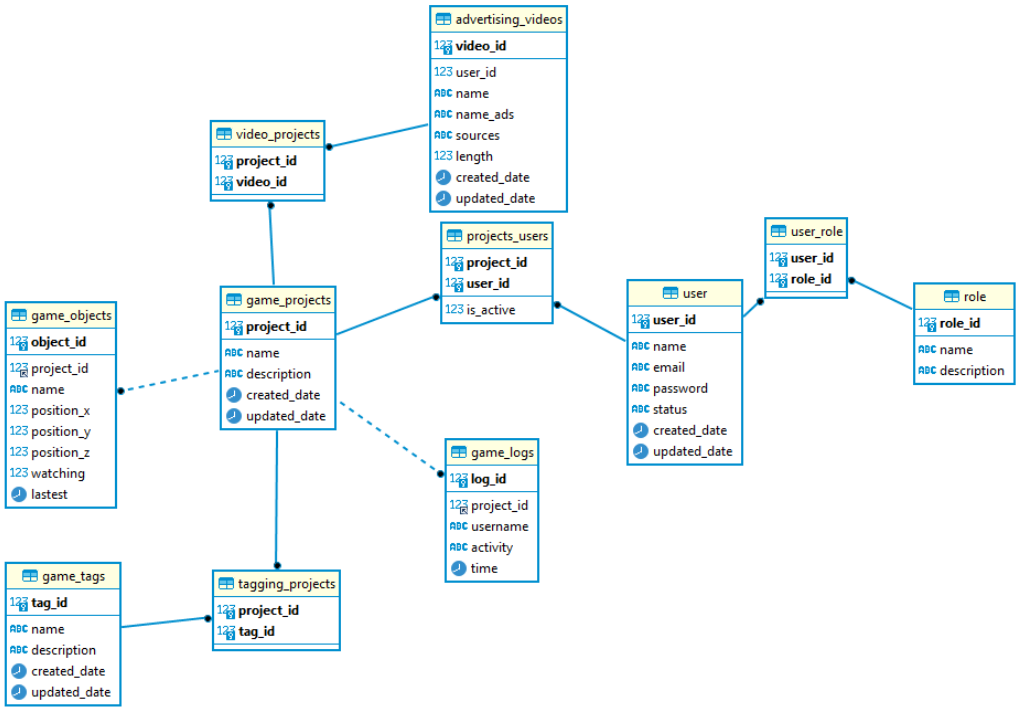
Use Case Name	Advertise list	
Actor	Advertiser	
Brief Description	มีหน้าที่สำหรับการแสดงผลข้อมูลวีดีโอโฆษณาทั้งหมด	
Pre – Conditions	-	
Post – Conditions	-	
Flow of Event	User	System
		1.แสดงผลข้อมูลวีดีโอโฆษณาทั้งหมด
Exception	หากไม่มีวีดีโอโฆษณา จะไม่มีข้อมูลวีดีโอให้แสดงผล	

ตารางที่ 3-32 Use case Description : Setting

Use Case Name	Setting	
Actor	Advertiser , Game Developer	
Brief Description	ระบบการตั้งค่าบทบาทสำหรับผู้ใช้งาน	
Pre – Conditions	ต้องมีบทบาทมากกว่า 1 บทบาท	
Post – Conditions	-	
Flow of Event	User	System
	1.กด setting role หรือ switch role 3.เลือกบทบาทที่ต้องการใช้งาน	2.แสดงรายการบทบาททั้งหมดของผู้ใช้งาน
Exception	-	

3.6 ER Diagram

ER Diagram แสดง Database ของระบบ ดังภาพ



ภาพที่ 3-6 ER Diagram

3.6 Data Dictionary

ตารางที่ 3-33 Data Dictionary : advertising_videos

advertising_videos จัดเก็บข้อมูลของวีดีโอสำหรับโฆษณา

Key	Column Name	Data Type	Nullable	Description	Reference
PK	video_id	Int(11)	NOT NULL	Running number	
	user_id	Int(11)	NOT NULL	รหัสผู้ใช้งาน	
	name	varchar(255)	NOT NULL	ชื่อไฟล์วีดีโอที่อัพโหลด	
	name_ads	varchar(255)	NOT NULL	ชื่อโฆษณาของวีดีโอ	
	sources	longtext	NOT NULL	ที่อยู่ URL วีดีโอ	
	length	float	NOT NULL	จำนวนเวลารับชมวีดีโอ	
	created_date	datetime	NOT NULL	วันที่สร้างวีดีโอ	
	updated_date	datetime	NOT NULL	วันเวลาที่อัปเดต	

ตารางที่ 3-34 Data Dictionary : game_logs

game_logs จัดเก็บข้อมูลกิจกรรมต่างๆของเกมโปรเจคที่พัฒนา

Key	Column Name	Data Type	Nullable	Description	Reference
PK	log_id	Int(11)	NOT NULL	Running number	
FK	project_id	Int(11)	NOT NULL	รหัสของโปรเจค	game_projects
	username	varchar(255)	NOT NULL	ชื่อผู้ใช้ของกิจกรรม	
	activity	varchar(255)	NOT NULL	ชื่อของกิจกรรม	

ตารางที่ 3-35 Data Dictionary : game_objects

game_objects จัดเก็บข้อมูลเกี่ยวกับวข้อมูลวัตถุของ plugin unity

Key	Column Name	Data Type	Nullable	Description	Reference
PK	object_id	Int(11)	NOT NULL	Running number	
FK	project_id	Int(11)	NOT NULL	รหัสของโปรเจค	game_projects
	name	varchar(255)	NOT NULL	ชื่อของวัตถุ	
	position_x	float	NULL	ตำแหน่งวัตถุแกน x	
	position_y	float	NULL	ตำแหน่งวัตถุของแกน y	
	position_z	float	NULL	ตำแหน่งวัตถุของแกน z	
	watching	bigint(20)	NULL	จำนวนเวลาที่รับชม	
	lastest	datetime	NULL	วันเวลาที่อัพเดต	

ตารางที่ 3-36 Data Dictionary : game_projects

game_projects จัดเก็บข้อมูลเกี่ยวกับวโปรเจคเกมที่พัฒนา

Key	Column Name	Data Type	Nullable	Description	Reference
PK	project_id	Int(11)	NOT NULL	Running number	
	name	varchar(255)	NOT NULL	ชื่อของโปรเจคเกม	
	description	varchar(255)	NULL	รายละเอียดเกี่ยวกับโปรเจคเกม	
	created_date	datetime	NOT NULL	วันเวลาที่สร้าง	
	updated_date	datetime	NOT NULL	วันเวลาที่อัปเดต	

ตารางที่ 3-37 Data Dictionary : game_tags

game_tags จัดเก็บข้อมูลเกี่ยวกับแท็กของประเภทเกม

Key	Column Name	Data Type	Nullable	Description	Reference
PK	tag_id	Int(11)	NOT NULL	Running number	
	name	varchar(255)	NOT NULL	ชื่อแท็กเกม	
	description	varchar(255)	NULL	รายละเอียดของแท็กเกม	
	created_date	datetime	NOT NULL	วันเวลาที่สร้าง	
	updated_date	datetime	NOT NULL	วันเวลาที่อัปเดต	

ตารางที่ 3-38 Data Dictionary : projects_users

projects_users จัดเก็บข้อมูลเกี่ยวกับระหว่างผู้ใช้งานและโปรเจคเกม

Key	Column Name	Data Type	Nullable	Description	Reference
PK, FK	project_id	Int(11)	NOT NULL	รหัสของโปรเจคเกม	game_projects
PK, FK	user_id	Int(11)	NOT NULL	รหัสของผู้ใช้งาน	user
	Is_active	tinyint(1)	NOT NULL	สถานะการใช้งาน	

ตารางที่ 3-39 Data Dictionary : role

role จัดเก็บข้อมูลเกี่ยวกับบทบาทต่างๆสำหรับการใช้งานภายในระบบ

Key	Column Name	Data Type	Nullable	Description	Reference
PK	role_id	Int(11)	NOT NULL	Running number	
	name	varchar(255)	NULL	ชื่อบทบาท	
	description	varchar(255)	NULL	รายละเอียดของบทบาท	

ตารางที่ 3-40 Data Dictionary : tagging_projects

tagging_projects จัดเก็บข้อมูลระหว่างข้อมูลแท็กและโปรเจคเกม

Key	Column Name	Data Type	Nullable	Description	Reference
PK, FK	project_id	Int(11)	NOT NULL	รหัสของโปรเจคเกม	game_projects
PK, FK	tag_id	Int(11)	NOT NULL	รหัสของแท็กเกม	game_tags

ตารางที่ 3-41 Data Dictionary : user

User จัดเก็บข้อมูลเกี่ยวกับผู้ใช้งานทั้งหมดภายในระบบ

Key	Column Name	Data Type	Nullable	Description	Reference
PK	user_id	Int(11)	NOT NULL	Running number	
	name	varchar(255)	NOT NULL	ชื่อของผู้ใช้งาน	
	email	varchar(255)	NOT NULL	อีเมลของผู้ใช้งาน	
	password	varchar(255)	NOT NULL	รหัสของผู้ใช้งาน	
	status	varchar(255)	NULL	สถานะของผู้ใช้งาน	
	created_date	datetime	NOT NULL	เวลาที่สร้างของผู้ใช้งาน	
	updated_date	datetime	NOT NULL	เวลาที่อัปเดตของผู้ช้งาน	

ตารางที่ 3-42 Data Dictionary : user_role

user_role จัดเก็บข้อมูลระหว่างผู้ใช้งานและบทบาทของผู้ใช้งาน

Key	Column Name	Data Type	Nullable	Description	Reference
PK, FK	user_id	Int(11)	NOT NULL	รหัสของผู้ใช้งาน	user
PK, FK	role_id	Int(11)	NOT NULL	รหัสของบทบาท	role

ตารางที่ 3-43 Data Dictionary : video_projects

video_projects จัดเก็บข้อมูลระหว่างวีดีโอโฆษณาและโปรเจคเกม

Key	Column Name	Data Type	Nullable	Description	Reference
PK, FK	project_id	Int(11)	NOT NULL	รหัสของโปรเจคเกม	game_projects
PK, FK	video_id	Int(11)	NOT NULL	รหัสของวีดีที่โฆษณา	advertising_videos

บทที่ 4
การพัฒนาระบบ

จากการศึกษาค้นคว้าข้อมูลที่เกี่ยวข้องกับการพัฒนาแอปพลิเคชันสามารถนำข้อมูลที่ได้มาประยุกต์ใช้และออกแบบการทำงานของระบบเพื่อใช้เป็นแนวทางในการพัฒนาระบบให้มีประสิทธิภาพ โดยมีรายละเอียดดังนี้

4.1 เครื่องมือที่ใช้ในการพัฒนาระบบ

4.1.1 เครื่องมือ

- Node version 16.7.13
- TypeScript version 4.4.2
- React version 18.2.0
- Ant Design version 5.0.3
- Ant Design Icon version 4.7.0
- Ant Design Charts version 1.4.2
- Axios version 1.1.3
- Spring Boot version 2.7.3
- Apache POI version 5.0.0
- MinIO version 8.2.1
- MariaDB Java Client version 2.1.2
- Unity version 2021.3

4.2 การพัฒนาระบบของโครงงาน

4.2.1 โค้ดของระบบปลั๊กอินภายใน Unity game engine โดย Editor เป็นส่วนหนึ่งของ Game engine ที่ผู้ใช้งานใช้สำหรับการพัฒนาเกม

4.2.1.1 การแสดงผลตัวเลือกของปลั๊กอินบน Editor โดยฟังก์ชั่นของ SignIn() และ ShowWindow() ถูกเรียกใช้งานตามคุณลักษณะของบรรทัดที่ 10 และ 13 ตามลำดับ จากฟังก์ชั่น ShowWindow() บรรทัดที่ 14 เป็นฟังก์ชั่นสำหรับการเรียกใช้ฟังก์ชั่นของ OnGUI() จะถูกใช้งานจากบรรทัดที่ 15 ถึง 17 มีหน้าที่แสดงหน้าต่างให้ผู้ใช้งานนำรหัสเชื่อมต่อโปรเจคเกมเข้าสู่ระบบปลั๊กอิน

```
6       [System.Serializable]
7       [CanEditMultipleObjects]
        Beeba * | IL code
8       public class FreeMasonEditor : EditorWindow {
9
10          [MenuItem("Freemason/Sign in")]
            Beeba * | IL code
11          private static void SignIn() { }
12
13          [MenuItem("Freemason/Projects")]
            Beeba | IL code
14          private static void ShowWindow  () {
15              var projects = GetWindow<FreeMasonEditor>();
16              projects.titleContent = new GUIContent("Projects");
17              projects.Show();
18          }
19
            Beeba * | IL code
20          private void OnGUI() {
21              GUI.backgroundColor = Color.red;
22              Projects.Id = EditorGUILayout.TextField("ID",Projects.Id);
23              Projects.Name = EditorGUILayout.TextField("Name",Projects.Name);
24
25              GUILayout.FlexibleSpace();
26              EditorGUILayout.BeginHorizontal();
27              GUILayout.Label("Reset form project.",EditorStyles.boldLabel);
28              if(GUILayout.Button(
29                      "Reset" ,
30                      GUILayout.Width(100),
31                      GUILayout.Height(30)
32                      ))
33              {
34                  Projects.Id = "0x2c4";
35                  Projects.Name = "Beeba";
36              }
37              EditorGUILayout.EndHorizontal();
38          }
39      }
```

ภาพที่ 4-1 ส่วนการแสดงผลตัวเลือกของปลั๊กอินบน Editor

4.2.1.2 ส่วนของการเก็บข้อมูลจากการกรอกข้อมูลของฟังก์ชั่นบรรทัดที่ 20 จากรูปภาพที่ 4-1 ใช้สำหรับเก็บค่าคงที่ ที่สามารถถูกเปลี่ยนแปลงได้และสามารถถูกเรียกใช้งานจากคลาสอื่นๆได้ ขณะเปลี่ยนแปลง

```
41      public class Projects {
            Beeba  IL code
42          public static string Id {
43              get {
44                  #if UNITY_EDITOR
45                  return EditorPrefs.GetString("ID","0x2c4");
46                  #else
47                  return false;
48                  #endif
49              }
50              set {
51                  #if UNITY_EDITOR
52                  EditorPrefs.SetString("ID",value);
53                  #endif
54              }
55          }
56
            Beeba  IL code
57          public static string Name {
58              get {
59                  #if UNITY_EDITOR
60                  return EditorPrefs.GetString("Name","Beeba");
61                  #else
62                  return false;
63                  #endif
64              }
65              set {
66                  #if UNITY_EDITOR
67                  EditorPrefs.SetString("Name",value);
68                  #endif
69              }
70          }
71      }
```

ภาพที่ 4-2 ส่วนของการเก็บค่าคงที่และถูกเรียกใช้งานจากคลาสอื่นๆ

4.2.1.3 ส่วนการแปลงวัตถุทั้งหมดที่เป็นของปลั๊กอินภายใน Game Engine เปลี่ยนการแสดงผลบนพื้นผิวของวัตถุให้เป็นวีดีโอและตั้งค่าเกี่ยวกับวีดีโอที่แสดงผลทั้งหมด

```
100    public class AdvertisePlayer {
101        private VideoPlayer videoPlayer;
102        private AdvertisementPrefabs target;
103
104        public VideoPlayer Video { get => (videoPlayer); }
105        public bool IsPlaying { get => (videoPlayer.isPlaying); }
106
107        public AdvertisePlayer(AdvertisementPrefabs prefab) {
108            this.target = prefab;
109            this.videoPlayer = target.gameObject.AddComponent<VideoPlayer>();
110        }
111
112        public IEnumerator Advertising(AdvertiseVideo video) {
113            videoPlayer.playOnAwake = false;
114            videoPlayer.isLooping = true;
115            videoPlayer.source = VideoSource.Url;
116            videoPlayer.url = video.sources;
117            videoPlayer.targetTexture = RenderOnTexture();
118            videoPlayer.SetDirectAudioMute( (ushort)0, true );
119            videoPlayer.Prepare();
120            while(!videoPlayer.isPrepared) {
121                yield return new WaitForSeconds(2);
122                break;
123            }
124            videoPlayer.Play();
125            while (videoPlayer.isPlaying) {
126                yield return null;
127            }
128        }
129
130        private RenderTexture RenderOnTexture() {
131            RenderTexture renderTexture = new RenderTexture(1280,720,0,RenderTextureFormat.ARGB32);
132            renderTexture.Create();
133            if(videoPlayer.targetTexture != null) {
134                videoPlayer.targetTexture.Release();
135            }
136            return renderTexture;
137        }
138    }
```

ภาพที่ 4-3 ส่วนของการแปลงพื้นผิววัตถุให้เป็นวีดีโอของวัตถุทั้งหมดภายในปลั๊กอิน

4.2.1.4 ส่วนการควบคุมวีดีโอและกำหนดวัตถุทั้งหมดภายในปลั๊กอิน มีการเรียกใช้คลาสและวัตถุต่างๆในการตรวจสอบรวมถึงการใช้งาน จากภาพ 4-4.1 เป็นการประกาศค่าตัวแปรต่างและเรียกใช้คลาสภายในปลั๊กอิน เริ่มต้นบรรทัดที่ 142 ถึง 152 และมีฟังก์ชั่น SetStartAdvertising() จากบรรทัดที่ 161 เป็นการเรียกข้อมูลวีดีโอโฆษณาทั้งหมดจากคลาส VideoController และเข้าสู่ฟังก์ชั่น Marketing() บรรทัดที่ 167 เป็นฟังก์ชั่นจัดการโฆษณาให้กับวัตถุทั้งหมดภายในปลั๊กอิน ซึ่งจำเป็นต้องอาศัย ฟังก์ชั่น AroundTarget(...) เป็นฟังก์ชั่นในการตรวจสอบว่าวัตถุ 1 วัตถุมีวัตถุที่เป็นลักษณะเดียวกันห่างในระยะที่กำหนดหรือไม่ เป็นฟังก์ชั่นที่ช่วยในการจัดการโฆษณา และฟังก์ชั่น GroupManage(...) เป็นฟังก์ชั่นในการแบ่งกลุ่มของวัตถุทั้งหมดภายในปลั๊กอินช่วยในการจัดการโฆษณา

```
140    public class AdvertiseController
141    {
142        private Advertisements advertisements;
143        private VideoController videoController;
144        private List<List<AdvertisementPrefabs>> groupGroups;
145        private List<AdvertisementPrefabs> targets;
146        private float limitRadius = 100f;
147        private int limitNearby = 2;
148        private int groupAdvertise = 1;
149        private int countTarget = 0;
150        private bool isAlertNearby = false;
151
152        public VideoController VideoController { get => (videoController); }
153
154        public AdvertiseController (Advertisements ads) {
155            this.advertisements = ads;
156            this.targets = advertisements.Targets;
157            this.videoController = new VideoController();
158
159        }
160
161        public IEnumerator SetStartAdvertising() {
162            yield return videoController.FindVideos();
163            Marketing();
164            yield return null;
165        }
```

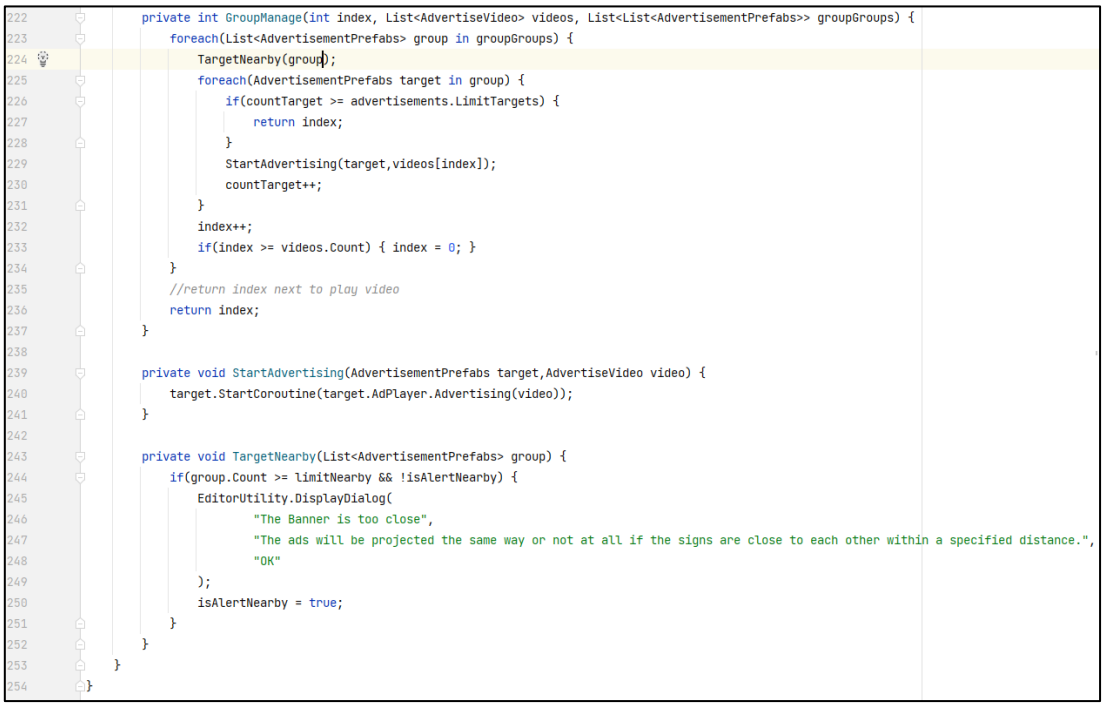
ภาพที่ 4-4.1 การประกาศค่าตัวแปรต่าง ๆ บนคลาส

```
167     private void Marketing() {
168         int i = 0;
169         int remaining = 0;
170         List<AdvertiseVideo> videos = advertisements.AdvertiseController.VideoController.Videos;
171         List<AdvertisementPrefabs> around;
172         this.groupGroups = new List<List<AdvertisementPrefabs>>();
173         foreach(AdvertisementPrefabs target in targets) {
174             if(target.Information.groupAdvertise <= 0){
175                 around = AroundTarget(target, limitRadius);
176                 if(around.Count > 0) {
177                     groupGroups.Add(around);
178                     groupAdvertise++;
179                 }
180             }
181         }
182         i = GroupManage(i,videos,groupGroups);
183         foreach(AdvertisementPrefabs target in targets) {
184             if(countTarget >= advertisements.LimitTargets){
185                 break;
186             }
187             if(i > videos.Count-1 ) {
188                 i = 0;
189             }
190             if(target.Information.groupAdvertise <= remaining) {
191                 StartAdvertising(target,videos[i]);
192                 countTarget++;
193             }
194             i++;
195         }
196     }
```

ภาพที่ 4-4.2 ฟังก์ชั่นวิธีการจัดการโฆษณาให้กับวัตถุทั้งหมด

```
199     private List<AdvertisementPrefabs> AroundTarget(AdvertisementPrefabs mainTarget, float limit) {
200         float distance;
201         int groupAd = (int)groupAdvertise;
202         AdvertisementPrefabs tempTarget  = targets.Find(x => (x.Information.instanceId == mainTarget.Information.instanceId));
203         List<AdvertisementPrefabs> around = new List<AdvertisementPrefabs>();
204 
205         foreach(AdvertisementPrefabs target in targets) {
206             if(mainTarget.Information.instanceId != target.Information.instanceId && target.Information.groupAdvertise <= 0) {
207                 distance = Vector3.Distance(target.Information.position ,mainTarget.Information.position);
208 
209                 if(distance <= limit ) {
210                     around.Add(target);
211                     mainTarget.Information.groupAdvertise = groupAd;
212                     target.Information.groupAdvertise = groupAd;
213                 }
214             }
215         }
216         if(mainTarget.Information.groupAdvertise > 0) {
217             around.Add(mainTarget);
218         }
219         return around;
220     }
```

ภาพที่ 4-4.3 ฟังก์ชั่นการตรวจสอบวัตถุโดยรอบ



```
222         private int GroupManage(int index, List<AdvertiseVideo> videos, List<List<AdvertisementPrefabs>> groupGroups) {
223             foreach(List<AdvertisementPrefabs> group in groupGroups) {
224                 TargetNearby(group);
225                 foreach(AdvertisementPrefabs target in group) {
226                     if(countTarget >= advertisements.LimitTargets) {
227                         return index;
228                     }
229                     StartAdvertising(target,videos[index]);
230                     countTarget++;
231                 }
232                 index++;
233                 if(index >= videos.Count) { index = 0; }
234             }
235             //return index next to play video
236             return index;
237         }
238 
239         private void StartAdvertising(AdvertisementPrefabs target,AdvertiseVideo video) {
240             target.StartCoroutine(target.AdPlayer.Advertising(video));
241         }
242 
243         private void TargetNearby(List<AdvertisementPrefabs> group) {
244             if(group.Count >= limitNearby && !isAlertNearby) {
245                 EditorUtility.DisplayDialog(
246                         "The Banner is too close",
247                         "The ads will be projected the same way or not at all if the signs are close to each other within a specified distance.",
248                         "OK"
249                 );
250                 isAlertNearby = true;
251             }
252         }
253     }
254 }
```

ภาพที่ 4-4.4 ฟังก์ชั่นจัดการวัตถุด้วยการแบ่งกลุ่ม

4.2.1.5 ส่วนที่เป็นศูนย์กลางของปลั๊กอินในการรับข้อมูลวัตถุทั้งหมดรับรหัสการเชื่อมต่อกับโปรเจคเกมสั่งใช้งานให้วัตถุแสดงผลวีดีโอ ตรวจสอบวัตถุ และส่งข้อมูลให้กับเซิฟเวอร์ ซึ่งตามวัฏจักรชีวิตของคลาสที่เป็นศูนย์กลางของปลั๊กอินจะเข้าสู่ฟังก์ชั่น

4.2.1.5.1 Awake() ในช่วงเวลาที่ติดตั้งการโฆษณาเข้าไปในฉากจึงเริ่มการกำหนดเรียกใช้งานจากคลาสต่าง ๆ รวมถึงเปิดรับข้อมูลวัตถุที่จะส่งข้อมูลเข้ามาให้กับคลาสนี้

4.2.1.5.2 OnEnable() ทุกครั้งที่มีการโหลดคลาสนี้เข้าจะเริ่มสั่งใช้งานการควบคุมโฆษณา

4.2.1.5.3 Start() ในช่วงเวลาเริ่มเกมจะสั่งใช้งานฟังก์ชั่นบรรทัดที่ 84 จากภาพ 4-5.3 เพื่อส่งข้อมูลไปยังเซิฟเวอร์ทุก ๆ 60 วินาที

4.2.1.5.4 Update() ในช่วงเวลาภายในเกมตรวจสอบว่ามีวัตถุภายในปลั๊กอินเกินที่กำหนดหรือไม่ หากเกินกำหนดเริ่มใช้ฟังก์ชั่นบรรทัดที่ 76 จากภาพที่ 4-5.2

4.2.1.5.5 onDisable() เมื่อไม่มีการใช้งานปลั๊กอินภายในเกมจะหยุดการใช้งานจัดการโฆษณา

ฟังก์ชั่นเสริมในการให้สคริปต์ภายในวัตถุตัวอย่างถูกเพิ่มเข้ามาในปลั๊กอิน คือ ฟังก์ชั่น OnTarget() บรรทัดที่ 65 และถอนออกจากปลั๊กอิน คือฟังก์ชั่น NotTarget() บรรทัด 72 จากภาพที่ 4-5.2

```
15  public class Advertisements : MonoBehaviour
16  {
17      private List<AdvertisementPrefabs> targets;
18      private AdvertiseController advertiseController;
19      private CameraController cameraController;
20      private PrefabController prefabController;
21      private int project;
22      private bool isAlert;
23
24      public List<AdvertisementPrefabs> Targets { get => (targets); }
25      public AdvertiseController AdvertiseController { get => (advertiseController); }
26      public int LimitTargets { get => (18); }
27      public bool isOverTargets { get => (targets.Count > LimitTargets); }
28
29      private static Advertisements instance;
30
31      private void Awake() {
32          Application.runInBackground = true;
33          instance = this;
34          int startIndex = 16;
35          int endIndex = Projects.Id.Length - startIndex;
36          string title = Projects.Id.Substring(startIndex,endIndex);
37          project = Int32.Parse(title);
38          targets = new List<AdvertisementPrefabs>();
39          advertiseController = new AdvertiseController(this);
40          cameraController = new CameraController();
41          prefabController = new PrefabController(project);
42      }
43
44      private async void OnEnable() {
45          isAlert = false;
46          await Task.Delay(TimeSpan.FromSeconds(0.2));
47          StartCoroutine(advertiseController.SetStartAdvertising());
48      }
49
```

ภาพที่ 4-5.1 การประกาศค่าตัวแปรต่าง ๆ บนคลาสและการรับข้อมูลตามวัฏจักรชีวิต

```
50      private void Start() {
51          InvokeRepeating("PostAPI", 60, 60);
52      }
53
54      private void Update() {
55          if(isOverTargets && !isAlert) {
56              LimitPrefabs();
57              isAlert = true;
58          }
59      }
60
61      private void OnDisable() {
62          StopCoroutine("advertiseController.SetStartAdvertising()");
63      }
64
65      public static async void  OnTarget(AdvertisementPrefabs target) {
66          await Task.Delay(TimeSpan.FromSeconds(0.2));
67          if(target.enabled){
68              instance.targets.Add(target);
69          }
70      }
71
72      public static void NotTarget(AdvertisementPrefabs target) {
73          instance.targets.Remove(target);
74      }
75
76      private void LimitPrefabs() {
77          EditorUtility.DisplayDialog(
78              "The number of billboards exceeded the limit",
79              "You have exceeded the specified amount. Those banners will not be advertised.",
80              "OK"
81          );
82      }
83
```

ภาพที่ 4-5.2 ตามกฎวัฏจักรชีวิตของ Game engine และมีฟังก์ชั่นใช้สำหรับการเพิ่มลบวัตถุภายในระบบปลั๊กอิน รวมถึงฟังก์ชั่นเสริมในการแจ้งเตือน

```
83
84      private void PostAPI() {
85          StopCoroutine("prefabController.Post()");
86          prefabController.listGameObject = targets;
87          StartCoroutine(prefabController.Post());
88      }
89
90      public static bool IsView(AdvertisementPrefabs target) {
91          List<bool> isValue = new List<bool>();
92          foreach(Camera cam in instance.cameraController.Cameras) {
93              isValue.Add(instance.cameraController.IsInView(cam,target.gameObject));
94          }
95          return isValue.Find( value => (value == true) );
96      }
97
98  }
```

ภาพที่ 4-5.3 ฟังก์ชั่นสำหรับการสั่งใช้งานให้ส่งข้อมูลไปยังเซิฟเวอร์และฟังก์ชั่นเสริมในการมองเห็น

4.2.1.6 สคริปต์ที่ถูกบันทุกภายในวัตถุจากปลั๊กอินใช้กฎตามวัฏจักรชีวิตของ Game engine และกำหนดค่าตัวแปรต่าง ๆ

4.2.1.6.1 OnEnable() บรรทัดที่ 25 ภาพที่ 4-6.1 เมื่อมีการใช้งานวัตถุจะกำหนดข้อมูลของวัตถุและเพิ่มเครื่องเล่นโฆษณา เครื่องนับเวลา บนวัตถุ รวมถึงการเพิ่มข้อมูลวัตถุเข้าสู่ปลั๊กอิน

4.2.1.6.2 OnDisable() บรรทัดที่ 40 ภาพที่ 4-6.1 เมื่อวัตถุไม่ถูกใช้งานจะถอนข้อมูลวัตถุออกจากปลั๊กอิน

4.2.1.6.3 Update() บรรทัดที่ 46 ภาพที่ 4-6.2 เมื่ออยู่ภายในเกมจะตรวจสอบการมองเห็นวัตถุ หากมองเห็นจะเริ่มใช้งานเครื่องนับเวลา รวมถึงการเปลี่ยนแปลงข้อมูลตำแหน่งและเวลาของวัตถุ

4.2.1.6.4 OnBecameVisible() บรรทัดที่ 55 ภาพที่ 4-6.2 เมื่อมีกล้องใดมองเห็นวัตถุให้กำหนดค่าการมองเห็นภายในวัตถุและสั่งใช้งานฟังก์ชั่นภายในปลั๊กอินตรวจสอบการมองเห็นจากกล้องภายในปลั๊กอินที่ได้รับ

4.2.1.6.5 OnBecameInvisible() บรรทัดที่ 60 ภาพที่ 4-6.2 เมื่อไม่มีกล้องใดมองเห็นวัตถุให้กำหนดค่าการมองเห็นภายในวัตถุและสั่งปิดการใช้งานเครื่องนับเวลา

```
public class AdvertisementPrefabs : MonoBehaviour
{
    private AdvertisePlayer adPlayer;
    private InformationPrefab information;
    private TimerPrefab timer;
    private bool isVisible;

    public AdvertisePlayer AdPlayer { get => (adPlayer); }
    public InformationPrefab Information { get => (information); }
    public TimerPrefab Timer { get => (timer); }

    private void Awake() {
        Application.runInBackground = true;
    }

    private void OnEnable() {
        isVisible = false;
        TimeSpan currentTime = new TimeSpan();
        information = new InformationPrefab(
            gameObject.GetInstanceID(),
            gameObject.name,
            gameObject.transform.position,
            currentTime,
            currentTime
        );
        adPlayer = new AdvertisePlayer(this);
        timer = new TimerPrefab(this);
        Advertisements.OnTarget(this);
    }

    private void OnDisable() {
        Advertisements.NotTarget(this);
    }
```

ภาพที่ 4-6.1 สคริปต์ที่ถูกบันทึกภายในวัตถุของปลั๊กอิน

```
    private void Update()
    {
        if(isVisible && adPlayer.IsPlaying) {
            timer.BeginTimer();
        }
        information.position = gameObject.transform.position;
        information.watched = timer.watched;
    }

    private void OnBecameVisible() {
        isVisible = true;
        Advertisements.IsView(this);
    }

    private void OnBecameInvisible() {
        isVisible = false;
        timer.EndTimer();
    }
}
}
```

ภาพที่ 4-6.2 ตามกฎวัฏจักรชีวิตของ Game engine

4.2.1.7 เครื่องจับเวลาจะถูกเรียกใช้งานจากวัตถุจาก ฟังก์ชั่น BeginTimer() บรรทัดที่ 28 ภาพที่ 4-7.1 จะทำการกำหนดค่าภายในและใช้งานฟังก์ชั่น UpdateTimer() บรรทัดที่ 40 จากภาพที่ 4-7.2 เป็นฟังก์ชั่นเริ่มนับเวลาเพิ่มขึ้นเรื่อย ๆ และมีฟังก์ชั่น EndTimer() บรรทัดที่ 34 ภาพที่ 4-7.1 ถูกเรียกใช้เมื่อต้องการหยุดนับเวลา ฟังก์ชั่นเสริม UpdatePost() บรรทัดที่ 49 และ ToPosting() บรรทัดที่ 57 จากภาพที่ 4-7.2 เป็นฟังก์ชั่นในการส่งข้อมูลเวลาที่นับได้ให้กับวัตถุ

```
8   public class TimerPrefab
9   {
10      private float elapsed;
11      private bool isTimerGoing;
12      private AdvertisementPrefabs prefab;
13      private static TimerPrefab instance;
14
15      public TimeSpan timePlaying { get; set; }
16      public TimeSpan watched { get; set; }
17      public AdvertisementPrefabs advertisementPrefab { get => (prefab); }
18      public bool isGoing { get => (isTimerGoing); }
19      private bool isPosting { get; set; }
20
21      public TimerPrefab(AdvertisementPrefabs _prefab) {
22          instance = this;
23          prefab = _prefab;
24          watched = new TimeSpan();
25          isPosting = false;
26      }
27
28      public void BeginTimer() {
29          isTimerGoing = true;
30          elapsed = 0f;
31          prefab.StartCoroutine(UpdateTimer());
32      }
33
34      public void EndTimer() {
35          isTimerGoing = false;
36          watched += timePlaying;
37          prefab.StopCoroutine("UpdateTimer()");
38      }
```

ภาพที่ 4-7.1 เครื่องจับเวลาภายในวัตถุ

```
40          private IEnumerator UpdateTimer() {
41              while(isTimerGoing)
42              {
43                  elapsed += Time.deltaTime;
44                  timePlaying = TimeSpan.FromSeconds(elapsed);
45                  yield return null;
46              }
47          }
48
49          private IEnumerator UpdatePost() {
50              while(isPosting) {
51                  timePlaying = new TimeSpan();
52                  watched = new TimeSpan();
53                  yield return null;
54              }
55          }
56
57          public static void ToPosting() {
58              instance.watched = new TimeSpan();
59          }
60      }
61  }
```

ภาพที่ 4-7.2 เครื่องจับเวลาภายในวัตถุ

4.2.1.8 ส่วนของการรับข้อมูลวีดีโอโฆษณาและส่งข้อมูลให้กับส่วนอื่นๆภายในปลั๊กอิน

```
11  public class VideoController
12  {
13      private string allVideos = "https://server.beeba.ml/application/advertise_video/project/";
14      private List<AdvertiseVideo> videos;
15
16      public List<AdvertiseVideo> Videos { get => (videos); }
17
18      public IEnumerator FindVideos() {
19          int startIndex = 16;
20          int endIndex = Projects.Id.Length - startIndex;
21          string title = Projects.Id.Substring(startIndex,endIndex);
22          int project = Int32.Parse(title);
23          UnityWebRequest request = UnityWebRequest.Get(allVideos + project);
24          yield return request.SendWebRequest();
25          if (request.isNetworkError || request.isHttpError)
26          {
27              Debug.Log(request.error);
28          }
29          else
30          {
31              List<AdvertisingVideoModel> videoModel = JsonConvert.DeserializeObject<List<AdvertisingVideoModel>>(request.downloadHandler.text);
32              videos = new List<AdvertiseVideo>();
33              foreach(AdvertisingVideoModel videoObj in videoModel) {
34                  videos.Add(new AdvertiseVideo(videoObj.video.id,videoObj.video.name,videoObj.video.sources));
35              }
36          }
37      }
38  }
```

ภาพที่ 4-8 ส่วนของการรับข้อมูลวีดีโอโฆษณา

4.2.1.9 ส่วนการตรวจสอบระหว่างวัตถุและกล้องใช้สำหรับการตรวจสอบความถูกต้องในการมองเห็นสามารถเพิ่มข้อมูลหรือลักษณะวิธีการมองเห็นของวัตถุได้

```
40  public class CameraController
41  {
42      private List<Camera> cameras;
43
44      public List<Camera> Cameras { get => (cameras); }
45
46      public CameraController() {
47          cameras = new  List<Camera>();
48          cameras.AddRange(Camera.allCameras);
49      }
50
51      public bool IsInView(Camera cam, GameObject toCheck) {
52          Vector3 pointOnScreen = cam.WorldToScreenPoint(toCheck.GetComponentInChildren<Renderer>().bounds.center);
53          if(pointOnScreen.z < 0) {
54              return false;
55          }
56
57          //Is in FOV
58          if( (pointOnScreen.x < 0) || (pointOnScreen.x > Screen.width) ||
59              (pointOnScreen.y < 0) || (pointOnScreen.y > Screen.height) ) {
60              return false;
61          }
62          return true;
63      }
64  }
```

ภาพที่ 4-9 ส่วนการตรวจสอบระหว่างวัตถุและกล้อง

4.2.1.10 แบบอย่างการส่งข้อมูลไปยังเซิฟเวอร์ เป็นต้นแบบของข้อมูลที่ส่งไปยังเซิฟเวอร์เพื่อความถูกต้องและสะดวกต่อการเรียกใช้งาน

```
16      [Serializable]
17      public class GameObjectForm {
18          public int id;
19          public string name;
20          public double positionX;
21          public double positionY;
22          public double positionZ;
23          public DateTime watching;
24
25          public GameObjectForm(
26              int Id,
27              string Name,
28              double PositionX,
29              double PositionY,
30              double PositionZ,
31              TimeSpan Watched
32          ) {
33              id = Id;
34              name = Name;
35              positionX = PositionX;
36              positionY = PositionY;
37              positionZ = PositionZ;
38              watching = DateTime.Today + Watched;
39              // watching.Add(Watched);
40          }
41      }
```

ภาพที่ 4-10 แบบอย่างการส่งข้อมูลไปยังเซิฟเวอร์

4.2.1.11 ส่วนการส่งข้อมูลไปยังเซิฟเวอร์นำข้อมูลวัตถุทั้งหมดจากปลั๊กอินนำมาทำแบบอย่าง 4.2.1.10 และกำหนดค่าในการส่งข้อมูลไปยังเซิฟเวอร์



```
public class PrefabController
{
    private int projectRepository;
    public List<AdvertisementPrefabs> listGameObject { get; set; }

    public PrefabController(int id) {
        projectRepository = id;
    }

    public IEnumerator Post() {
        Freemason.Timer.Model.TimerPrefab.ToPosting();
        List<GameObjectForm> forms = new List<GameObjectForm>();
        foreach(AdvertisementPrefabs obj in listGameObject) {
            forms.Add(new GameObjectForm(
                obj.GetInstanceID(),
                obj.gameObject.name,
                obj.Information.position.x,
                obj.Information.position.y,
                obj.Information.position.z,
                obj.Information.watched
            ));
        }
        GameObjectForm[] arrayForms = forms.ToArray();
        string json = JsonConvert.SerializeObject(arrayForms);
        JsonSerializerSettings microsoftDateFormatSettings = new JsonSerializerSettings();
        string microsoftJson = JsonConvert.SerializeObject(arrayForms, microsoftDateFormatSettings);
        string url = ("https://server.beeba.ml/plugin/project/"+ projectRepository +"/objects/save");
        UnityWebRequest request = new UnityWebRequest();
        request.url = url;
        request.method = UnityWebRequest.kHttpVerbPOST;
        request.SetRequestHeader("Content-Type", "application/json; charset=UTF-8");
        request.SetRequestHeader("Accept", "application/json");
        request.SetRequestHeader("Accept-Encoding", "gzip, deflate, br");
        request.downloadHandler = new DownloadHandlerBuffer();
        request.uploadHandler = new UploadHandlerRaw(string.IsNullOrEmpty(microsoftJson) ? null : Encoding.UTF8.GetBytes(microsoftJson));
        Debug.Log(request.uploadHandler);
```

ภาพที่ 4-11.1 ส่วนการส่งข้อมูลไปยังเซิฟเวอร์



```
            Debug.Log(request.uploadHandler);
            yield return request.SendWebRequest();
            if(request.result != UnityWebRequest.Result.Success) {
                Debug.Log(request.downloadHandler.text);
                Debug.Log(request.error);
            }
            else {
                string results = request.downloadHandler.text;
                Debug.Log(results);
            }
            request.Dispose();
            yield return null;
        }

        TimeSpan DateTimeToTimeSpan(DateTime? ts)
        {
            if (!ts.HasValue) return TimeSpan.Zero;
            else return new TimeSpan(0, ts.Value.Hour, ts.Value.Minute, ts.Value.Second, ts.Value.Millisecond);
        }
    }
}
```

ภาพที่ 4-11.2 ส่วนการส่งข้อมูลไปยังเซิฟเวอร์

4.2.1.12 แบบอย่างข้อมูลภายในปลั๊กอินใช้ในเป็นแบบอย่างข้อมูล ใช้ในการตรวจสอบความถูกต้องของข้อมูลและสะดวกในการเรียกใช้งาน

```
8   public class AdvertisingVideoModel {
9       public GameAdsModel project { get; set; }
10      public VideoAdsModel video { get; set; }
11  }
12
13  public class GameAdsModel {
14      public int id { get; set; }
15      public string description { get; set; }
16      public string name { get; set; }
17  }
18
19  public class VideoAdsModel {
20      public int id { get;set;}
21      public string name { get; set; }
22      public string nameAds { get; set; }
23      public string sources { get; set; }
24  }
```

ภาพที่ 4-12.1 แบบอย่างข้อมูลภายในปลั๊กอิน

```
26      public class AdvertiseVideo {
27          private int _id;
28          private string _name;
29          private string _source;
30          private double _length;
31
32          public AdvertiseVideo (
33              int videoId,
34              string videoName,
35              string videoSource
36          ) {
37              _id = videoId;
38              _name = videoName;
39              _source = videoSource;
40          }
41
42          public int id {
43              get => (_id);
44              set {_id = value;}
45          }
46          public string name {
47              get => (_name);
48              set {_name = value;}
49          }
50          public string sources {
51              get => (_source);
52              set {_source = value;}
53          }
54          public double length {
55              get => (_length);
56              set {_length = value;}
57          }
58      }
```

ภาพที่ 4-12.2 แบบอย่างข้อมูลภายในปลั๊กอิน

```
63      public class InformationPrefab
64      {
65          public int instanceId { get; set; }
66          public int groupAdvertise { get; set; }
67          public string name { get; set; }
68          public Vector3 position;
69          public TimeSpan watched { get;set; }
70          public TimeSpan lastest { get;set; }
71
72          public InformationPrefab (
73              int _intanceId,
74              string _name,
75              Vector3 _position,
76              TimeSpan _watched,
77              TimeSpan _lastest
78          ) {
79              instanceId = _intanceId;
80              name = _name;
81              position = _position;
82              watched = _watched;
83              lastest = _lastest;
84              groupAdvertise = 0;
85          }
86
87          public void Log () {
88              Debug.Log(
89                  "intanceId: " + instanceId +"\n" +
90                  "name: " + name +"\n" +
91                  "watched: " + watched +"\n" +
92                  "groupAdvertise: "+ groupAdvertise + "\n"+
93                  "{\n x: " + position.x +"\n" +
94                  " y: " + position.y +"\n" +
95                  " z: " + position.z +"\n}"
96              );
97          }
98      }
```

ภาพที่ 4-12.3 แบบอย่างข้อมูลภายในปลั๊กอิน

4.2.2 โค้ดของการแสดงผลบนเว็บแอปพลิเคชัน

4.2.2.1 ส่วนของการเข้าสู่ระบบของเว็บแอปพลิเคชัน ฟังก์ชั่นการเข้าสู่ระบบคือ onFinish() บรรทัดที่ 31 ภาพที่ 4-13.1 เป็นการรับค่าจะเซิฟเวอร์และตรวจสอบจำนวนบทบาทของผู้ใช้งานและให้ผู้ใช้งานเข้าสู่ส่วนที่กำหนด และมีฟังก์ชั่นสำหรับการเรียกใช้งานการสมัครของผู้ใช้งานจากภาพที่ 4-13.2

```
23  const Login = () => {
24      const [form] = Form.useForm()
25      const navigate = useNavigate()
26      const [error, setError] = useState()
27      const [username, setUsername] = useState<string>( initialState: '')
28      const [password, setPassword] = useState<string>( initialState: '')
29      const [openDrawer, setOpenDrawer] = useState<boolean>( initialState: false)
30
31      const onFinish = async () => {
32          try {
33              const {data} = await axios.post( url: "/login",  data: {name: username, password: password})
34              localStorage.setItem("access_token", data.access_token)
35              const user: User | null = await updateUser(navigate)
36              form.resetFields()
37              if (user && user?.roles) {
38                  if (user.roles.length > 1) {
39                      navigate( to: '/application')
40                  } else {
41                      if (user.roles.find((value :Role ) => value.name === "CUSTOMER_ADVERTISING")) {
42                          navigate( to: '/application/customers')
43                      }
44                      if (user.roles.find((value :Role ) => value.name === "GAME_DEVELOPER")) {
45                          navigate( to: '/application/games')
46                      }
47                  }
48
49              }
50          } catch (error: AxiosError | any) {
51              if (error?.response?.status === 401) {
52                  message.error( content: "Incorrect username or password")
53              } else {
54                  message.error( content: "Something went wrong!")
55              }
56              setError(error)
57          }
58      }
```

ภาพที่ 4-13.1 ส่วนของการเข้าสู่ระบบ

```
60      useEffect( effect: () => {
61          if (localStorage.getItem( key: 'authToken')) {
62              navigate( to: '/')
63          }
64      }, deps: [navigate])
65
66      return (
67          <div className={"login-page"}>
68              <div className={"container"}>
69                  <div className={"forms-container"}...>
147                 <div className="panels-container"...>
164                 <DrawerRegister open={openDrawer} setOpen={setOpenDrawer}/>
165                 <FloatButton.Group icon={<MenuOutlined/>} type="primary" trigger="click">
166                     <FloatButton icon={<AppstoreOutlined/>} onClick={() => {
167                         navigate( to: "/")
168                     }}/>
169                     <FloatButton icon={<FormOutlined/>} onClick={() => {
170                         setOpenDrawer( value: true)
171                     }}/>
172                 </FloatButton.Group>
173             </div>
174
175             // </div>
176     )
177 }
```

ภาพที่ 4-13.2 ส่วนของการเข้าสู่ระบบและเรียกใช้ฟังก์ชั่นการสมัคร

4.2.2.2 แบบฟอร์มการสมัครเข้าใช้งานเว็บแอปพลิเคชัน

```
8   const SignIn = () => {
9       const [form] = Form.useForm()
10      const navigate = useNavigate()
11      const [error, setError] = useState()
12      const [username, setUsername] = useState<string>( initialState: '')
13      const [password, setPassword] = useState<string>( initialState: '')
14
15      const onFinish = async () => {
16          try {
17              const {data} = await axios.post( url: "/login",  data: {name: username, password: password})
18              localStorage.setItem("access_token", data.access_token)
19              form.resetFields()
20              navigate( to: '/application')
21          } catch (error: AxiosError | any) {
22              if(error?.response?.status === 401) {
23                  message.error( content: "Incorrect username or password")
24              } else {
25                  message.error( content: "Something went wrong!")
26              }
27              setError(error)
28          }
29      }
30
31      const onFinishFailed = (errorInfo: unknown) => {...}
34
35      return (
36          <>
37              <Form
38                  name={"sign-in"}
39                  form={form}
40                  labelCol={{span: 8}}
41                  initialValues={{remember: false}}
42                  onFinish={onFinish}
43                  onFinishFailed={onFinishFailed}
44                  autoComplete={"off"}
45                  ...>
96              </>
97      )
98  }
```

ภาพที่ 4-14 แบบฟอร์มการสมัครเข้าใช้งานเว็บแอปพลิเคชัน

4.2.2.3 หน้าแรกสำหรับการแสดงผลให้ผู้ใช้งานประเภทนักโฆษณา ซึ่งจะแสดงผลรายการเกี่ยวกับโฆษณาที่ผู้ใช้งานได้เพิ่มเข้าในระบบ และฟังก์ชันการเพิ่มโฆษณา รวมถึงการแสดงผลแบบตารางให้ผู้ใช้งานมองได้สะดวกขึ้น

```
14  const Advertisement = () => {
15      const [activeTabKey, setActiveTabKey] = useState<string>( initialState: 'advertisement')
16      const [dataWatched, setDataWatched] = useState<Array<number>>( initialState: [])
17      const [ads, setAds] = useState<AdvertisingVideo[]>( initialState: [])
18      const [openDrawerAd, setOpenDrawerAd] = useState<boolean>( initialState: false)
19      const [user, setUser] = useState<User | null>(getUserStorage())
20      const [refresh, setRefresh] = useState<boolean>( initialState: false)
21      const [games, setGames] = useState<Array<Project>>( initialState: [])
22      const [dataTreeGraph, setDataTreeGraph] = useState<TreeGraphData>()
23      const tabListNoTitle = [...]
33
34      const onTabChange = (key: string) => {
35          setActiveTabKey(key)
36      }
37
38      const updateAds = async (): Promise<AdvertisingVideo[]> => {
39          const data = await getAdsVideo(user)
40          return data
41      }
42
43      const updateTreeGraph = async () => {...}
65
66      useEffect( effect: () => {
67          setUser(getUserStorage())
68      }, deps: [])
69
70      useEffect( effect: () => {
71          updateAds().then((value : AdvertisingVideo[] ) => setAds(value))
72          const update = updateTreeGraph()
73      }, deps: [user, openDrawerAd, refresh])
74
75      useEffect( effect: () => {
76          const data: Array<number> = []
77          ads.forEach((value : AdvertisingVideo ) => {
78              data.push((Math.floor( x: value.length! / 60)) + ((value.length! % 60) / 100))
79          })
80          setDataWatched(data)
81      }, deps: [ads])
82
```

ภาพที่ 4-15.1 ฟังก์ชันการทำงานของการแสดงผลหน้าแรกให้ผู้ใช้งานสำหรับนักโฆษณา



```
    return (
        <>
            <Card
                tabList={tabListNoTitle}
                activeTabKey={activeTabKey}
                onTabChange={(key :string ) => {
                    onTabChange(key)
                }}
                tabBarExtraContent={
                    <Button type={"primary"} icon={<PlusOutlined/>} onClick={() => setOpenDrawerAd( value: true)}>
                        Add Advertising
                    </Button>
                }
            >
                <DrawerNewAdvertise open={openDrawerAd} setOpen={setOpenDrawerAd}/>
                {activeTabKey === "advertisement" &&
                    <List
                        grid={{gutter: 16}}
                        dataSource={ads}
                        renderItem={(item, index) => (
                            <List.Item>
                                <Suspense fallback={<Spin tip="Loading..."/>}>
                                    <CardComponent video={item} key={index} refresh={refresh} setRefresh={setRefresh}/>
                                </Suspense>
                            </List.Item>
                        )}
                    />
                }
                {
                    activeTabKey === "content" &&
                    <>
                        <TableAdvertising videos={ads}/>
                    </>
                }
            </Card>
        </>
    )
}
```

ภาพที่ 4-15.2 ส่วนการแสดงผลหน้าแรกให้ผู้ใช้งานสำหรับนักโฆษณา



```
const TableAdvertising = ({videos}: ComponentAdvertise) => {
    const [sourceVideo, setSourceVideo] = useState<string>( initialState: "")
    const [isModalOpen, setIsModalOpen] = useState( initialState: false)
    const columns = [...]

    const showModal = (value: string) => {
        setSourceVideo(value)
        setIsModalOpen( value: true)
    }

    const handleOk = () => {
        setIsModalOpen( value: false)
    }

    const handleCancel = () => {
        setIsModalOpen( value: false)
    }

    return (
        <>
            <Modal
                title={"Preview video"}
                open={isModalOpen}
                onOk={handleOk}
                onCancel={handleCancel}
                okText={<ArrowRightOutlined/>}
                okButtonProps={{style: {width: "50px", height: "50px", borderRadius: "50%"}}}
                cancelButtonProps={{style: {display: "none"}}}
            ...>
            <Table dataSource={videos} columns={columns}  scroll={{x: true}}/>
        </>
    )
}
```

ภาพที่ 4-15.3 ส่วนประกอบการแสดงผลแบบตารางของรายการโฆษณา

```tsx
const CardAdvertising = ({video,refresh,setRefresh}: ComponentAdvertise) => {
    const {confirm} = Modal
    const navigate = useNavigate()
    const [confirmOpen, setConfirmOpen] = useState<boolean>( initialState: false)
    const [confirmLoading, setConfirmLoading] = useState<boolean>( initialState: false)
    const [time, setTime] = useState<number>( initialState: 0)
    useEffect( effect: () => {
        if (video) {
            setTime( value: (Math.floor( x: video.length! / 60)) + ((video.length! % 60) / 100))
        }
    }, deps: [video])

    const showConfirm = () => {...}

    return (
        <Card
            hoverable
            style={{width: 280}}
            cover={
                <iframe
                    style={{borderRadius: "4px"}}
                    width={"100%"}
                    height={"100%"}
                    src={video?.sources.toString()}
                    title={video?.name.toString()}
                    allowFullScreen
                    loading={"lazy"}
                />
            }
            actions={[
                <SettingOutlined key="setting" onClick={() => navigate( to: "/application/customers/setting")}/>,
                <DeleteOutlined key={"delete"} onClick={showConfirm}/>,
                <EllipsisOutlined key="ellipsis"/>,
            ]}
        ...>
    )
}
```

ภาพที่ 4-15.4 ส่วนประกอบการแสดงผลแบบการ์ดของรายการโฆษณา

4.2.2.4 ส่วนของการตั้งค่าและการเลือกโฆษณาลงโปรเจคเกมสำหรับผู้ใช้งานประเภทนักโฆษณาจะแสดงข้อมูลเป็นตารางจากภาพ 4-16.2 และส่วนประกอบของปุ่มการโฆษณาสู่โปรเจคเกมจากภาพ 4-16.3 และ 4-16.4

```
const SettingCampaign = () => {
    const {Text} = Typography
    const {Panel} = Collapse
    const text: string = (
        "So perhaps, you've generated some fancy text," +
        " and you're content that you can now copy and paste your fancy text in the comments section of funny cat videos, " +
        "but perhaps you're wondering how it's even possible to change the font of your text? Is it some sort of hack?" +
        " Are you copying and pasting an actual font?"
    )
    const [expandIconPosition, setExpandIconPosition] = useState<ExpandIconPosition>( initialState: 'end')
    const [ads, setAds] = useState<AdvertisingVideo[]>( initialState: [])
    const [games, setGames] = useState<Project[]>( initialState: [])

    const onChange = (key: string | string[]) => {...}

    const genExtra = () => (
        <SettingOutlined
            onClick={(event : MouseEvent<HTMLSpanElement> ) => {...}}
        />
    )

    const updateGames = async (): Promise<{ projects: Project[], ads: AdvertisingVideo[] }> => {...}

    useEffect( effect: () => {
        updateGames().then((value : {ads: AdvertisingVideo[], projects: Project[]} ) => {
            setGames(value.projects)
            setAds(value.ads)
        })
    }, deps: [])

    return (
        <>
            <Collapse
                defaultActiveKey={[]}
                onChange={onChange}
                expandIconPosition={expandIconPosition}
                ...>
            </>
    )}
```

ภาพที่ 4-16.1 การตั้งค่าและการเลือกโฆษณาลงโปรเจคเกม

```
const TableGame = ({advertises, setAdvertises, dataSource, setDataSource}: TableGameComponent) => {
    const [data, setData] = useState<TableComponent[]>( initialState: [])
    const columns: ColumnsType<Project> = [...]

    useEffect( effect: () => {
        const list: TableComponent[] = []
        if (dataSource) {
            dataSource.forEach((value : Project , index : number ) => {
                list.push({...value, key: index})
            })
            setData(list)
        }
    }, deps: [dataSource])


    return (
        <Table columns={columns} dataSource={data} scroll={{x:true}}/>
    )
}
```

ภาพที่ 4-16.2 ส่วนประกอบการแสดงผลรายการโปรเจคเกมที่ให้ผู้ใช้งานโฆษณา

```
const ButtonAdvertise = ({advertises, project, setAdvertises, setProjects}: ComponentButtonAdvertise) => {
    const [isSame, setIsSame] = useState<boolean | undefined>( initialState: false)
    const [loading, setLoading] = useState<boolean>( initialState: false)

    const advertiseVideo = async () => {
        setLoading( value: true)
        await addVideoToProject(project.id)
        await delay( MINUTE: 3000)
        if (setAdvertises) {
            setAdvertises(await getAdsVideo(getUserStorage()))
        }
        if (setProjects) {
            setProjects(await getGames())
        }
        setLoading( value: false)
    }

    const removeVideo = async () => {
        setLoading( value: true)
        await removeVideoOnProject(project.id)
        await delay()
        if (setAdvertises) {
            setAdvertises(await getAdsVideo(getUserStorage()))
        }
        if (setProjects) {
            setProjects(await getGames())
        }
        setLoading( value: false)
    }

    const verifyId = (element:AdvertisingVideo) => {
        return advertises.filter((value : AdvertisingVideo ) => element.id === value.id).length > 0
    }
```

ภาพที่ 4-16.3 ส่วนประกอบฟังก์ชันของปุ่มเพิ่มโฆษณาเข้าสู่โปรเจคเกม

```
    useEffect( effect: () => {
        if (advertises.length > 0) {
            // console.log(project.advertisingVideos, advertises)
            setIsSame(project.advertisingVideos?.some(verifyId))
        }
    }, deps: [advertises, project])

    return (
        <>
            {isSame === true ?
                <Button
                    loading={loading}
                    type={"primary"}
                    icon={<SyncOutlined spin/>}
                    onClick={removeVideo}
                >
                    Advertising
                </Button>
                :
                <Button
                    loading={loading}
                    onClick={advertiseVideo}
                >
                    Advertise
                </Button>
            }
        </>
    )
}
```

ภาพที่ 4-16.4 ส่วนประกอบการแสดงผลของปุ่มเพิ่มโฆษณาเข้าสู่โปรเจคเกม

4.2.2.5 ส่วนประกอบของฟังก์ชันการเพิ่มโฆษณาของผู้ใช้งาน ฟังก์ชันจะตรวจสอบไฟล์ที่อัพโหลดเป็นประเภท mp4 หากไม่เป็นจะถูกแจ้งเตือนกลับไป



```
dayjs.extend(customParseFormat)
const DrawerNewAdvertise = ({open, setOpen}: ComponentDrawer) => {
    const {Option} = Select
    const {Dragger} = Upload
    const [form] = Form.useForm()
    const [api, contextHolder] = notification.useNotification()
    const [fileList, setFileList] = useState<UploadFile[]>( initialState: [])
    const [uploading, setUploading] = useState<boolean>( initialState: false)
    const [user, setUser] = useState<User | null>( initialState: null)
    const placement = "topLeft"
    const props: UploadProps = {name: 'file'...}

    const onClose = () => {...}

    const updateFileList = (file: RcFile) => {
        let newFileList = [...fileList, file]
        newFileList = newFileList.slice(-1)
        newFileList = newFileList.map((file : UploadFile | RcFile ) => {
            return file
        })
        setFileList(newFileList)
    }

    const onFinish = async (value: any) => {
        setUploading( value: true)
        const file: RcFile =
            (value.upload.fileList.length > 0) ? value.upload.fileList.at(value.upload.fileList.length - 1) : value.upload.file
        if (file.type === 'video/mp4') {...} else {
            if (file.name) {...} else {
                api.error({message: "Video ads are required."...})
            }
        }
        setUploading( value: false)
    }

    const onFinishFailed = (errorInfo: unknown) => {...}

    useEffect( effect: () => {...}, deps: [open])
```

ภาพที่ 4-17 ส่วนประกอบของฟังก์ชั่นการเพิ่มโฆษณาของผู้ใช้งาน

4.2.2.6 หน้าการแสดงผลโปรจคเกมทั้งหมดของผู้ใช้งานประเภทผู้พัฒนาเกมที่เข้าสู่ระบบฟังก์ชันจะเรียกข้อมูลจากเซิฟเวอร์ของผู้ใช้งานมาแสดงผลเป็นตารางและมีฟังก์ชันเพิ่มคือการสร้างโปรเจคเกม และเข้าร่วมโปรเจคเกม

```
const AllProject = () => {
    const {Search} = Input
    const {Title} = Typography
    const [user, setUser] = useState<User | null>(getUserStorage())
    const [isModalJoinOpen, setIsModalJoinOpen] = useState<boolean>( initialState: false)
    const [isModalOpen, setIsModalOpen] = useState<boolean>( initialState: false)
    const [projects, setProjects] = useState<Array<Project>>( initialState: [])
    const items: MenuProps['items'] = [...]

    const updateProject = async () => setProjects(await getProjects(user))

    const onSearch = (searchValue: string) => {
        if (searchValue.length > 0) {
            const result: Array<Project> | undefined = projects.filter((value : Project ) => value.name.includes(searchValue))
            if (result) {
                setProjects(result)
            } else {
                setProjects( value: [])
            }
        } else {
            setProjects(getProjectsStorage())
        }
    }

    const handleMenuClick: MenuProps['onClick'] = (e) => {
        switch (e.key) {
            case 'create' :
                setIsModalOpen( value: true)
                break
            case 'add' :
                setIsModalJoinOpen( value: true)
                break
            default:
        }
    }

    useEffect( effect: () => {
        const data = updateProject()
    }, deps: [user])
```

ภาพที่ 4-18.1 ฟังก์ชันสำหรับการแสดงผลตารางของรายการโปรเจคเกม

```
    return (
        <>
            <div className={"content-project"}>
                <Card
                    className={"card-project content"}
                    title={
                        <Title level={3}>
                            <CodeSandboxOutlined/>
                            {" Game Development Projects"}
                        </Title>
                    }
                    extra={
                        <Row gutter={[8, 16]}...>
                    }
                >
                    <TableProject dataSource={projects}/>
                    <ModalForm
                        modalOpen={isModalOpen}
                        setModalOpen={setIsModalOpen} user={user}
                        setUpdateProjects={setProjects}
                    />
                    <ModalJoinProject
                        modalOpen={isModalJoinOpen}
                        setModalOpen={setIsModalJoinOpen}
                        setUpdateProjects={setProjects}
                    />
                </Card>
                <Space direction={"horizontal"} size={"middle"} className={"card-project footer"}>
                    <p>Privacy Policy</p>
                    <p>Cookie</p>
                </Space>
            </div>
        </>
    )
}
```

ภาพที่ 4-18.2 การแสดงผลตารางของรายการโปรเจคเกม

```tsx
const TableProject = ({dataSource}: TableModel) => {
    const {Title} = Typography
    const [Data, setData] = useState<Array<TableProjectModel>>( initialState: [])
    const props = {
        loading: false,
        title: undefined,
        showHeader: true,
    }
    const columns: ColumnsType<TableProjectModel> = [...]

    useEffect( effect: () => {
        const projects: Array<TableProjectModel> = []
        dataSource?.forEach( callbackfn: (value : Project , index : number ) => {
            projects.push({
                ...value,
                key: index,
                member: value.users.length,
                advertising: (value.advertisingVideos?.length) ? value.advertisingVideos?.length : 0
            })
        })
        setData(projects)
    }, deps: [dataSource])

    return (
        <Table
            {...props}
            className={"content_projects"}
            columns={columns}
            dataSource={Data}
            scroll={{x: true}}
            pagination={{
                defaultPageSize: 10,
                showSizeChanger: true,
                pageSizeOptions: ['10', '20', '30']
            }}
        />

    )
}
```

ภาพที่ 4-18.3 ส่วนประกอบการแสดงผลตารางโปรเจคเกม

4.2.2.7 หน้าการแสดงผลโปรเจคเกมทั้งหมดของผู้ใช้งานประเภทผู้พัฒนาเกมที่เข้าสู่ระบบฟังก์ชันจะเรียกข้อมูลจากเซิฟเวอร์ของผู้ใช้งานมาแสดงผลเป็นตารางและมีฟังก์ชันเพิ่มคือการสร้างโปรเจคเกม และเข้าร่วมโปรเจคเกม

```
16    const GameProject = () => {
17        const {id} = useParams<{ id: string }>()
18        const {Title} = Typography
19        const [modalAddPeopleOpen, setModalAddPeopleOpen] = useState<boolean>( initialState: false)
20        const [project, setProject] = useState<Project | null>( initialState: null)
21        const [path, setPath] = useState<JSX.Element[]>( initialState: [])
22        const [view, setView] = useState<JSX.Element>(<></>)
23        const [onView, setOnView] = useState<string>( initialState: "")
24
25        const handleMenuClick: MenuProps['onClick'] = (event) => {
26            setOnView(event.key)
27        }
28
29        const viewPage = () => {...}
64
65        const updateProject = async () => {
66            if (id) {
67                setProject(await getProject(Number(id)))
68            }
69        }
70
71        useEffect( effect: () => {
72            const update = updateProject()
73        }, deps: [])
74
75
76        useEffect( effect: () => {
77            setView(viewPage())
78        }, deps: [onView, project])
79
```

ภาพที่ 4-19.1 ส่วนประกอบการแสดงผลตารางโปรเจคเกม



```
    return (
        <Card title={
            <Title level={3} className={"project title"}>
                <Avatar style={{backgroundColor: '#1890ff'}} icon={<AntDesignOutlined/>}/>
                <a>{project?.name}</a>
            </Title>
        } extra={
            <Row gutter={[8, 16]}...>
        }>
            <Breadcrumb style={{margin: "0 0 16px"}}>
                <Breadcrumb.Item>
                    Application
                </Breadcrumb.Item>
                <Breadcrumb.Item onClick={() => {
                    setOnView( value: "")
                }}>
                    <a>Dashboard project</a>
                </Breadcrumb.Item>
                {path.map((value :Element , index :number ) =>
                    <Breadcrumb.Item key={index}>{value}</Breadcrumb.Item>
                )}
            </Breadcrumb>
            <ModalAddPeople
                dataSource={project}
                modalOpen={modalAddPeopleOpen}
                setModalOpen={setModalAddPeopleOpen}
            />
            {view}
        </Card>
    )
}
```

ภาพที่ 4-19.2 ส่วนประกอบการแสดงผลตารางโปรเจคเกม

4.2.2.8 หน้าการแสดงผลโปรเจคเกมที่ประกอบส่วนต่าง ๆ จากหน้าการแสดงผลและสามารถถูกเลือกการแสดงผลเฉพาะหน้าได้ ซึ่งฟังก์ชันหลักของโปรเจคเกมจะมีส่วนประกอบ คือ รายละเอียดโปรเจคเกม บริการของปลั๊กอิน รายการโฆษณาบนโปรเจคเกม สำหรับฟังก์ชันเสริม เช่น การตั้งค่า หรือรายการกิจกรรมของผู้ใช้งานภายในโปรเจคเกมแสดงผลเพื่อแสดงข้อมูลรายละเอียดต่างๆมากขึ้น

```
const ViewProject = ({dataSource}: ComponentProject) => {
    const {Title} = Typography
    return (
        <>
            <Row gutter={{xs: 8, sm: 16, md: 24, lg: 32}} style={{maxHeight: "100vh"}}>
                <Col span={10}...>
                <Col span={14}>
                    <Row gutter={[16, 24]}>
                        <Col flex={"auto"}>
                            <Row gutter={[16, 24]} justify={"space-between"}>
                                <Col>
                                    <PercentActivityProject dataSource={dataSource}/>
                                </Col>
                                <Col>
                                    <ActivitiesProject dataSource={dataSource}/>
                                </Col>
                            </Row>
                        </Col>
                        <Col flex="auto">
                            <Card
                                hoverable
                                title={<Title level={5} className={"project title"}...>}
                                style={{width: "100%"}}
                            >
                                <MemberProject dataSource={dataSource}/>
                            </Card>
                        </Col>
                    </Row>
                </Col>
            </Row>
            <UnityPluginProject dataSource={dataSource}/>
            <ViewAdsService dataSource={dataSource}/>
        </>
    )
}
```

ภาพที่ 4-20.1 หน้าการแสดงผลข้อมูลแผนภูมิของโปรเจคเกม

```
const ViewAdsService = ({dataSource}:ComponentProject) => {
    const {Title} = Typography
    return(
        <Card
            hoverable
            style={{margin: "2rem 0", width: "100%"}}
            title={
                <Title level={4} className={"project title"}>
                    <ControlOutlined/>
                    Advertise Service
                </Title>
            }
        >
            <AdsTableProject dataSource={dataSource}/>
        </Card>
    )
}
```

ภาพที่ 4-20.2 หน้าแสดงผลข้อมูลของการโฆษณามาสู่โปรเจคเกม



```
const AdsTableProject = ({dataSource}: ComponentProject) => {
    const [data, setData] = useState<Array<TableAdvertiseProject>>( initialState: [])
    const [selectedRowKeys, setSelectedRowKeys] = useState<React.Key[]>( initialState: [])
    const [sourceVideo, setSourceVideo] = useState<string>( initialState: "")
    const [isModalOpen, setIsModalOpen] = useState( initialState: false)
    const hasSelected = selectedRowKeys.length > 0
    const columns: ColumnsType<TableAdvertiseProject> = [...]

    const showModal = (value: string) => {
        setSourceVideo(value)
        setIsModalOpen( value: true)
    }

    const handleOk = () => {...}

    const handleCancel = () => {...}

    const onSelectChange = (newSelectedRowKeys: React.Key[]) => {
        console.log('selectedRowKeys changed: ', newSelectedRowKeys)
        setSelectedRowKeys(newSelectedRowKeys)
    }

    const rowSelection = {
        selectedRowKeys,
        onChange: onSelectChange,
    }

    useEffect( effect: () => {
        if (dataSource) {
            const value: Array<TableAdvertiseProject> = []
            dataSource.advertisingVideos?.forEach( callbackfn: (ads : AdvertisingVideo , index : number ) => {
                value.push({...ads, key: index})
            })
            setData(value)
        }
    }, deps: [dataSource])

```

ภาพที่ 4-20.3 ฟังก์ชันการแสดงผลแบบตารางของข้อมูลโฆษณาบนโปรเจคเกม

```tsx
const UnityPluginProject = ({dataSource}: ComponentProject) => {
    const {Title} = Typography
    const [activeTabKey, setActiveTabKey] = useState<string>( initialState: 'information')
    const [gameObjects, setGameObjects] = useState<GameObject[]>( initialState: [])
    const [modalLinkUnityOpen, setModalLinkUnityOpen] = useState<boolean>( initialState: false)
    const tabList = [
        {
            key: 'information',
            tab: <><InfoCircleOutlined/>Information</>,
        },
        {
            key: 'table',
            tab: <><TableOutlined/>Table</>,
        },
        {
            key: 'chart',
            tab: <><FundViewOutlined />Scene</>,
            disabled: (gameObjects.length > 2) ? false : true
        },
    ]

    const onTabChange = (key: string) => {
        setActiveTabKey(key)
    };

    const updateGameObjects = () => {
        if (dataSource?.gameObjects) {
            setGameObjects(dataSource.gameObjects)
        }
    }

    useEffect( effect: () => {
        updateGameObjects()
    }, deps: [dataSource])

```

ภาพที่ 4-20.4 ส่วนฟังก์ชันการแสดงผลข้อมูลของปลั๊กอินที่เชื่อมต่อกับ Unity game engine



```
    return (
        <Card
            hoverable
            style={{margin: "2rem 0", width: "100%",minWidth:"-moz-fit-content",overflowX:"auto"}}
            title={
                <Title level={4} className={"project title"}>
                    <CodeSandboxOutlined/>
                    Unity plugin service
                </Title>
            }
            extra={
                <Button type={"primary"} onClick={() => setModalLinkUnityOpen( value: true)}>
                    <LinkOutlined/>
                    Key connect to Unity
                </Button>
            }
            tabList={tabList}
            activeTabKey={activeTabKey}
            onTabChange={(key : string ) => {
                onTabChange(key)
            }}
        >
            <ModalLinkUnity dataSource={dataSource} modalOpen={modalLinkUnityOpen} setModalOpen={setModalLinkUnityOpen}/>
            <div>
                {activeTabKey === "information" && <DescriptionProject dataSource={dataSource}/>}
                {activeTabKey === "table" && <GameObjectProject dataSource={dataSource}/>}
                {
                    activeTabKey === "chart" &&
                    <ScatterChart dataGameObject={gameObjects} config={chartConfig.ScatterByGameObject}/>
                }
            </div>

        </Card>
    )
}
```

ภาพที่ 4-20.5 ส่วนการแสดงผลข้อมูลของปลั๊กอินที่เชื่อมต่อกับ Unity game engine



```
const MemberProject = ({dataSource}: ComponentProject) => {
    const {Text} = Typography
    const [members, setMembers] = useState<TableMemberProject[]>( initialState: [])
    const props = {
        loading:false,
        title: undefined,
        showHeader: true,
    }
    const columns: ColumnsType<TableMemberProject> = [...]

    useEffect( effect: () => {
        const data: Array<TableMemberProject> = []
        dataSource?.users.forEach( callbackfn: (value : User , index : number ) => {
            data.push({...value, key: index})
        })
        setMembers(data)
    }, deps: [dataSource?.users])

    return (
        <>
            <Table dataSource={members} columns={columns} scroll={{x:true}} {...props}/>
        </>
    )
}
```

ภาพที่ 4-20.6 ส่วนการแสดงตารางของรายการสมาชิกภายในโปรเจคเกม

```
const ViewActionProject = ({dataSource}: ComponentProject) => {
    const {Title} = Typography
    return (
        <Row gutter={[16, 24]}>
            <Col flex={'auto'}>
                <Row gutter={[16, 24]} justify={"space-evenly"}  >
                    <Col>
                        <PercentActivityProject dataSource={dataSource}/>
                    </Col>
                    <Col>
                        <ActivitiesProject dataSource={dataSource}/>
                    </Col>
                </Row>
            </Col>
            <Col flex={'auto'}>
                <Card title={<Title level={5} className={"project title"}><ProfileOutlined/>Latest
                    Activities</Title>} hoverable>
                    <StepActivityProject dataSource={dataSource}/>
                </Card>
            </Col>
        </Row>
    )
}
```

ภาพที่ 4-20.7 ส่วนการแสดงข้อมูลกิจกรรมภายในโปรเจคเกม

```
const ViewSettingProject = ({dataSource}: ComponentProject) => {
    const {Title} = Typography
    const [current, setCurrent] = useState( initialState: 'add-tags')

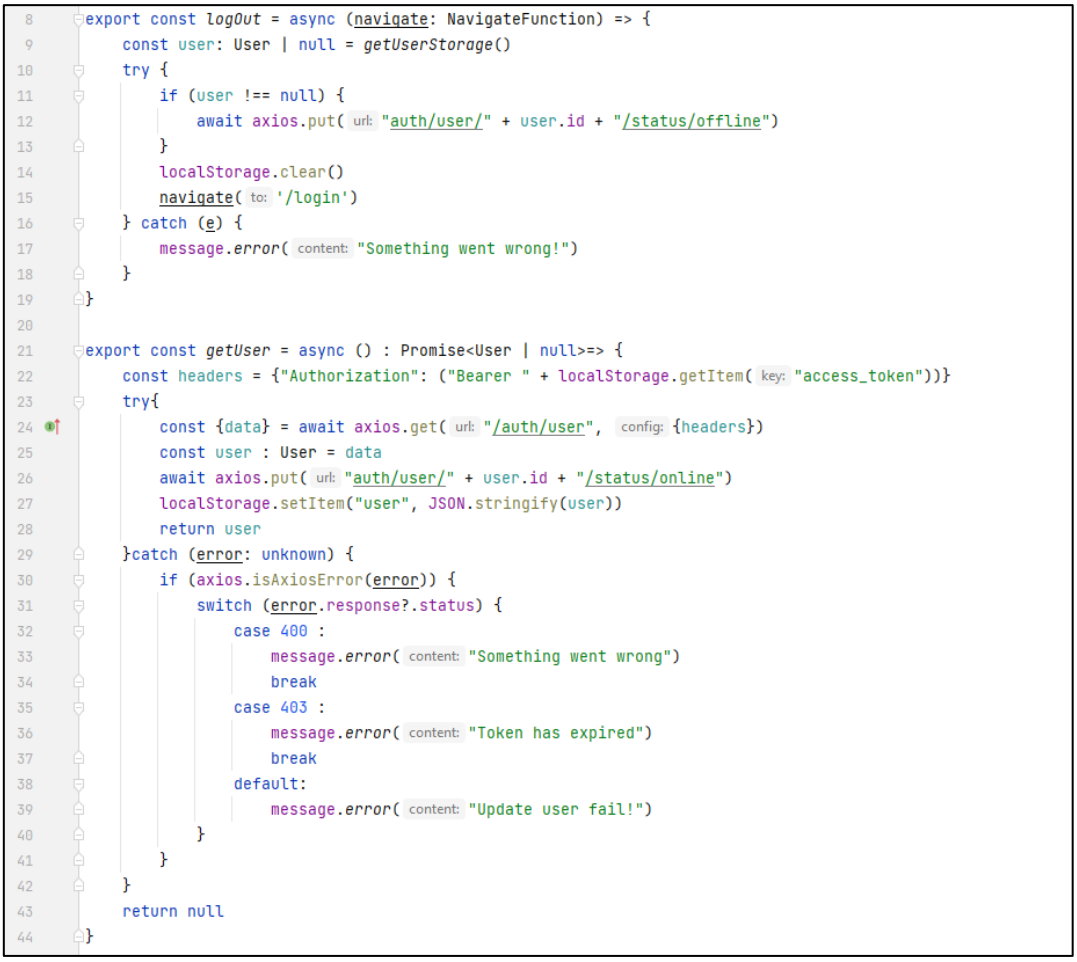
    const onClick: MenuProps['onClick'] = (event) => {
        setCurrent(event.key)
    }

    return (
        <Row justify={"center"}>
            <Col flex={"auto"} span={16}>
                <Divider orientation="left"><Title level={3}>General</Title></Divider>
                <p...>
                <Alert
                    message="Note"
                    description="You can change and leave a project but
                    can't remove other member and everyone have a claim to change data on the project"
                    type="info"
                    showIcon
                />
                <Menu mode="horizontal" selectedKeys={[current]} onClick={onClick}>
                    <Menu.Item key={"add-tags"} icon={<TagsOutlined/>}...>
                    <Menu.Item key={"edit"} icon={<FormOutlined/>}...>
                    <Menu.Item key="leave" icon={<ExportOutlined/>}...>
                </Menu>
                <div style={{margin: "2rem"}}>
                    {current === "edit" && <FormEditProject dataSource={dataSource}/>}
                    {current === "add-tags" && <FormAddTagProject dataSource={dataSource}/>}
                    {current === "leave" && <FormLeaveProject dataSource={dataSource}/>}
                </div>

            </Col>
        </Row>
    )
}
```

ภาพที่ 4-20.8 ส่วนการแสดงข้อมูลการตั้งค่าภายในโปรเจคเกม

4.2.2.9 ส่วนของการฟังก์ชันช่วยเหลือสำหรับการเรียกใช้โดยรวมของหน้าการแสดงผลต่างๆ



```
export const logOut = async (navigate: NavigateFunction) => {
    const user: User | null = getUserStorage()
    try {
        if (user !== null) {
            await axios.put( url: "auth/user/" + user.id + "/status/offline")
        }
        localStorage.clear()
        navigate( to: '/login')
    } catch (e) {
        message.error( content: "Something went wrong!")
    }
}

export const getUser = async () : Promise<User | null>=> {
    const headers = {"Authorization": ("Bearer " + localStorage.getItem( key: "access_token"))}
    try{
        const {data} = await axios.get( url: "/auth/user",  config: {headers})
        const user : User = data
        await axios.put( url: "auth/user/" + user.id + "/status/online")
        localStorage.setItem("user", JSON.stringify(user))
        return user
    }catch (error: unknown) {
        if (axios.isAxiosError(error)) {
            switch (error.response?.status) {
                case 400 :
                    message.error( content: "Something went wrong")
                    break
                case 403 :
                    message.error( content: "Token has expired")
                    break
                default:
                    message.error( content: "Update user fail!")
            }
        }
    }
    return null
}
```

ภาพที่ 4-21.1 แสดงฟังก์ชันช่วยเหลือสำหรับหน้าการแสดงผลต่างๆ

```
export const getUserStorage = (): User | null => {
    let user: User | null = null
    const storage = localStorage.getItem( key: "user")
    if (storage) {
        user = JSON.parse(storage)
    }
    return user
}

export const updateUser = async (navigate: NavigateFunction): Promise<User | null> => {
    const headers = {"Authorization": ("Bearer " + localStorage.getItem( key: "access_token"))}
    try {
        const result = await axios.get( url: "/auth/user",  config: {headers})
        const user: User = result.data
        await axios.put( url: "auth/user/" + user.id + "/status/online")
        localStorage.setItem("user", JSON.stringify(user))
        return user
    } catch (error: unknown) {
        if (axios.isAxiosError(error)) {
            switch (error.response?.status) {
                case 400 :
                    message.error( content: "Something went wrong")
                    break
                case 403 :
                    message.error( content: "Token has expired")
                    break
                default:
                    message.error( content: "Update user fail!")
            }
            localStorage.clear()
            navigate( to: '/login')
        }
    }
    return null
}
```

ภาพที่ 4-21.2 แสดงฟังก์ชันช่วยเหลือสำหรับหน้าการแสดงผลต่างๆ

```
export const getProject = async (id: number): Promise<Project | null> => {
    try {
        const {data} = await axios.get( url: '/game/project/' + id)
        localStorage.setItem("project-" + id, JSON.stringify(data))
        return data
    } catch (e) {
        console.log(e)
    }
    return null
}

export const getAdsVideo = async (user: User | null): Promise<Array<AdvertisingVideo>> => {
    try {
        const {data} = await axios.get( url: "/advertise_video/user/" + user?.id)
        localStorage.setItem("ads", JSON.stringify(data))
        return data
    } catch (e) {
    }
    return []
}

export const getGames = async (): Promise<Array<Project>> => {
    try {
        const {data} = await axios.get( url: '/game/projects')
        localStorage.setItem("games", JSON.stringify(data))
        return data
    } catch (e) {
        console.log(e)
    }
    return []
}
```

ภาพที่ 4-21.3 แสดงฟังก์ชันช่วยเหลือสำหรับหน้าการแสดงผลต่างๆ

```
export const getProjects = async (user: User | null): Promise<Array<Project>> => {
    if (user !== null) {
        try {
            const {data} = await axios.get( url: '/game/projects/' + user?.id)
            localStorage.setItem("projects", JSON.stringify(data))
            return data
        } catch (e) {
            console.log(e)
        }
    }
    return []
}

export const getProjectsStorage = (): Array<Project> => {
    let project: Array<Project> = []
    const storage = localStorage.getItem( key: "projects")
    if (storage) {
        project = JSON.parse(storage)
    }
    return project
}

export const generateInviteCode = (id: number): string => {
    const arr = new Uint8Array( length: 3)
    window.crypto.getRandomValues(arr)
    const fake: string = Array.from(arr, decToHex).join("").toUpperCase()
    return (fake + id)
}

export const inviteCodeToId = (inviteCode: string): number => Number(inviteCode.slice(6))

export const generatePluginCode = (id: number): string => {
    const arr = new Uint8Array( length: 8)
    window.crypto.getRandomValues(arr)
    const fake: string = Array.from(arr, decToHex).join("").toUpperCase()
    return (fake + id)
}
```

ภาพที่ 4-21.4 แสดงฟังก์ชันช่วยเหลือสำหรับหน้าการแสดงผลต่างๆ

```
export const editProject = async (projectId: number, userId: number, form: { name: string, description: string }): Promise<Project | null> => {
    try {
        const {data} = await axios.put( url: "/game/project/" + projectId + "/edit/" + userId, form)
        localStorage.setItem("project-" + projectId, JSON.stringify(data))
        return data
    } catch (e) {
        console.log(e)
    }
    return null
}

export const delay = (MINUTE: number = 2000) => new Promise( executor: resolve => setTimeout(resolve, MINUTE))

export const addTagProject = async (userId: number, form: { projectId: number, nameTag: string }) => {
    try {
        await axios.post( url: "/game/tag/add/project/" + userId, form)
    } catch (e) {
        message.error(e?.toString())
    }
}

export const removeProject = async (projectId: number, userId: number) => {
    await axios.delete( url: "/game/project/" + projectId + "/remove/" + userId)
}
```

ภาพที่ 4-21.5 แสดงฟังก์ชันช่วยเหลือสำหรับหน้าการแสดงผลต่างๆ



```
export const register = async (form: { name: string, email: string, password: string }, role: string): Promise<string> => {
    let alert: string = ""
    try {
        await axios.post( url: "/auth/user/save", form)
    } catch (error) {
        if (axios.isAxiosError(error)) {
            switch (error.response?.status) {
                case 403 :
                    alert = "User already exists"
                    break
                default:
                    alert = "Register fail!"
            }
        }
        return alert
    }
    try {
        await axios.post( url: "/auth/role/add/user",  data: {userName: form.name, roleName: role})
    } catch (e) {
        message.error( content: "Something went wrong!!")
        return alert
    }
    return alert
}

export const joinProject = async (inviteCode: string): Promise<User> => {
    const id: number = inviteCodeToId(inviteCode)
    const user: User | null = getUserStorage()
    if (user) {
        try {
            await axios.put( url: '/game/project/' + id + '/join/' + user.id)
        } catch (e) {
            message.error( content: "Can't join the project")
        }
    } else {
        message.error( content: "User not found")
    }
    return user!
}
```

ภาพที่ 4-21.6 แสดงฟังก์ชันช่วยเหลือสำหรับหน้าการแสดงผลต่างๆ



```
export const addVideoToProject = async (projectId: number) => {
    const storage = localStorage.getItem( key: "ads")
    if (storage) {
        const ads: AdvertisingVideo[] = JSON.parse(storage)
        const data: number[] = []
        ads.forEach((value : AdvertisingVideo ) => data.push(value.id))
        const body = {
            projectId: projectId,
            videoId: data
        }
        const headers = {"Authorization": ("Bearer " + localStorage.getItem( key: "access_token"))}
        try {
            await axios.post( url: "/advertise_video/add/project", body,  config: {headers})
        } catch (e) {
            message.error( content: "Something went wrong")
        }
    }
}

export const removeVideoOnProject = async (projectId: number) => {
    const storage = localStorage.getItem( key: "ads")
    if (storage) {
        const ads: AdvertisingVideo[] = JSON.parse(storage)
        const data: number[] = []
        ads.forEach((value : AdvertisingVideo ) => data.push(value.id))
        const body = {
            projectId: projectId,
            videoId: data
        }
        const headers = {"Authorization": ("Bearer " + localStorage.getItem( key: "access_token"))}
        try {
            await axios.delete( url: "/advertise_video/remove/project",  config: {data: body, headers: headers})
        } catch (e) {
            message.error( content: "Something went wrong")
        }
    }
}
```

ภาพที่ 4-21.7 แสดงฟังก์ชันช่วยเหลือสำหรับหน้าการแสดงผลต่างๆ



```
export const uploadAdvertiseVideo = async (file: UploadFile<any>, nameAdvertising: string): Promise<AdvertisingVideo | undefined> => {
    const headers = {
        "Access-Control-Allow-Origin": "*",
        'Content-Type': 'multipart/form-data'
    }
    const formData = new FormData()
    formData.append( name: "file", file.originFileObj!)
    formData.append( name: "nameAdvertising", nameAdvertising)
    formData.append( name: "username", getUserStorage()!.name.toString())
    try {
        const {data} = await axios.post( url: "/advertise_video/upload", formData, config: {headers})
        return data
    } catch (error) {
        if (axios.isAxiosError(error)) {
            switch (error.response?.status) {
                case 403 :
                    message.error( content: "You can't access upload advertise")
                    break
                default:
                    message.error( content: "Something went wrong!!")
            }
        }
    }
}

export const deleteAdvertiseVideo = async (videoId: number) => {
    const headers = {
        // "Authorization": ("Bearer " + localStorage.getItem("access_token"))
    }
    try {
        const {data} = await axios.delete( url: "/advertise_video/remove/video/" + videoId, config: {headers})
        return data
    } catch (error) {
        if (axios.isAxiosError(error)) {
            switch (error.response?.status) {
                case 403 :
                    message.error( content: "You can't access delete advertise")
                    break
                default:
```

ภาพที่ 4-21.8 แสดงฟังก์ชันช่วยเหลือสำหรับหน้าการแสดงผลต่างๆ



```
                    break
                default:
                    message.error( content: "Something went wrong!!")
            }
        }
    }
}

export const selectRoleToPath = (role:string) : string => {
    switch (role) {
        case "GAME_DEVELOPER" : return "/application/games"
        case "CUSTOMER_ADVERTISING": return "/application/customers"
        default: return "/application"
    }
}

export const getRoles = async ():Promise<Array<Role>> => {
    try{
        const {data} = await axios.get( url: "/auth/roles")
        return data
    } catch (e) {

    }
    return []
}
```

ภาพที่ 4-21.9 แสดงฟังก์ชันช่วยเหลือสำหรับหน้าการแสดงผลต่างๆ

บทที่ 5
ผลการดำเนินงาน

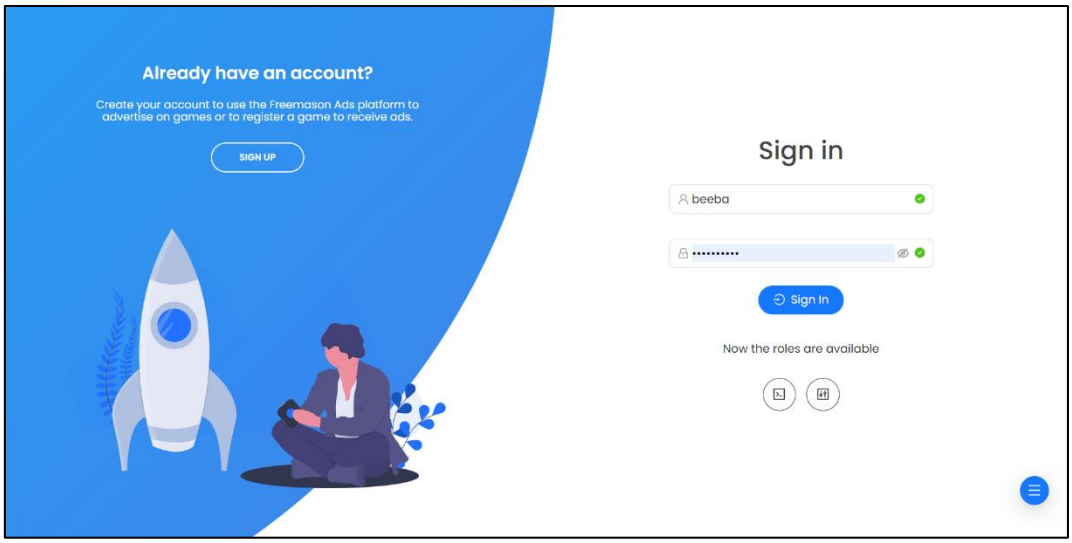
5.1 ผลการดำเนินงาน

โครงงานที่ได้จัดทำเว็บแอปพลิเคชันและปลั๊กอินภายใน Unity game engine โดยการแสดงผลบนเว็บแอปพลิเคชันการตอบสนองต่ออุปกรณ์ต่าง ๆ ที่มีความละเอียดน้อยกว่า 2K จะแสดงผลผิดพลาดจากที่ออกแบบและปลั๊กอินภายในระบบด้านความปลอดภัยของข้อมูล การตรวจสอบของการนับเวลาของวัตถุ และประสิทธิภาพการทำงาน อาจจะยังมีข้อบกพร่อง

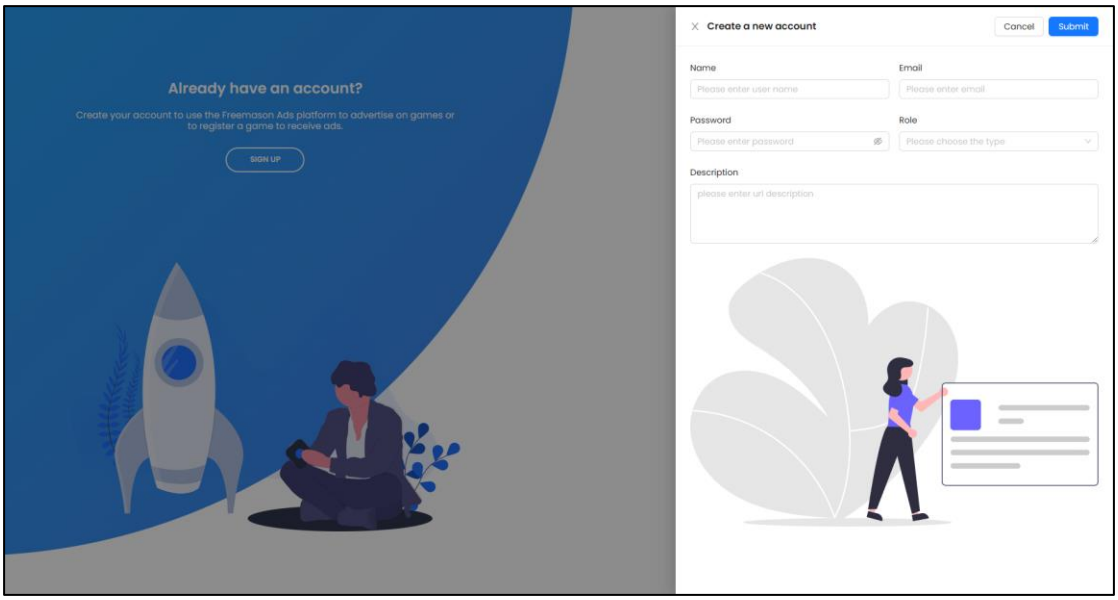
5.2 การแสดงผลของระบบ

5.2.1 เว็บแอปพลิเคชัน

5.2.1.1 หน้าการเข้าสู่ระบบและการสมัครบัญชีเพื่อเข้าใช้งาน สามารถให้ผู้ใช้งานกรอกข้อมูลเพื่อเข้าสู่ระบบเข้าใช้งานได้ หากผู้ใช้งานยังไม่มีบัญชีก็สามารถสมัครบัญชีการใช้งานได้

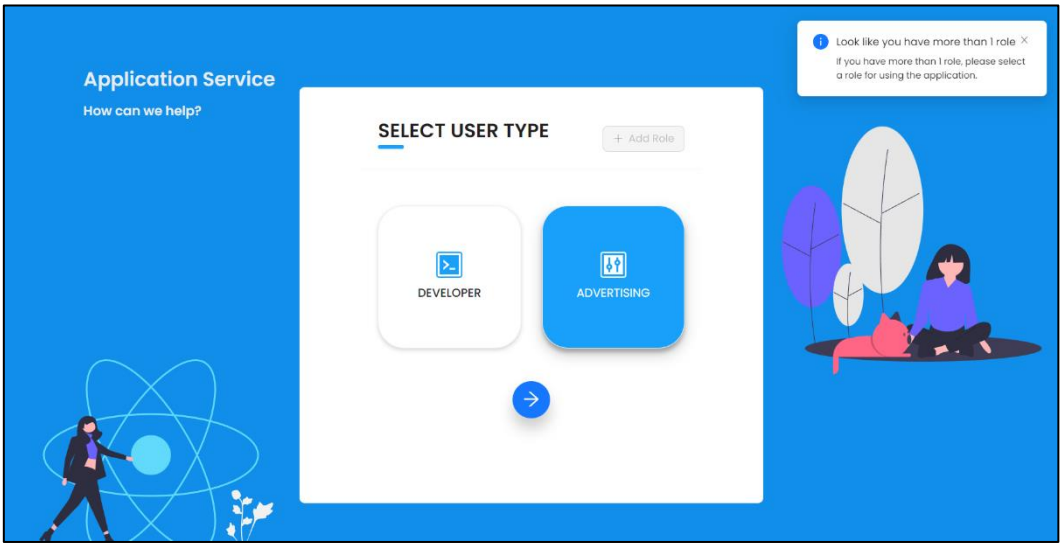


ภาพที่ 5-1 หน้าแสดงผลการเข้าสู่ระบบ



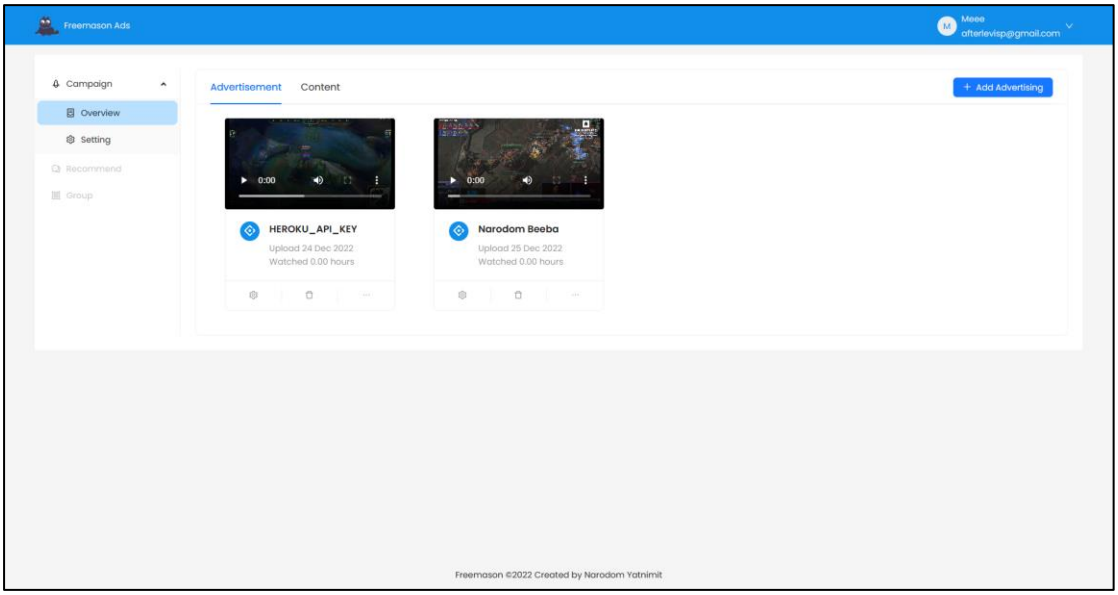
ภาพที่ 5-2 หน้าแสดงผลแบบฟอร์มการสมัครบัญชีเพื่อเข้าใช้งาน

5.2.1.2 หน้าเลือกบทบาทของผู้ใช้งาน หากผู้ใช้งานมีบทบาทสำหรับการใช้งานมากกว่า 1 ผู้ใช้งานต้องเลือกบทบาทเพื่อเข้าใช้งาน

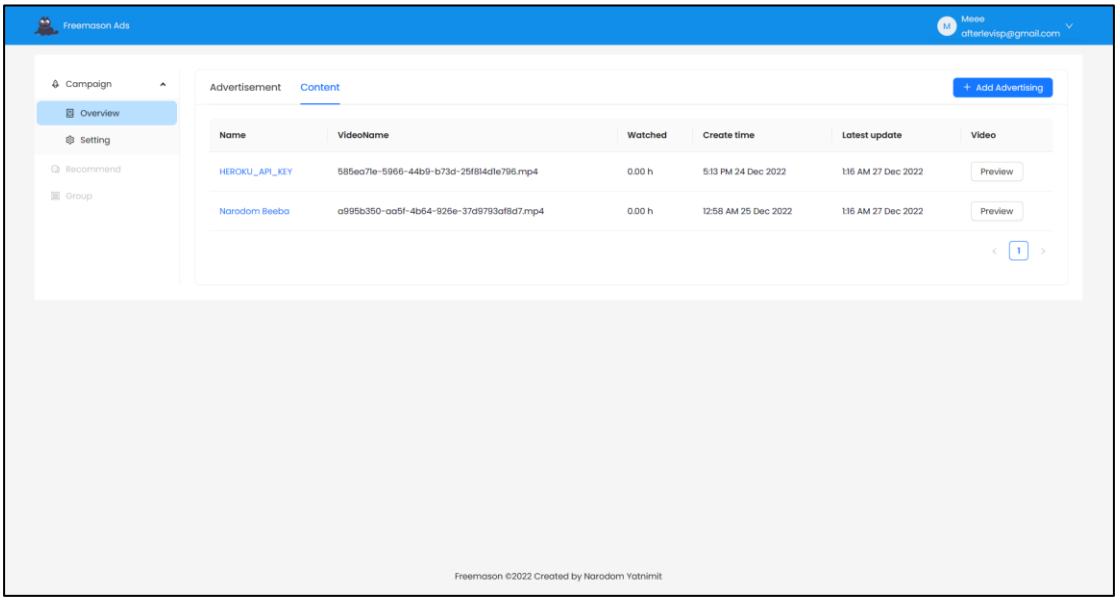


ภาพที่ 5-3 หน้าเลือกบทบาทของผู้ใช้งาน

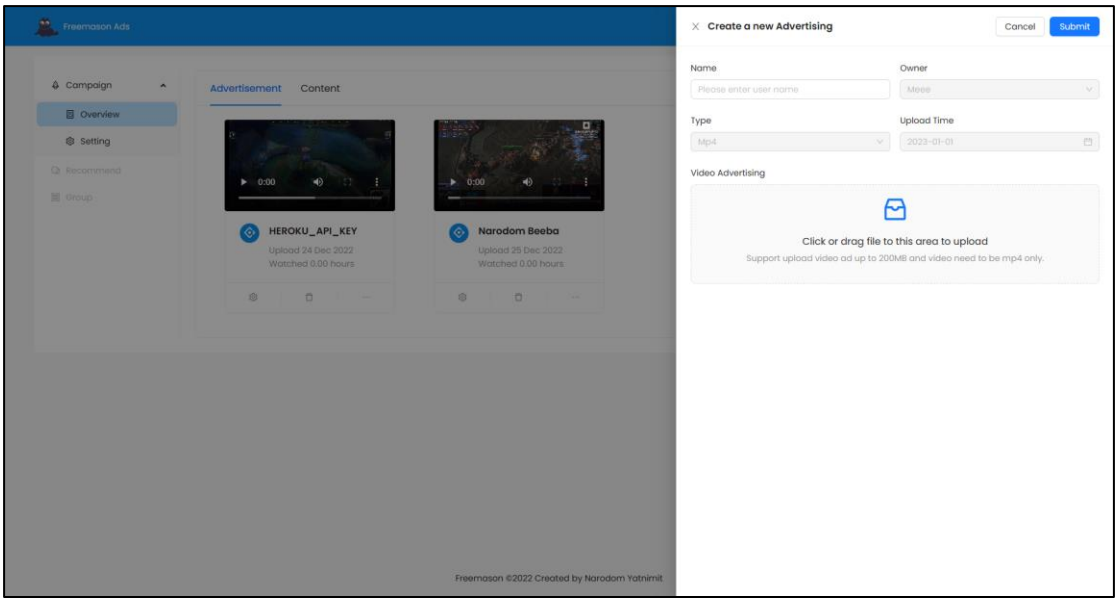
5.2.1.3 หน้าแสดงรายการโฆษณาของผู้ใช้งานประเภทนักโฆษณา เป็นหน้าแรกสำหรับผู้ใช้งานประเภทนักโฆษณา สามารถดูรายการโฆษณาแบบตารางได้ และสามารถเพิ่มวีดีโอโฆษณาได้



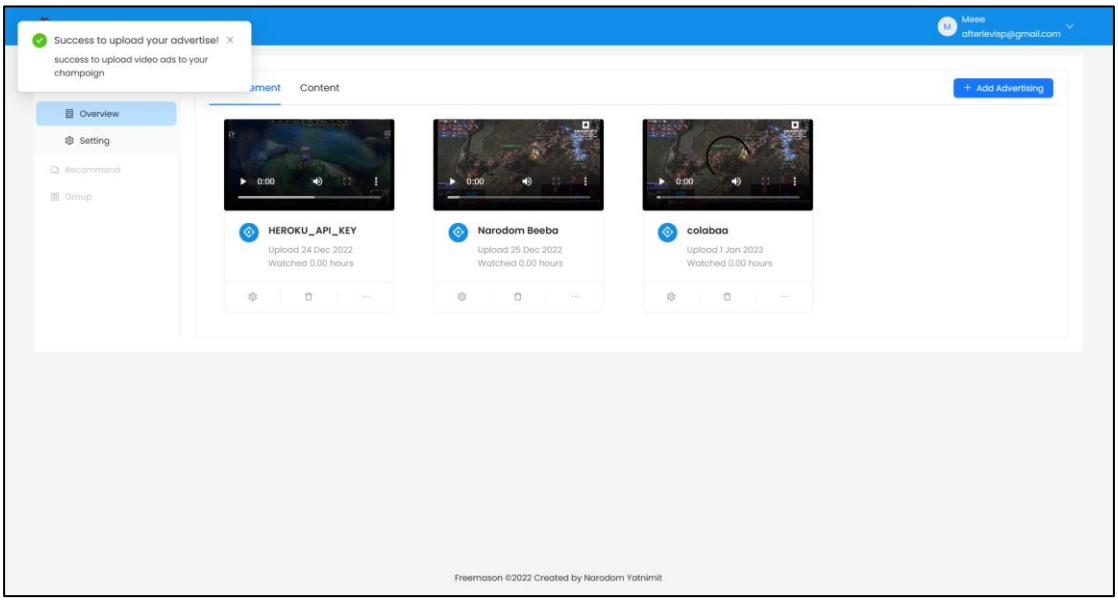
ภาพที่ 5-4 หน้าแสดงรายการโฆษณาของผู้ใช้งานประเภทนักโฆษณา



ภาพที่ 5-5 หน้าแสดงรายการโฆษณาของผู้ใช้งานประเภทนักโฆษณาแบบตาราง

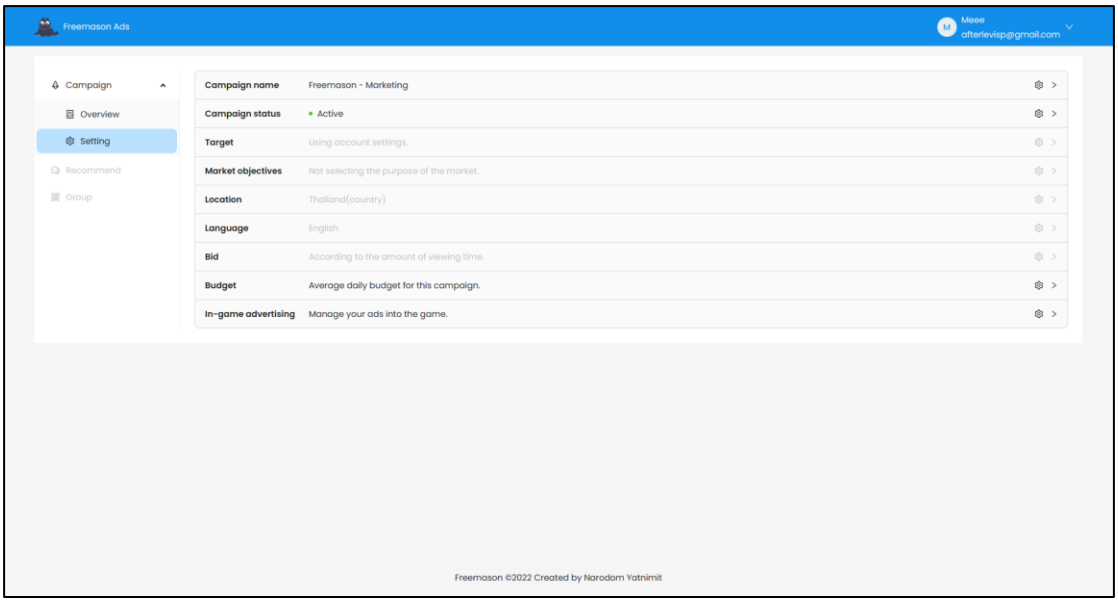


ภาพที่ 5-6 หน้าแบบฟอร์มสำหรับการเพิ่มวีดีโฆษณา

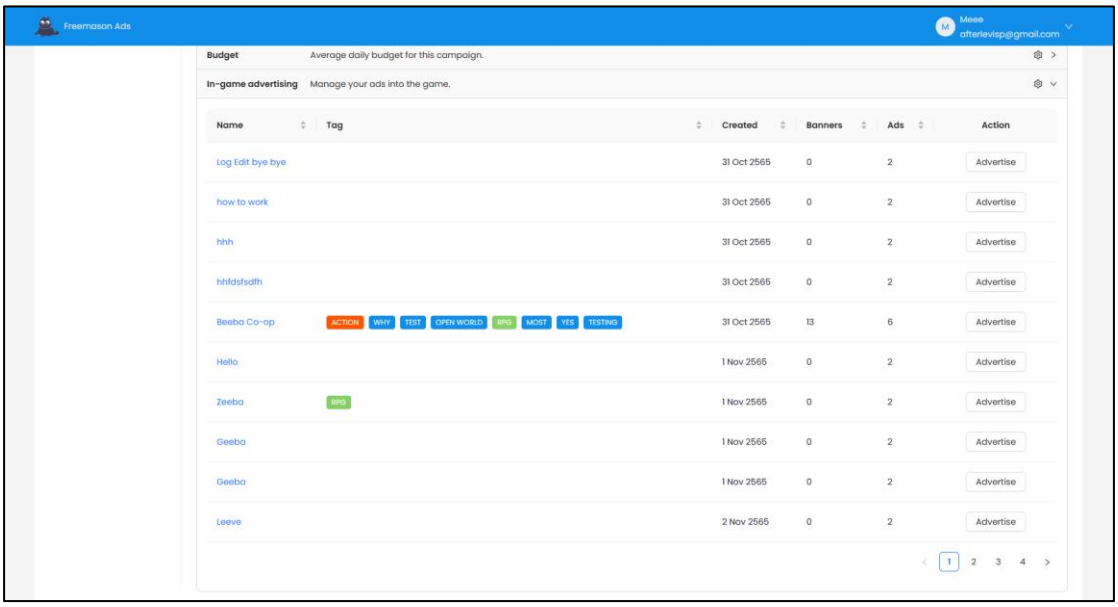


ภาพที่ 5-7 หลังจากการเพิ่มวีดีโอโฆษณา

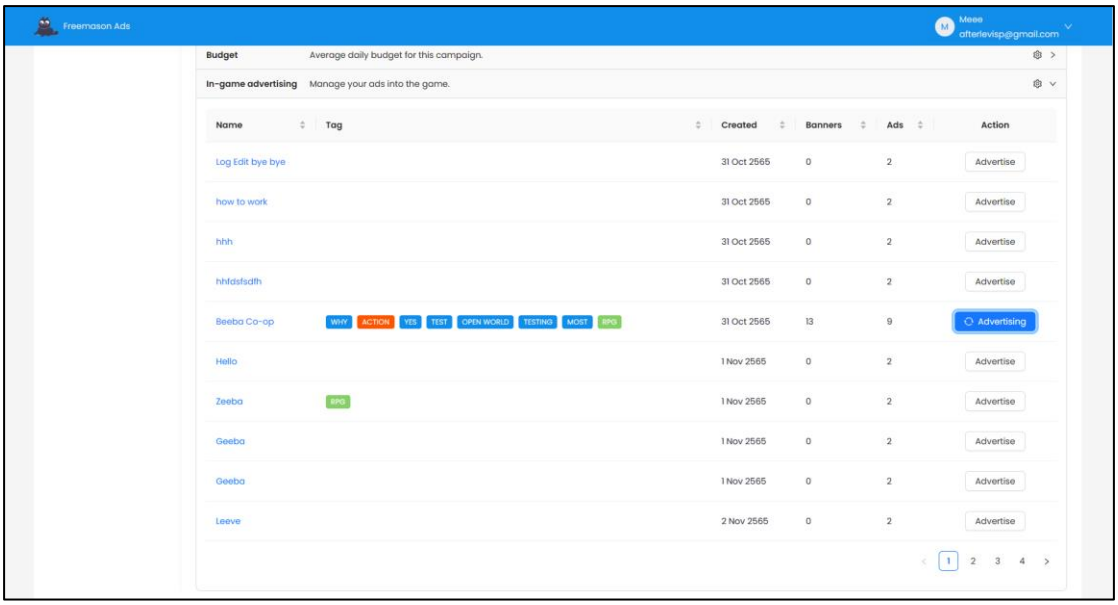
5.2.1.4 หน้าแสดงการตั้งค่าการโฆษณาของผู้ใช้งานประเภทนักโฆษณา สามารถแสดงรายการโปรเจคเกมที่สามารถโฆษราได้โดยผู้ใช้งานสามารถเลือกโปรเจคเกมที่ต้องการให้วีดีโอโฆษณาทั้งหมดของผู้ใช้งานเพิ่มโฆษณาสู่โปรเจคเกม



ภาพที่ 5-8 หน้าแสดงการตั้งค่าการโฆษณาของผู้ใช้งาน

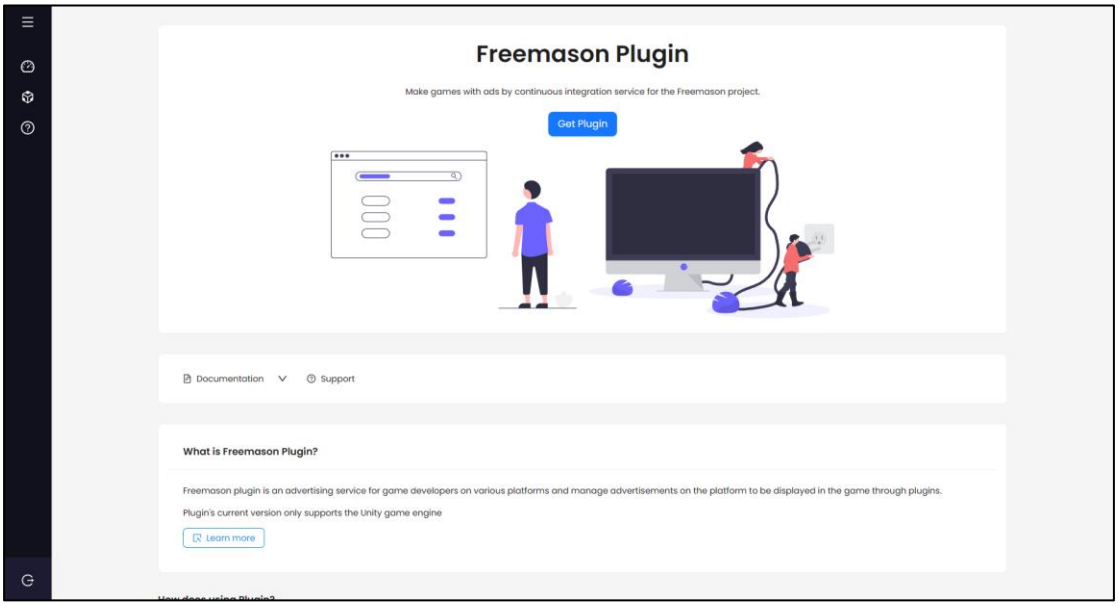


ภาพที่ 5-9 หน้าแสดงรายการโปรเจคเกมที่สามารถโฆษณา

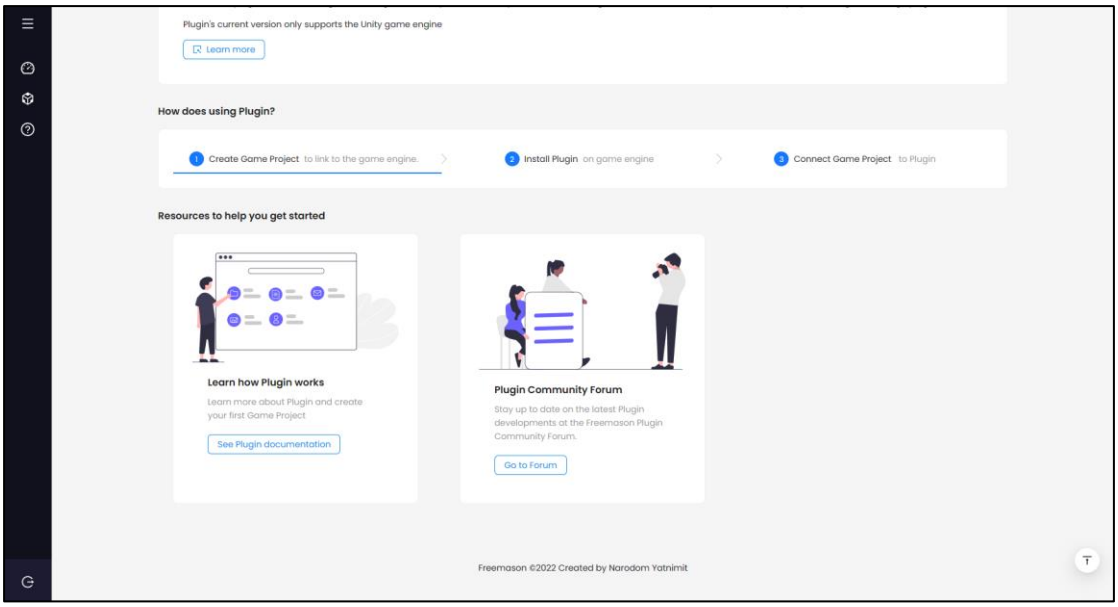


ภาพที่ 5-10 แสดงรายการที่ผู้ใช้งานลงโฆษณากับโปรเจคเกมที่เลือก

5.2.1.5 หน้าแรกแสดงผลสำหรับผู้ใช้งานประเภทผู้พัฒนาเกม แสดงรายละเอียดปลั๊กอินช่องทางการติดตั้ง วิธีการใช้งาน

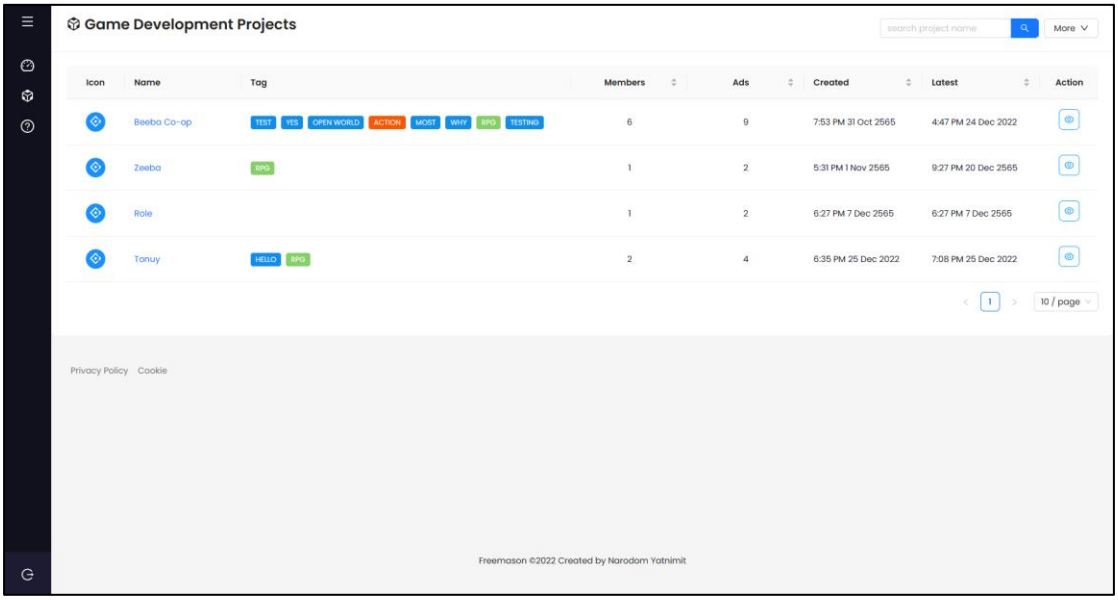


ภาพที่ 5-11 หน้าแรกแสดงผลสำหรับผู้ใช้งานประเภทผู้พัฒนาเกม

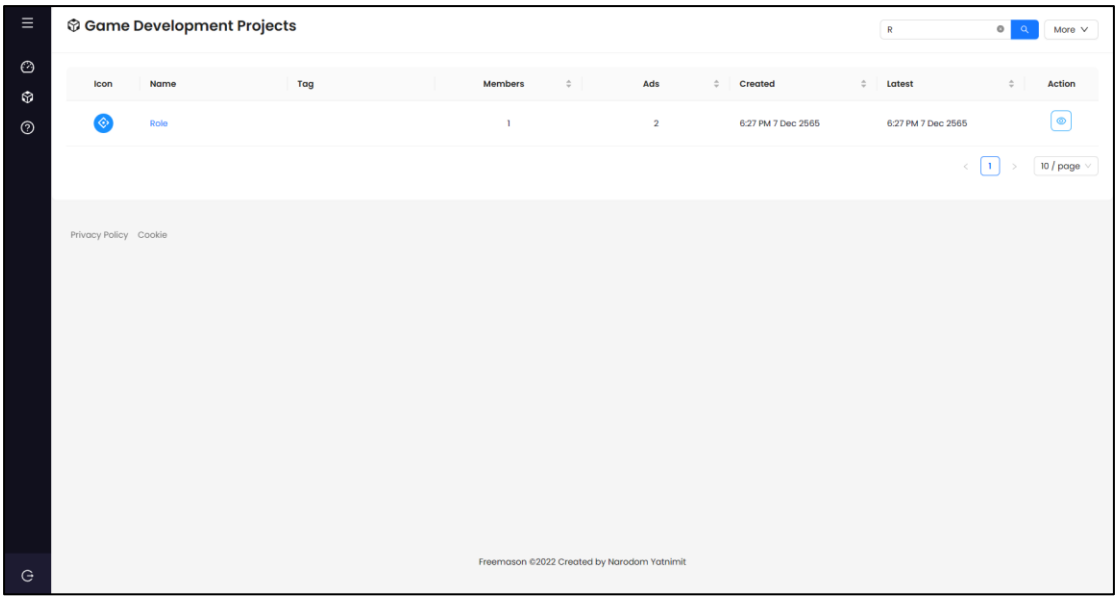


ภาพที่ 5-12 หน้าแรกแสดงผลสำหรับผู้ใช้งานประเภทผู้พัฒนาเกม

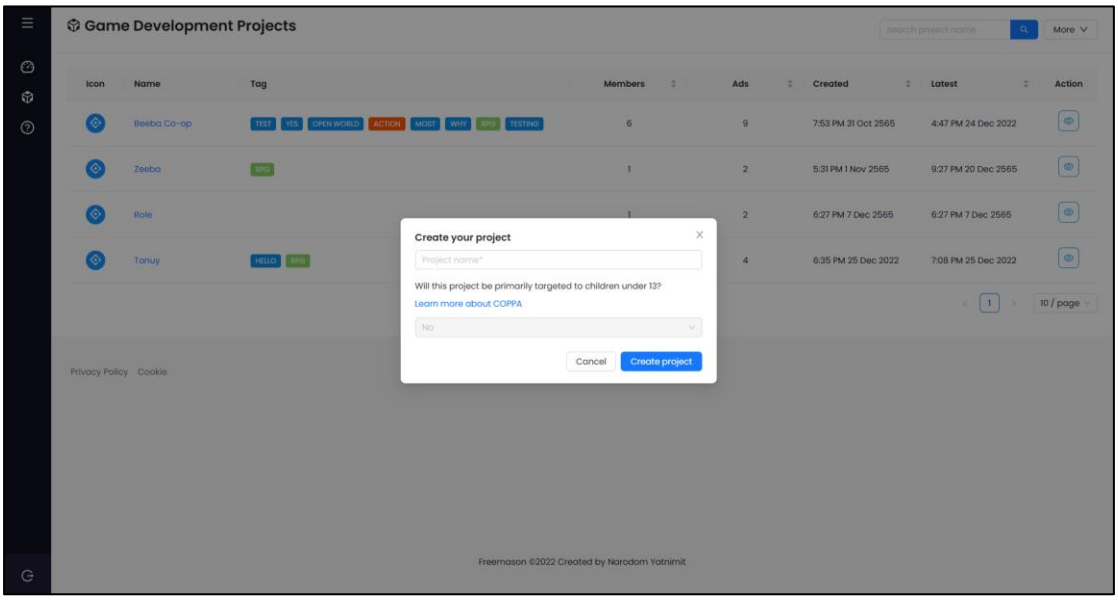
5.2.1.6 หน้าแสดงผลรายเการโปรเจคเกมทั้งหมดของผู้ใช้งานประเภทผู้พัฒนาเกม และฟังก์ชันการสร้างและเข้าร่วมโปรเจคเกม



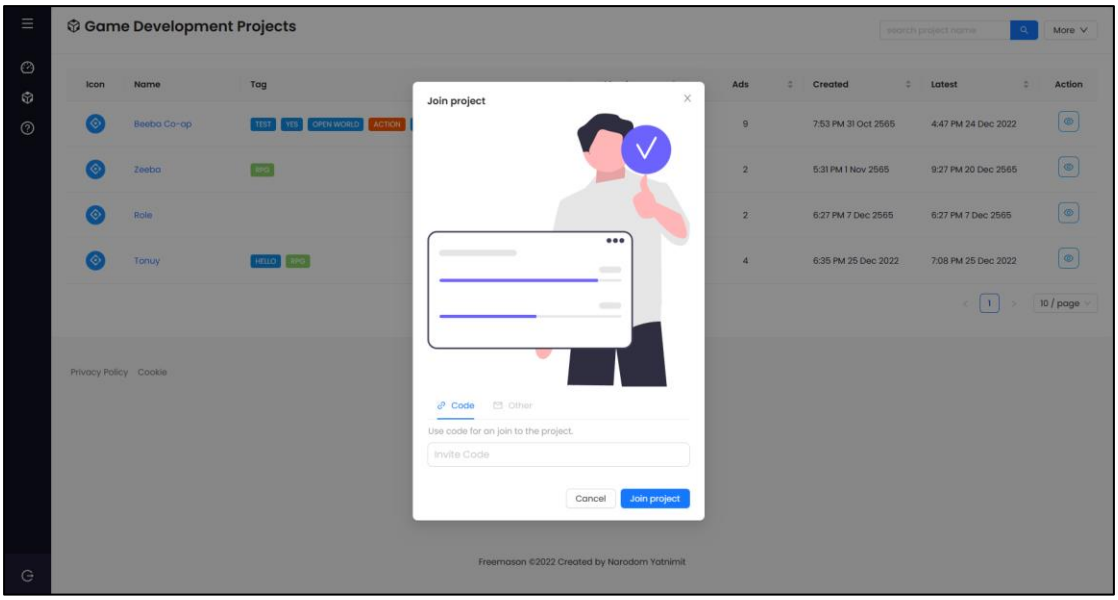
ภาพที่ 5-13 หน้าแรกแสดงผลสำหรับผู้ใช้งานประเภทผู้พัฒนาเกม



ภาพที่ 5-14 ฟังก์ชันการค้นหาโปรเจคเกม



ภาพที่ 5-15 ฟอร์มสร้างโปรเจคเกมสำหรับผู้ใช้งานประเภทผู้พัฒนาเกม



ภาพที่ 5-16 ฟอร์มใส่รหัสเข้าโปรเจคเกมสำหรับผู้ใช้งานประเภทผู้พัฒนาเกม

5.2.1.7 หน้าแสดงผลโปรเจคเกมและฟังก์ชันการใช้งานต่าง ๆ สำหรับผู้ใช้งานประเภทผู้พัฒนาเกม โดยมีฟังก์ชันการใช้งาน ดังนี้

5.2.1.7.1 การเพิ่มผู้ใช้งานเข้าสู่โปรเจคเกม ภาพที่ 5-22

5.2.1.7.2 การดูข้อมูลต่าง ๆ จากปลั๊กอินที่เชื่อมต่อ ภาพที่ 5-18 ถึง 5-20

5.2.1.7.3 การรับรหัสการเชื่อมต่อระหว่างโปรเจคเกมและปลั๊กอินภายใน Unity game engine

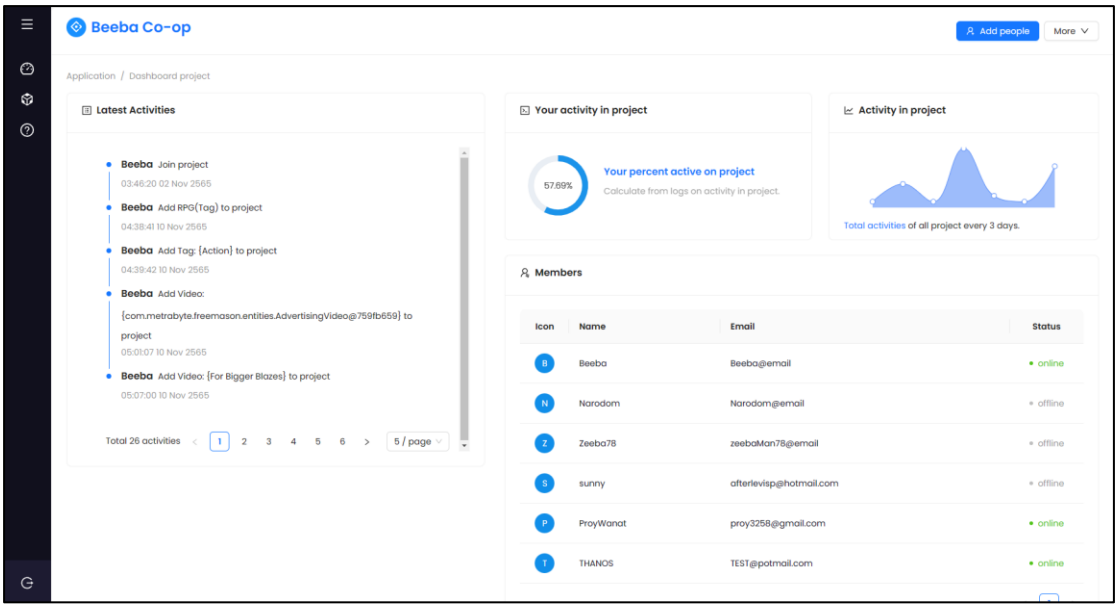
5.2.1.7.4 การดูข้อมูลรายการโฆษณาที่เพิ่มเข้าภายในโปรเจคเกม

5.2.1.7.5 การตั้งค่าต่างๆ ภายในโปรเจคเกม

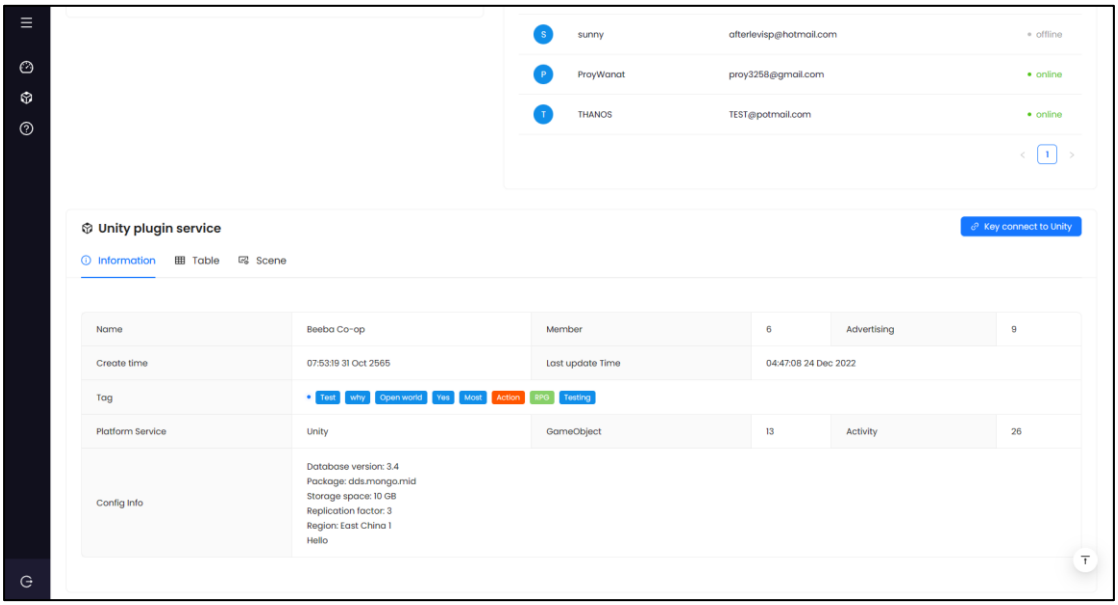
5.2.1.7.5.1 การเพิ่มแท็ก ภาพที่ 5-23

5.2.1.7.5.1 การแก้ไขรายละเอียดหรือชื่อโปรเจคเกม ภาพที่ 5-24

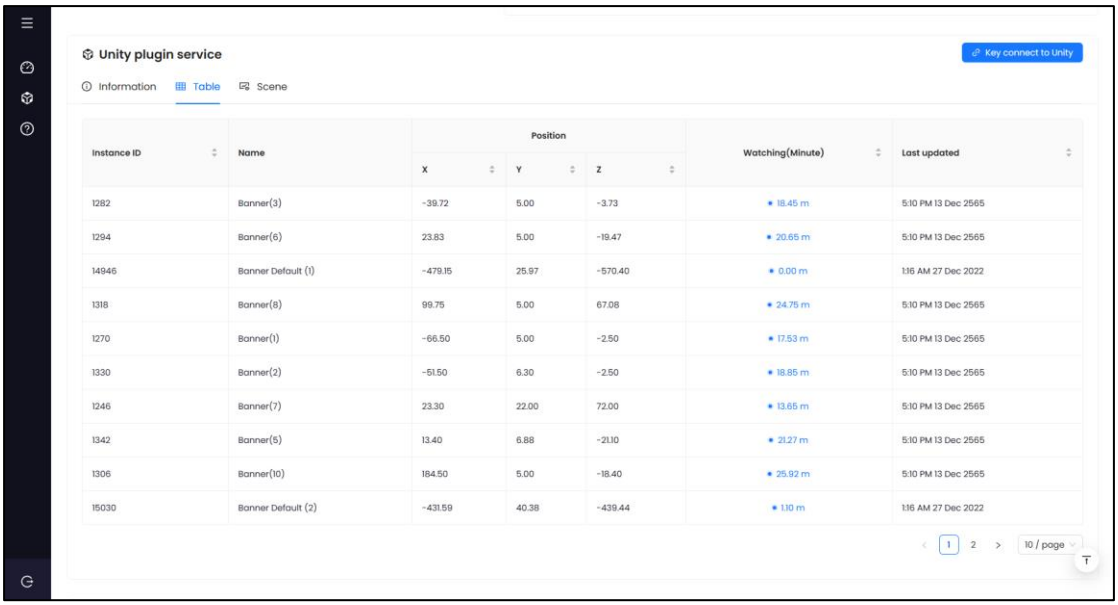
5.2.1.7.5.1 การออกจากโปรเจคของผู้ใช้งานภายในโปรเจค ภาพที่ 5-25



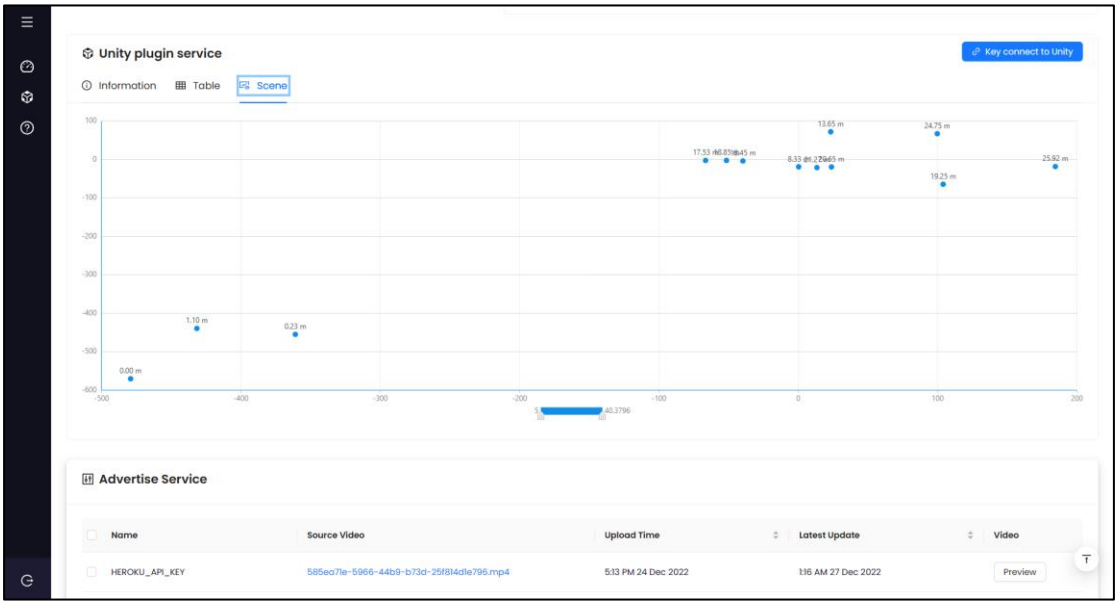
ภาพที่ 5-17 หน้าแรกแสดงผลโปรเจคเกม



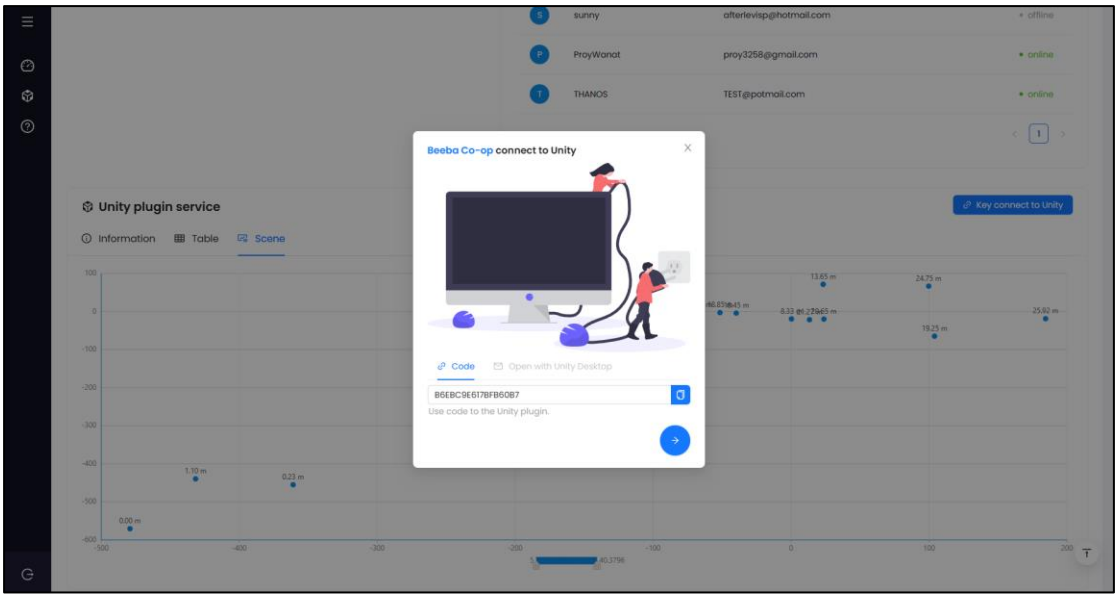
ภาพที่ 5-18 การบริการของปลั๊กอินภายในโปรเจคเกม



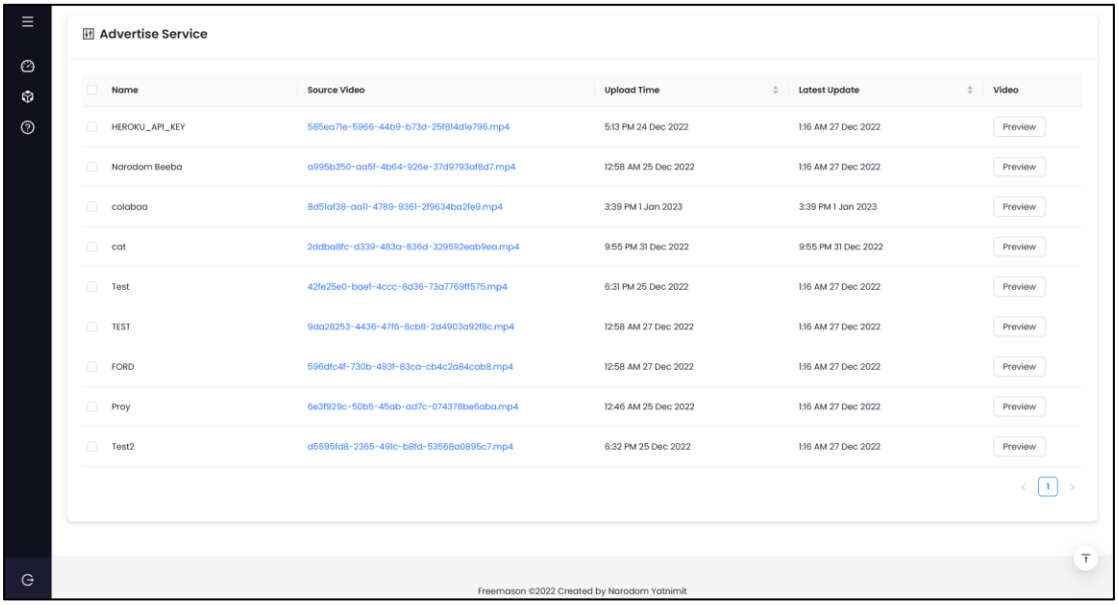
ภาพที่ 5-19 การแสดงผลแบบตารางของการบริการปลั๊กอินภายในโปรเจคเกม



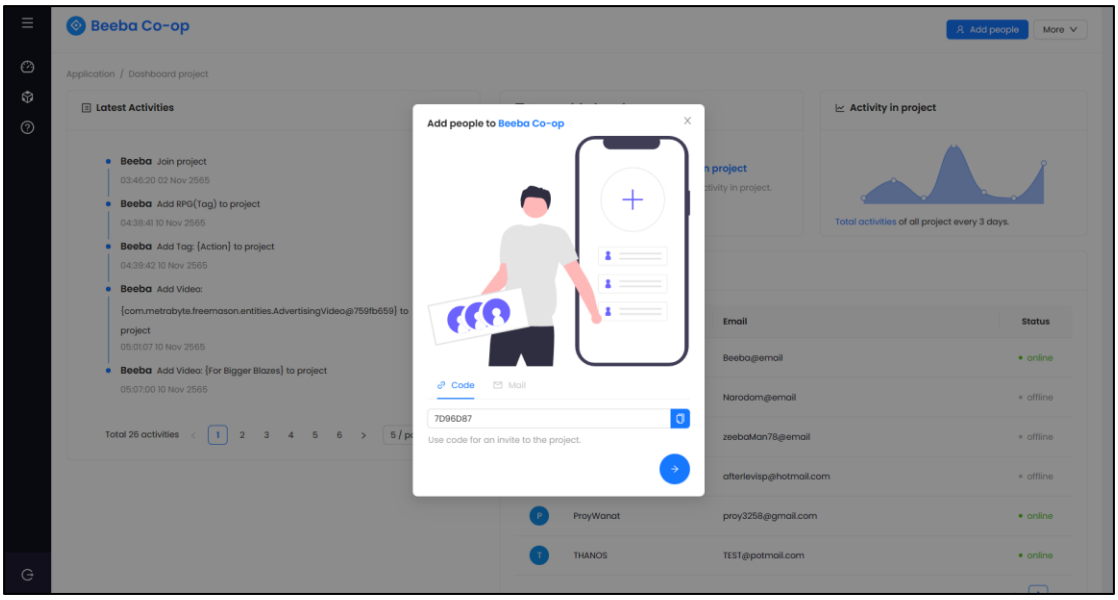
ภาพที่ 5-20 การแสดงผลแบบแผนภูมิของการบริการของการปลั๊กอินภายในโปรเจคเกม



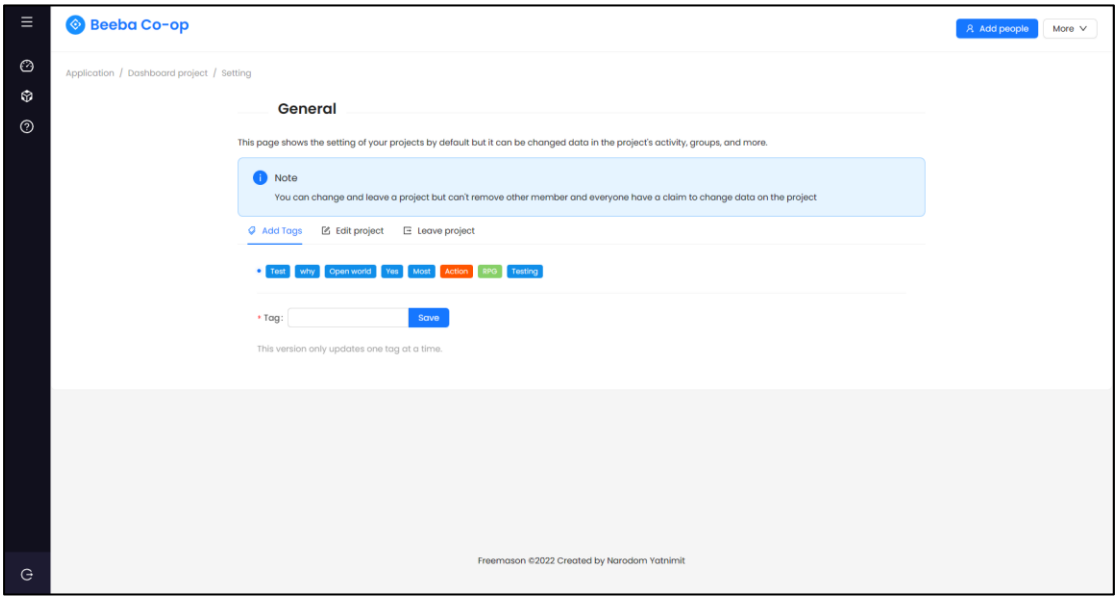
ภาพที่ 5-21 การแสดงรหัสการเชื่อมต่อระหว่างโปรเจคเกมและปลั๊กอินภายใน Unity game engine



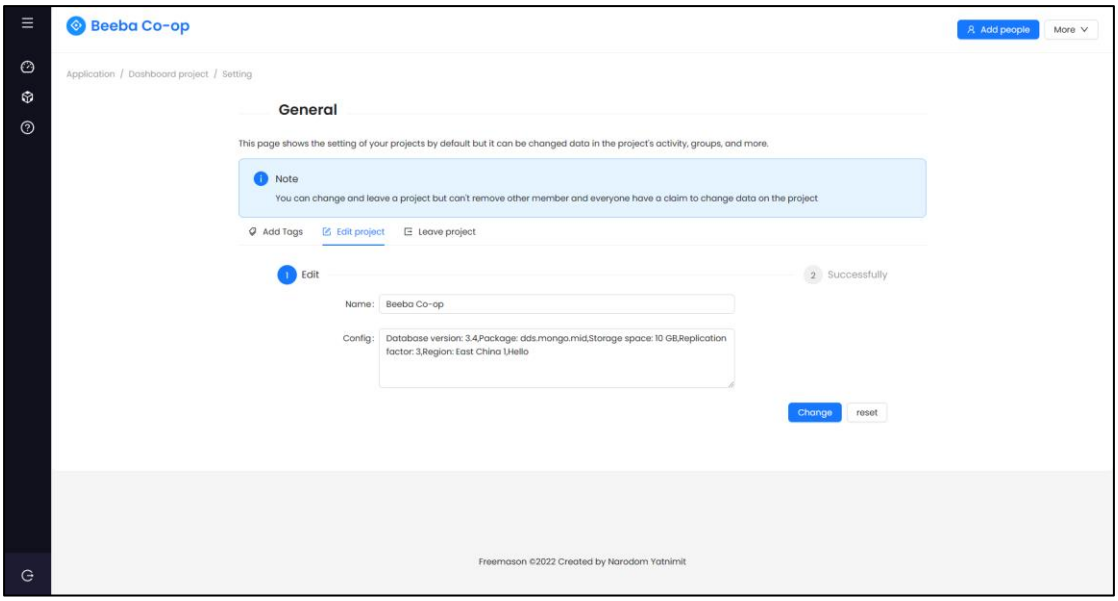
ภาพที่ 5-21 การบริการของการโฆษณาภายในโปรเจคเกม



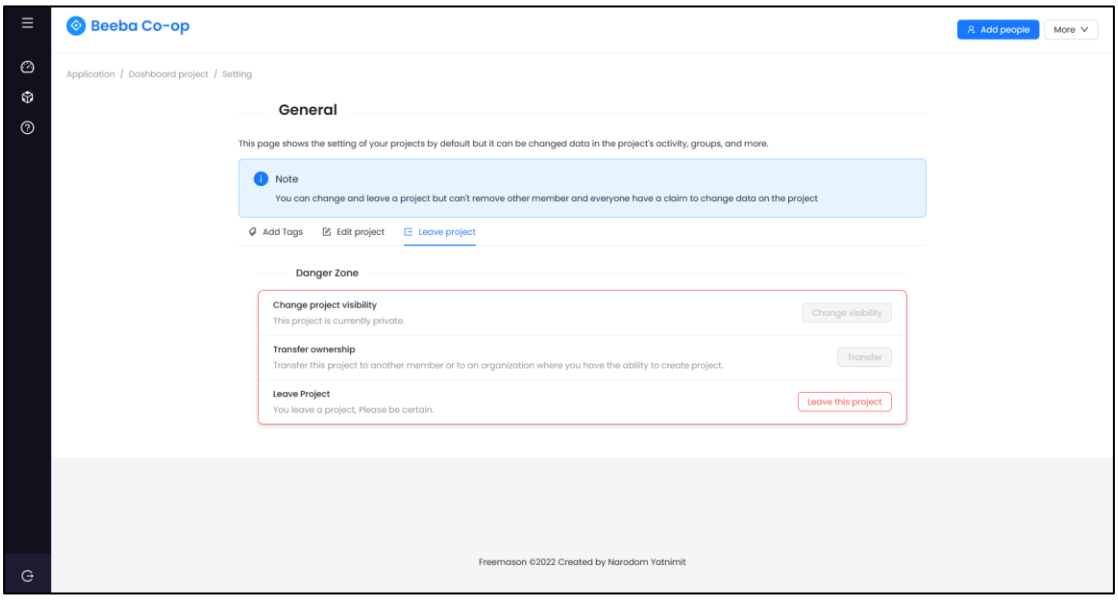
ภาพที่ 5-22 การแสดงรหัสการให้ผู้ใช้งานอื่นเข้าร่วมโปรเจคเกม



ภาพที่ 5-23 การตั้งค่าเพิ่มแท็กของโปรเจคเกม

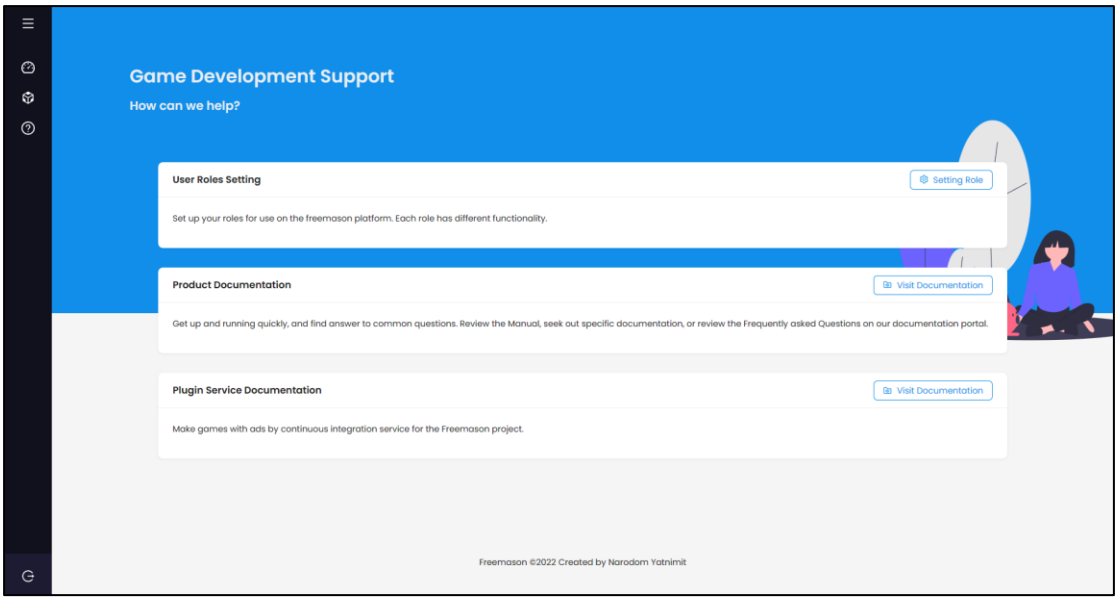


ภาพที่ 5-24 การตั้งค่าเปลี่ยนชื่อหรือรายละเอียดของโปรเจคเกม

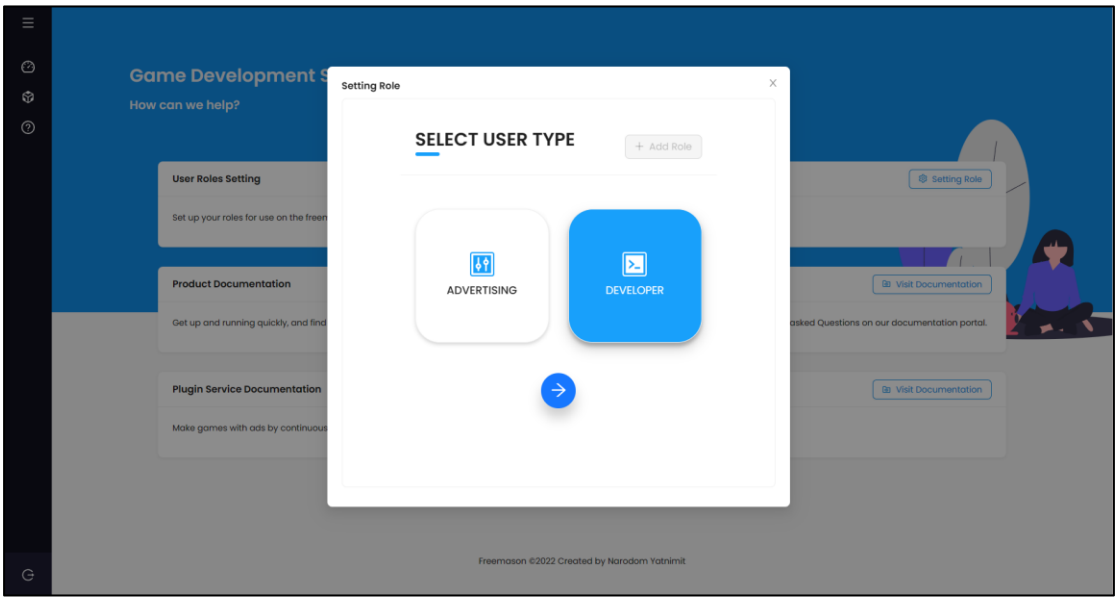


ภาพที่ 5-25 การตั้งค่าการออกจากโปรเจคเกม

5.2.1.8 หน้าแสดงการบริการและการช่วยเหลือ โดยมีฟังก์ชันเปลี่ยนบทบาทของผู้ใช้งานและเอกสารการใช้งาน

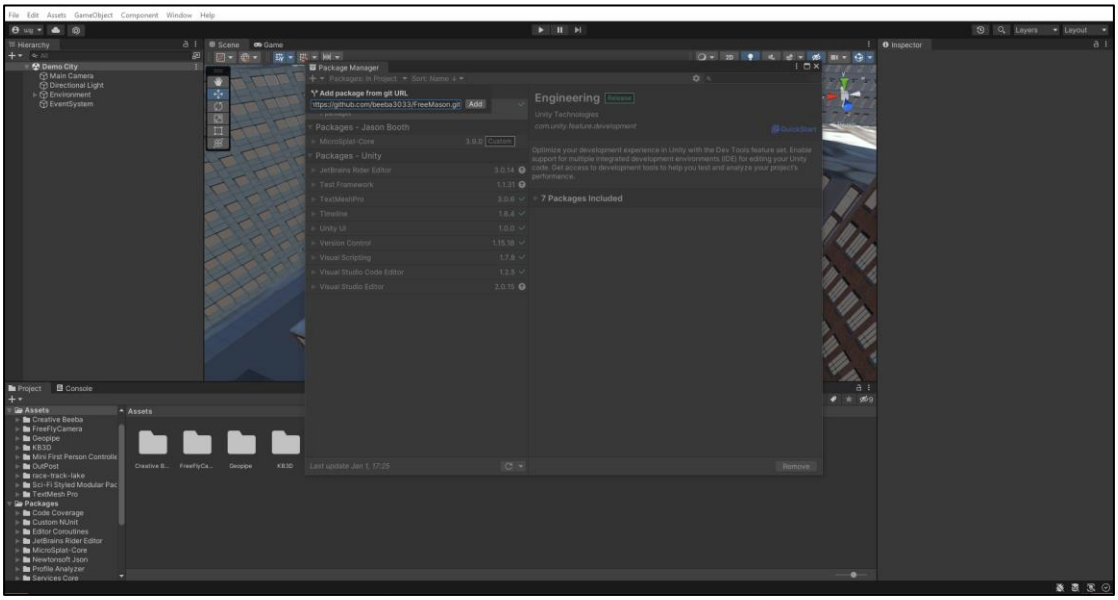


ภาพที่ 5-26 หน้าการแสดงของการบริการและช่วยเหลือ

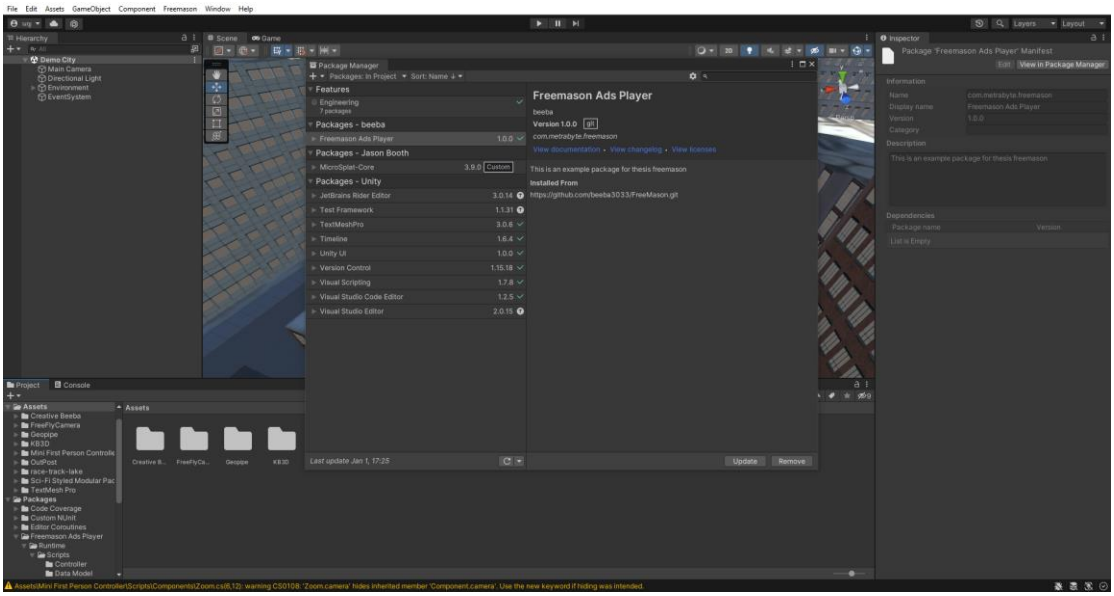


ภาพที่ 5-27 หน้าการตั้งค่าการเปลี่ยนแปลงบทบาทของผู้ใช้งาน

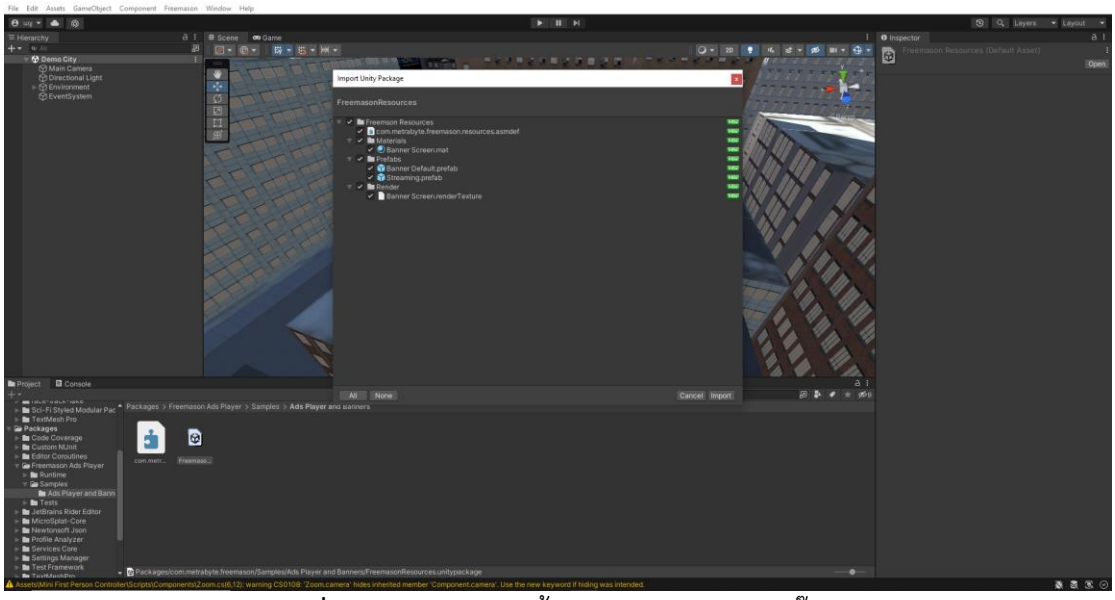
5.2.1.9 การติดตั้งและการใช้งานปลั๊กอินภายใน Unity game engine ภาพที่ 5-28 ถึง 5-38 การติดตั้งของโครงงานจะใช้ลิ้งค์ของ Git ในการติดตั้งปลั๊กอิน



ภาพที่ 5-28 หน้าการติดตั้งปลั๊กอินบน package manager ของ Unity game engine



ภาพที่ 5-29 หน้าการติดตั้งปลั๊กอินบน package manager ของ Unity game engine



ภาพที่ 5-30 หน้าการติดตั้งรับวัตถุตัวอย่างจากปลั๊กอิน



ภาพที่ 5-31 โฟลเดอร์เมื่อติดตั้งวัตถุตัวอย่างจากปลั๊กอิน



ภาพที่ 5-32 โฟลเดอร์เมื่อติดตั้งวัตถุตัวอย่างจากปลั๊กอิน



ภาพที่ 5-33 โฟลเดอร์เมื่อติดตั้งวัตถุตัวอย่างจากปลั๊กอิน



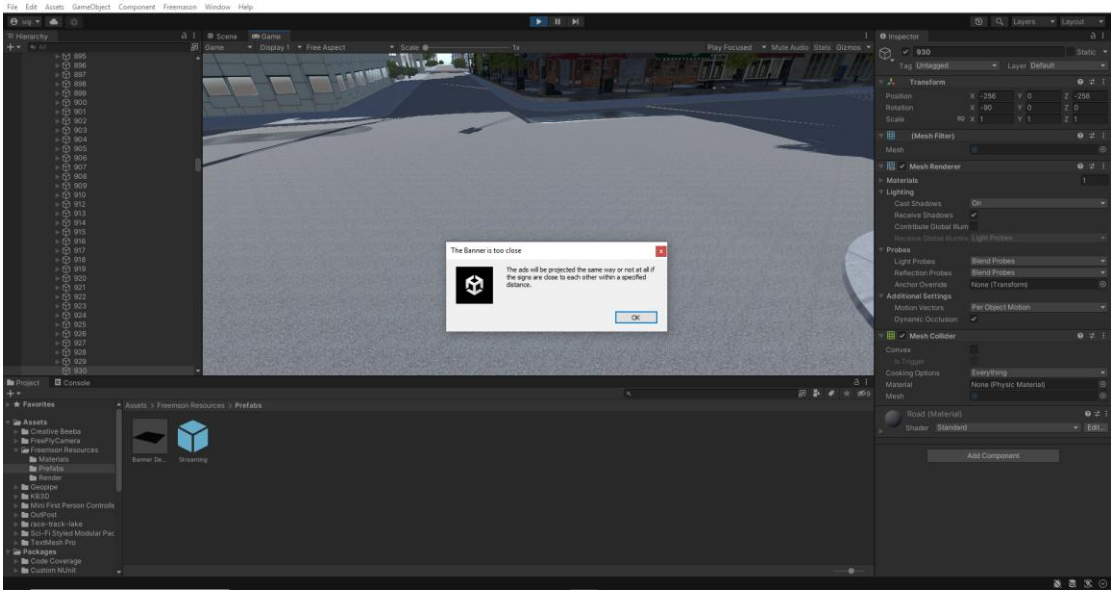
ภาพที่ 5-34 การเชื่อมต่อปลั๊กอินและโปรเจคเกมภายใน Editor ของ Unity game engine



ภาพที่ 5-35 การเชื่อมต่อปลั๊กอินและโปรเจคเกมภายใน Editor ของ Unity game engine



ภาพที่ 5-36 การวางวัตถุภายในเกมที่พัฒนา



ภาพที่ 5-37 การแจ้งเตือนเมื่อผู้ใช้งานวางวัตถุในระยะใกล้ตามเงื่อนไขที่กำหนด



ภาพที่ 5-38 การแสดงผลโฆษณาภายในเกม

บทที่ 6
บทสรุปและแนวทางการพัฒนาต่อ

6.1 สรุปผลการดำเนินงาน

- ได้ปลั๊กอินที่ทำให้การโฆษณาแบบแฝงภายในเกมเกิดขึ้นได้
- ได้ปลั๊กอินสำหรับผู้พัฒนาเกมใช้งานได้ง่าย
- ได้เว็บแอปพลิเคชันการแสดงผลรายละเอียดข้อมูลของปลั๊กอินสำหรับผู้ใช้งานผู้พัฒนาเกม
- ได้เว็บแอปพลิเคชันจัดการโฆษณาสำหรับผู้ใช้งานประเภทนักโฆษณา

6.2 ปัญหาที่พบระหว่างการดำเนินงาน

- การสร้างวัตถุการแสดงผลและการนำวีดีโอมาแสดงผลยังไม่ข้อผิดพลาด กรณีวัตถุมีความซับซ้อนการแสดงผลวีดีโอยังสามารถแสดงผลได้ ซึ่งการแสดงผลวีดีโอยังคงสามารถแสดงผลได้ แต่การแสดงผลอาจบกพร่องได้
- การตรวจสอบการมองเห็นวัตถุของกล้อง วิดีโอ และวัตถุ เนื่องจากการรับรู้วีดีโอบนวัตถุจำเป็นต้องมีการมองเห็นด้วยกล้อง ดังนั้นการกำหนดว่ากล้องเห็นวัตถุและวีดีโอรูปแบบไหนที่สามารถบอกการรับรู้วีดีโอได้
- ความถูกต้องการนับเวลาของการมองเห็นและวัตถุ จำเป็นต้องจับเวลาให้ถูกต้องเมื่อผู้ใช้งานภายในเกมไม่ได้มองเห็นโฆษณาจึงควรหยุดการจับเวลา แต่การนับเวลาหลายกรณีอาจเกิดข้อบกพร่องได้
- การกำเนิดของวัฏจักรของเกม การเรียกข้อมูลหรือวัตถุ จำเป็นต้องเป็นไปตามวัฏจักร ดังนั้นการติดตั้งหรือสร้างวัตถุแฝงจึงอาจขัดต่อการพัฒนาเกมของผู้พัฒนาเกมได้
- ขนาดของวัตถุต่อขนาดเกมที่พัฒนา ขนาดวัตถุของตัวอย่างภายในปลั๊กอินอาจจะมีขนาดมากน้อยกว่าวัตถุภายในที่ผู้พัฒนาเกมออกแบบ
- ประสิทธิภาพการทำงานของปลั๊กอิน การรับส่งหรือลำดับการทำงานต่าง ๆ อาจเกิดการใช้งานทรัพยากรเกิดจำเป็นได้
- การบรรจุภัณฑ์ปลั๊กอินสำหรับการใช้งาน จำเป็นต้องมีการออกแบบให้ผู้ใช้งานสามารถติดตั้งได้ง่าย

6.3 แนวทางการแก้ไขปัญหา

- การแก้ไขการแสดงผลวัตถุจำเป็นต้องมีการกำหนดรูปแบบของวัตถุ ดังนั้นการมีตัวอย่างวัตถุในการเป็นต้นแบบการแสดงผลทำให้การแสดงผลโฆษณาได้อย่างที่ควรเป็น
- นำข้อมูลกล้องที่อยู่ภายในฉากรับเข้าสู่ปลั๊กอิน เนื่องจากการนำข้อมูลกล้องมาตรวจสอบกับข้อมูลวัตถุที่อยู่ภายในปลั๊กอิน และสามารถนำข้อมูลกล้องมาเสริมด้านการมองเห็นด้วยองศา หรือขนาดของพื้นที่การมองเห็นแบบ 2 มิติ ก็สามารถทำได้
- ความถูกต้องการนับเวลาวิธีการแก้ไขคือ การมีเครื่องจับเวลารายวัตถุและเมื่อวัตถุถูกมองเห็นก็สั่งใช้งานเครื่องนับเวลาและเมื่อไม่มีการมองเห็นจึงหยุดการใช้งานเครื่องนับเวลา
- การกำหนดวัฏจักรของเกมจำเป็นต้องศึกษาการทำงาน ซึ่งการทำงานภายในระบบของปลั๊กอินจำเป็นควรติดตั้งค่าเริ่มต้นภายในฉากเพื่อรอการแสดงผล
- เพิ่มช่องทางการติดตั้งบรรจุภัณฑ์ปลั๊กอิน และการติดตั้งก่อนจะรับปลั๊กอินผ่านช่องทาง

6.4 แนวทางในการพัฒนาต่อ

- สามารถเพิ่มฟังก์ชันการทำงานต่าง ๆ บนเว็บแอปพลิเคชันได้ทั้งผู้ใช้งานผู้พัฒนาเกมและนักโฆษณา
- สามารถเพิ่มวิธีการมองเห็นโฆษณาต่าง ๆ ได้ อาจจะใช้หลักคณิตศาสตร์ในการคำนวณพื้นที่ของวีดีโอวัตถุบนกล้องขณะมองเห็นในรูปแบบ 2 มิติ
- เพิ่มระบบการนับเวลาที่กำหนดได้ ซึ่งตัวอย่างการนับเวลาต่าง ๆ เพื่อประสิทธิภาพการมองเห็นโฆษณา เช่น การเริ่มนับเวลาทุกครั้งที่มองเห็นโฆษณานานเกิน 5 วินาทีขึ้นไปเป็นต้น
- การออกแบบตัวอย่างวัตถุไม่จำเป็นต้องเป็นป้าย วัตถุในการแสดงผลสามารถเป็นรูปแบบตามที่ออกแบบได้ แต่อาจจะไม่สนับสนุนต่อวีดีโอที่โฆษณาได้ การออกแบบตัวอย่างวัตถุจึงจำเป็นต่อการใช้งานบนปลั๊กอิน
- ประสิทธิภาพการทำงานของระบบปลั๊กอินสามารถลดการใช้ทรัพยากรที่ไม่จำเป็นได้
- บรรจุภัณฑ์ปลั๊กอินสามารถออกแบบเพื่อให้ผู้ใช้งานเข้าใจได้ง่ายและง่ายต่อการติดตั้ง
  - นำ Web Application System Admin เชื่อมต่อเข้ากับ SSO Login

บรรณานุกรม

[1] [สืบค้นวันที่ 26 ธันวาคม 2564][ออนไลน์] Unity (game engine)

ที่มา https://th.wikipedia.org/wiki/ยูนิตี_(เกมเอนจิน)